ANALYSIS OF A .NET LOADER TAGGED AS AGENT TESLA

MalwareBazaar sample rPEq.exe (SHA-256 412bad1c…4775), family unconfirmed

Adetola Samuel O.

github.com/Cyber-Sam01

October 2026

TABLE OF CONTENTS

EXECUTIVE SUMMARY **3**
INTRODUCTION **4**

SECTION 1: LAB SETUP **5**
1.1 Why This Environment 6
1.2 Virtualization Platform: VirtualBox 6
1.3 Analysis VM: Windows 10 7
1.4 Network Analysis VM: REMnux 8
1.5 Isolating the Environment: Disabling Windows Defenses 8
1.6 Resolving Display Resolution Issues 11
1.7 Installing FLARE VM 14
1.8 Troubleshooting the Installation 16
1.9 Malware Sample Installation 20
1.10 Network Isolation Configuration 23
1.11 Final Checkpoint 32

SECTION 2: STATIC ANALYSIS REPORT **33**
2.1 Sample Selection and Safe Handling 34
2.2 Establishing Identity: Hashes and File Type 36
2.3 Existing Intelligence 39
2.4 PE Structure and Entropy 41
2.5 Strings 45
2.6 Capability Triage with capa 50
2.7 Decompilation with dnSpy 54
2.8 Findings: MITRE ATT&CK Mapping 57
2.9 Draft Detection 59
2.10 IOC TABLE 61

SECTION 3: DYNAMIC ANALYSIS REPORT **62**
3.1 Overview and Goals 63
3.2 Setup and Baseline 63
3.3 Execution 64
3.4 Process Activity 65
3.5 Network Activity 67
3.6 Registry Activity 68
3.7 File Activity 70
3.8 Findings and Limits 73
3.9 Effect on the Earlier Analysis 74
3.10 Lessons Learned 74

CONCLUSION **75**

EXECUTIVE SUMMARY

This report analyzes a 32-bit .NET executable, rPEq.exe (SHA-256 412bad1c…4775), that MalwareBazaar tagged as Agent Tesla. I built an isolated lab with a Windows 10 FLARE VM and REMnux, then analyzed the sample statically and dynamically.

The sample is a loader, not a finished stealer. It poses as a Windows Forms “PearlFarm” application, and about 92% of the file is image resources that look like noise. It contains code that appears to read bytes from a bitmap’s pixels, and a hex-encoded string that decodes to the name of the largest image, eQTo. When run, it started PowerShell to add a Windows Defender exclusion for its own file, and a second copy of the file started the same second.

Still unconfirmed: the malware family (vendors disagreed between Noon, AgentTesla and Formbook), the hidden second stage, network behavior, and persistence. Deliverables are an ATT&CK Navigator layer (v20), draft YARA rules, and the lab method.

INTRODUCTION

Cyberattacks are on the rise, and their impact is rising with them. That impact extends well beyond web application vulnerabilities like those in the OWASP Top 10. Malware attacks are, in many ways, more dangerous, capable of affecting multiple systems at once and causing severe consequences such as operational downtime, revenue loss, reputational damage, data breaches, or even the permanent loss of a device.

A few examples illustrate just how disastrous these attacks can be:

The WannaCry Ransomware Attack — This attack targeted Microsoft Windows systems by encrypting data and demanding ransom payments in bitcoin. Over 200,000 computers were affected worldwide. A security researcher, Marcus Hutchins, eventually discovered a kill switch that halted its spread, but by then the damage was done, with an estimated $4 billion in losses globally.

The 2021 Colonial Pipeline Ransomware Attack — Attackers gained initial access through a compromised VPN account and deployed DarkSide ransomware on Colonial's IT network. The operational technology (OT) systems were never directly compromised, but the company proactively shut down pipeline operations as a precaution. Even with segmentation holding, the loss of IT systems (billing, scheduling) was enough to halt physical fuel delivery. It's a clear demonstration that IT/OT segmentation reduces operational risk, but doesn't eliminate it.

These are just two examples spanning IT and OT industries, but there are many more like them. This project won't replicate ransomware-scale impact. What it does is walk through the same analytical process security teams use to understand and detect threats like these. In this lab activity, I perform a basic analysis of a malware sample, working to uncover its intent and map its tactics to known detection techniques.

SECTION 1:

LAB SETUP

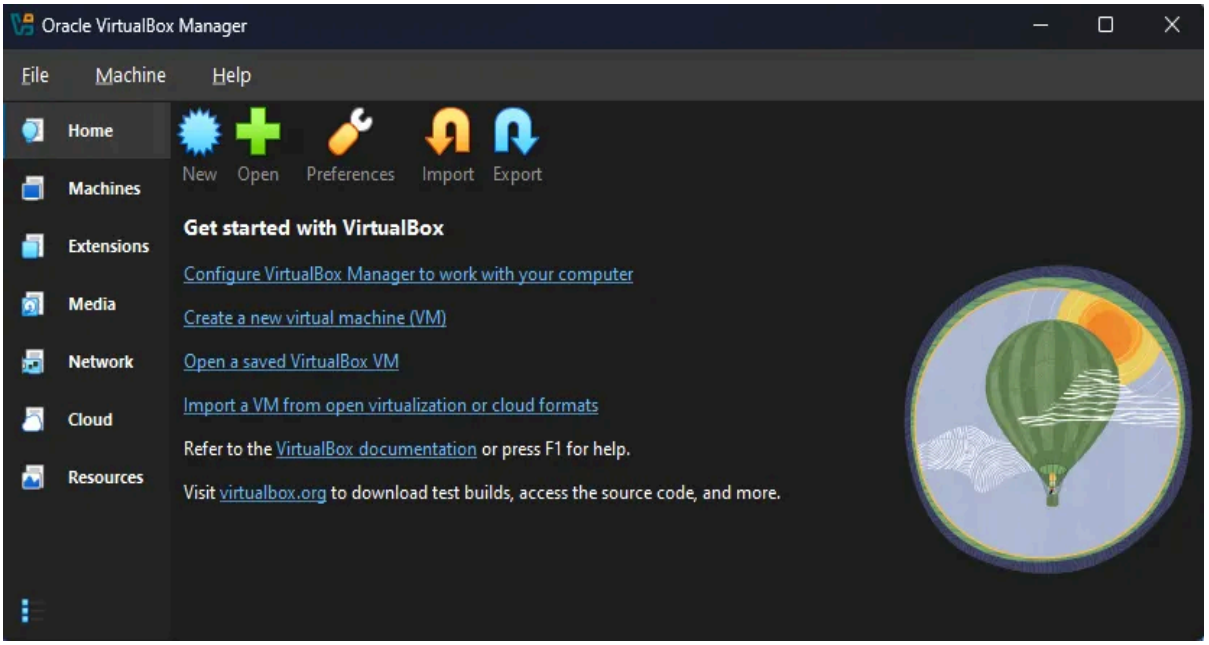
1.1 Why This Environment

Before touching the sample itself, I needed an environment where I could detonate malware safely, observe its behavior, and always be able to roll back to a clean state. That drove every tool choice below:

- Oracle VirtualBox as the virtualization platform — its snapshot feature lets me freeze a known-good state and revert to it after every test, rather than rebuilding a VM from scratch each time.
- Windows 10 as the victim/analysis OS — the overwhelming majority of malware, including the two case studies above, targets Windows, so this is the environment where the sample's real behavior will surface.
- FLARE VM, layered onto the Windows VM — a free, scripted toolkit from Mandiant that installs the full suite of static and dynamic analysis tools (disassemblers, debuggers, PE inspectors, and more) I'd otherwise have to source and configure one by one.
- REMnux, running alongside as a second VM — a Linux distribution built for the network side of analysis, so I can observe and fake out any command-and-control or network traffic the sample tries to generate.

Together, the Windows/FLARE VM plays the role of the infected host, while REMnux plays the role of the network the malware thinks it's talking to. This mirrors how real malware analysts isolate a sample from the internet while still letting it "believe" it has network access.

1.2 Virtualization Platform: VirtualBox



VirtualBox download page

1.3 Analysis VM: Windows 10

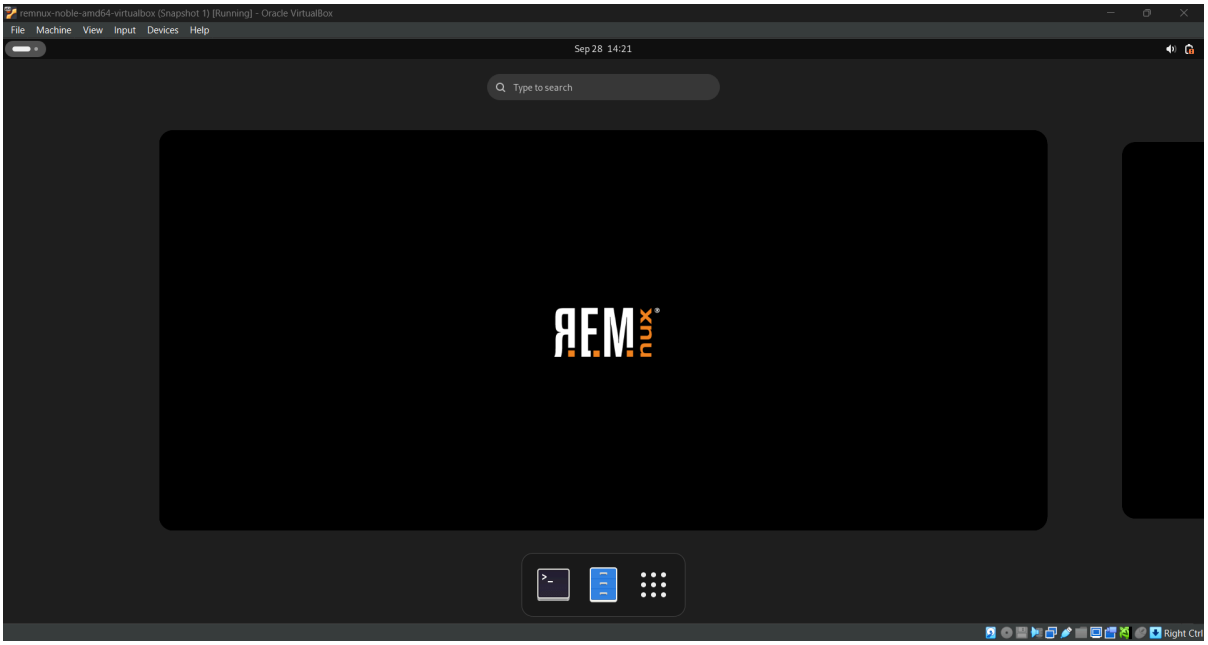
Once the VM was installed, and before making any changes to it, I took a baseline snapshot. This is the single most important habit in this whole setup: it gives me a clean, unmodified checkpoint to return to if anything goes wrong further down the line.



Baseline snapshot of the fresh Windows 10 VM

1.4 Network Analysis VM: REMnux

For REMnux, I headed to its official site, downloaded the installation image, and imported it into VirtualBox.



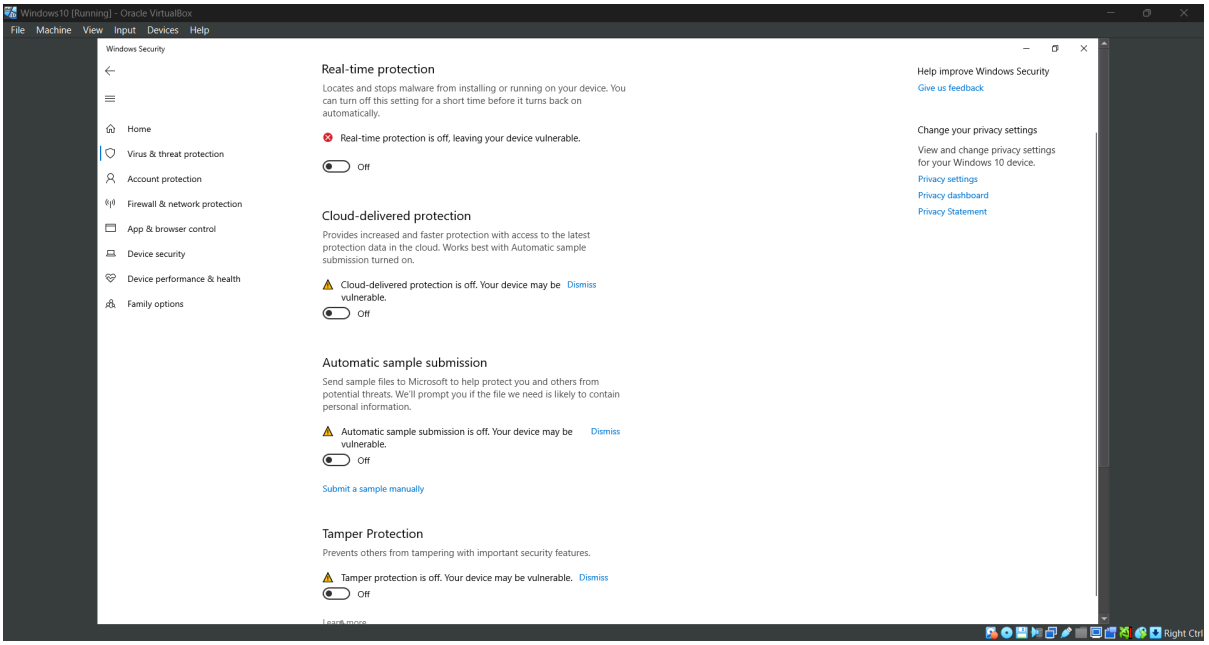
REMnux booting for the first time after import

1.5 Isolating the Environment: Disabling Windows Defenses

Before installing any analysis tooling, I disabled Windows' built-in defenses on the analysis VM. The reasoning here matters: Windows Defender and related protections are designed to catch and quarantine exactly the kind of tools and behavior I need to observe. Left on, they'd flag the FLARE VM toolset as malicious, block the sample from running as intended, or auto-update in the middle of an analysis session. So the goal wasn't to make the VM insecure, it was to remove anything that would interfere with observing the sample's true behavior.

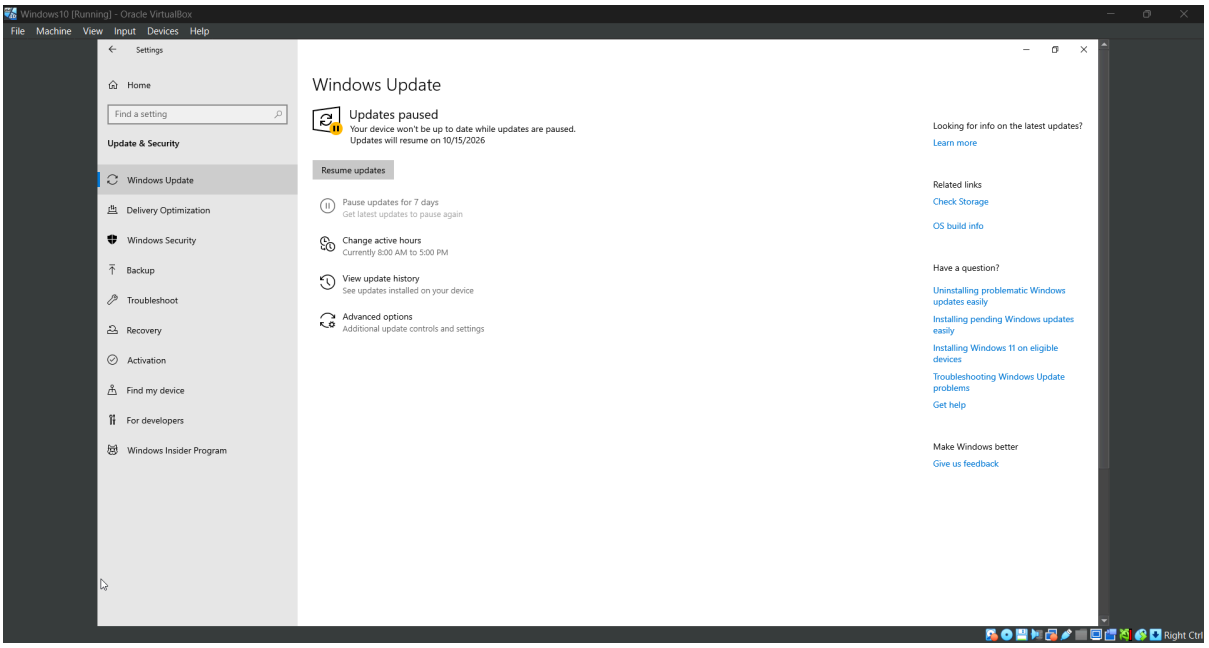
I worked through this in layers, starting with the Windows Security UI:

- Real-Time Protection
- Cloud-delivered Protection
- Automatic Sample Submission
- Tamper Protection



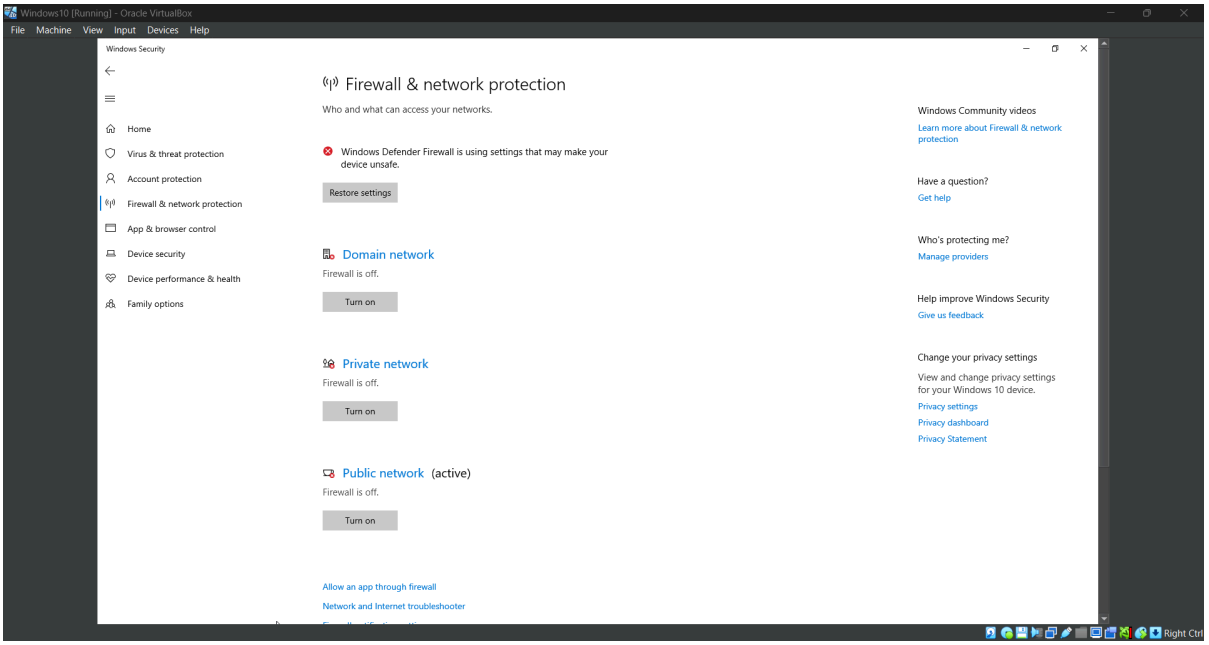
Disabling Defender's real-time and cloud-delivered protections

I also turned off Windows Update, partly to stop it from silently reinstalling protections I'd just disabled, and partly to keep the OS state stable and predictable across analysis sessions.

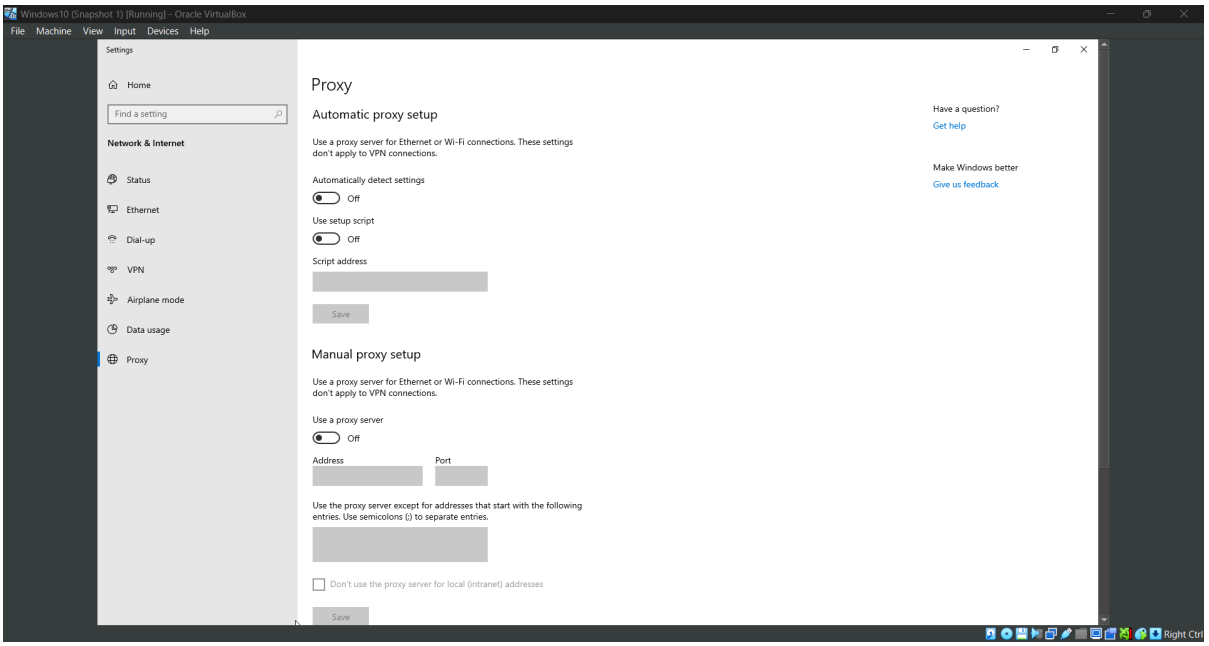


Disabling Windows Update

Next, the firewall and network protections. Malware often needs to reach out to a command-and-control server to reveal its full behavior, so an active firewall on the analysis VM would suppress exactly the traffic I want REMnux to observe.

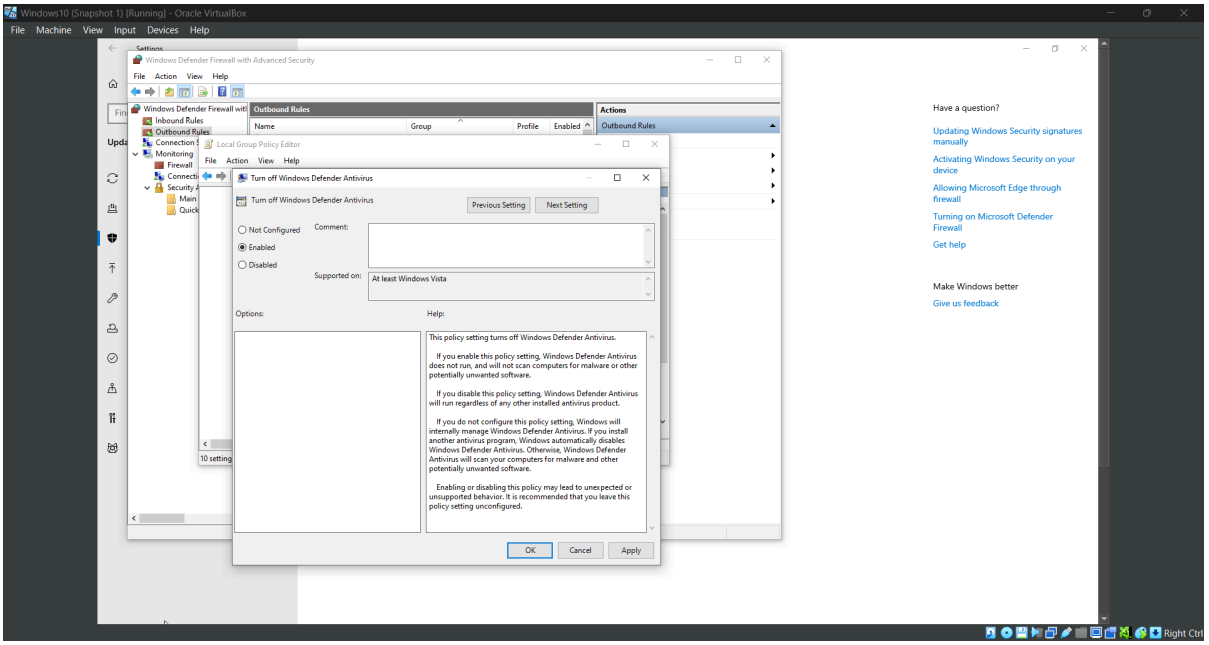


Disabling firewall and network protection



Disabling the proxy

The Security app toggles alone turned out not to be enough — FLARE VM's installer specifically checks Defender's status at the Group Policy level, not just the UI level. So I went a layer deeper via Group Policy Editor (gpedit.msc), navigating to **Computer Configuration → Administrative Templates → Windows Components → Windows Defender Antivirus**, and enabling **"Turn off Windows Defender Antivirus"** directly.



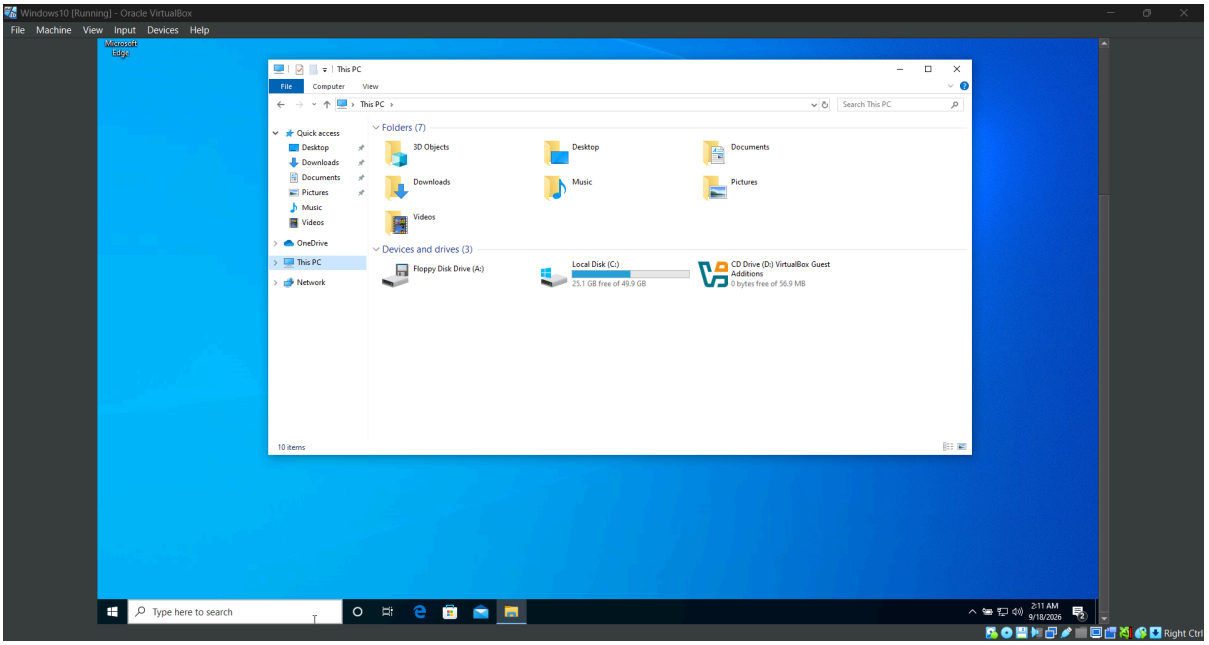
Disabling Windows Defender Antivirus via Group Policy

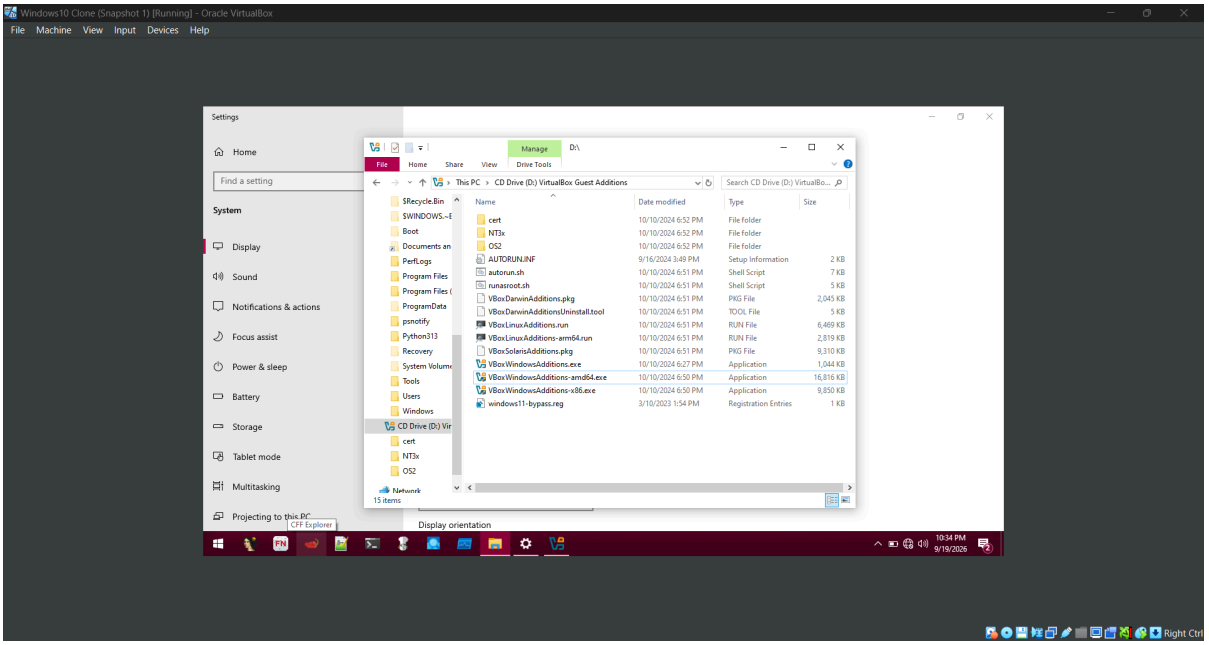
With defenses fully down, I took another snapshot; this one marking the "unprotected, but not yet infected" baseline.

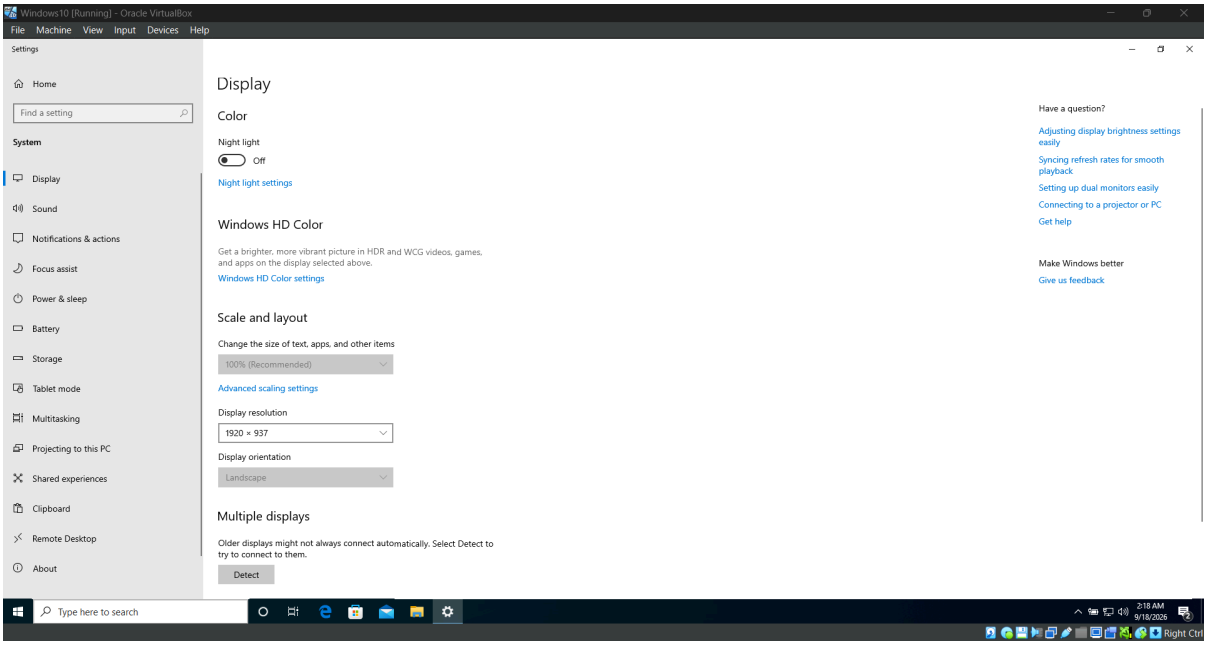
1.6 Resolving Display Resolution Issues

At this point I ran into a practical annoyance: the VM's display didn't scale properly, making it hard to work with. The fix was to install VirtualBox's Guest Additions, which give the VM proper awareness of the host window size:

- Right-click a blank area of the desktop and open **Display Settings** to check the current resolution.
- From the VirtualBox VM window, go to **View** → **Install Guest Additions image**, then run it from This PC.
- Restart the VM — the screen should resize automatically, or the new resolution options should now appear under Display Settings.

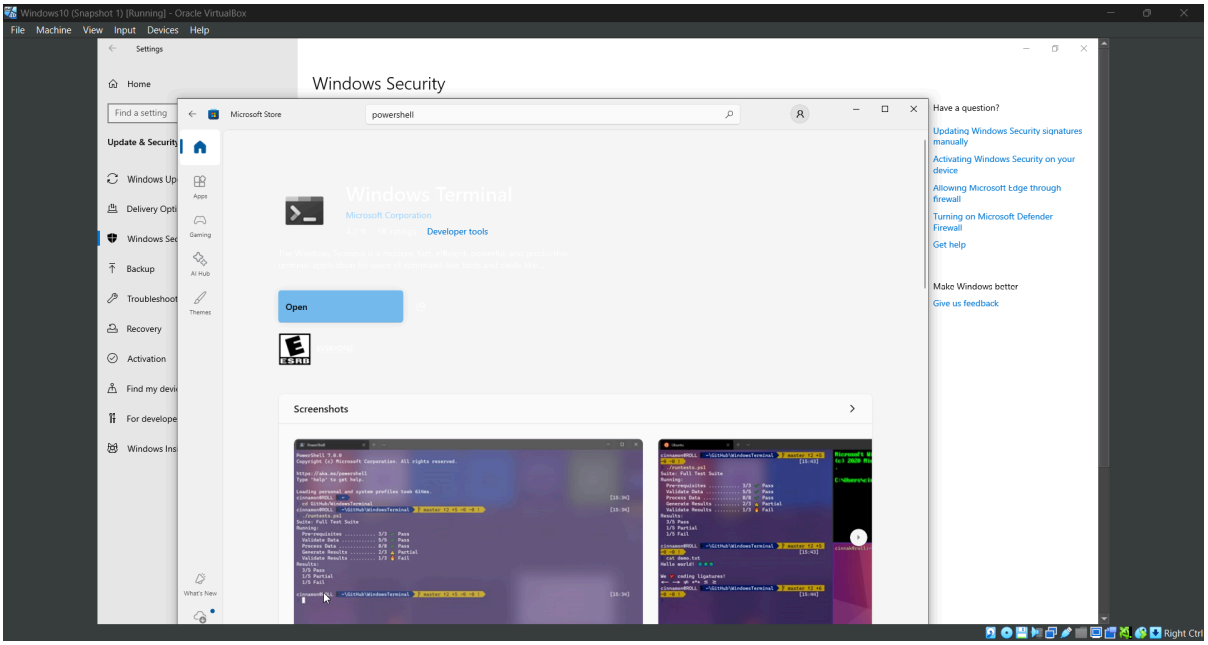

Windows10 [Running] - Oracle VirtualBox
File Machine View Input Devices Help
This PC
Folders (7)
3D Objects
Desktop
Documents
Downloads
Music
Pictures
Videos
Devices and drives (3)
Floppy Disk Drive (A:)
Local Disk (C:)
CD Drive (D:) VirtualBox Guest Additions
Type here to search
Right Ctrl

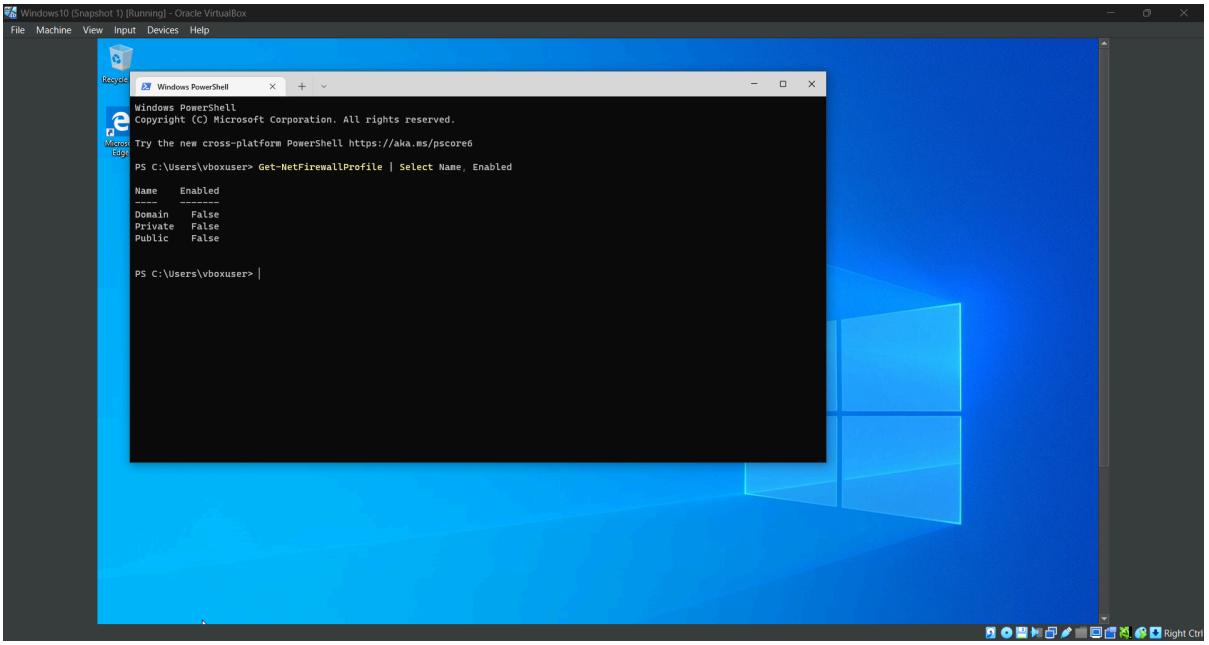

Windows10 Clone (Snapshot 1) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Settings
Home
System
Display
Sound
Notifications & actions
Focus assist
Power & sleep
Battery
Storage
Tablet mode
Multitasking
Projecting to this PC
Display orientation
Right Ctrl



Working through the Guest Additions install and resolution fix

I also installed a terminal app from the Microsoft Store at this stage, purely to make the command-line work ahead more comfortable.





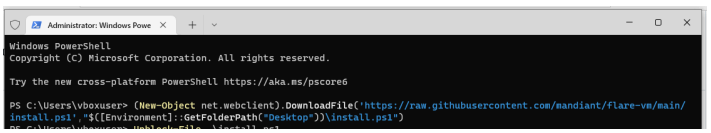
Installing Windows Terminal

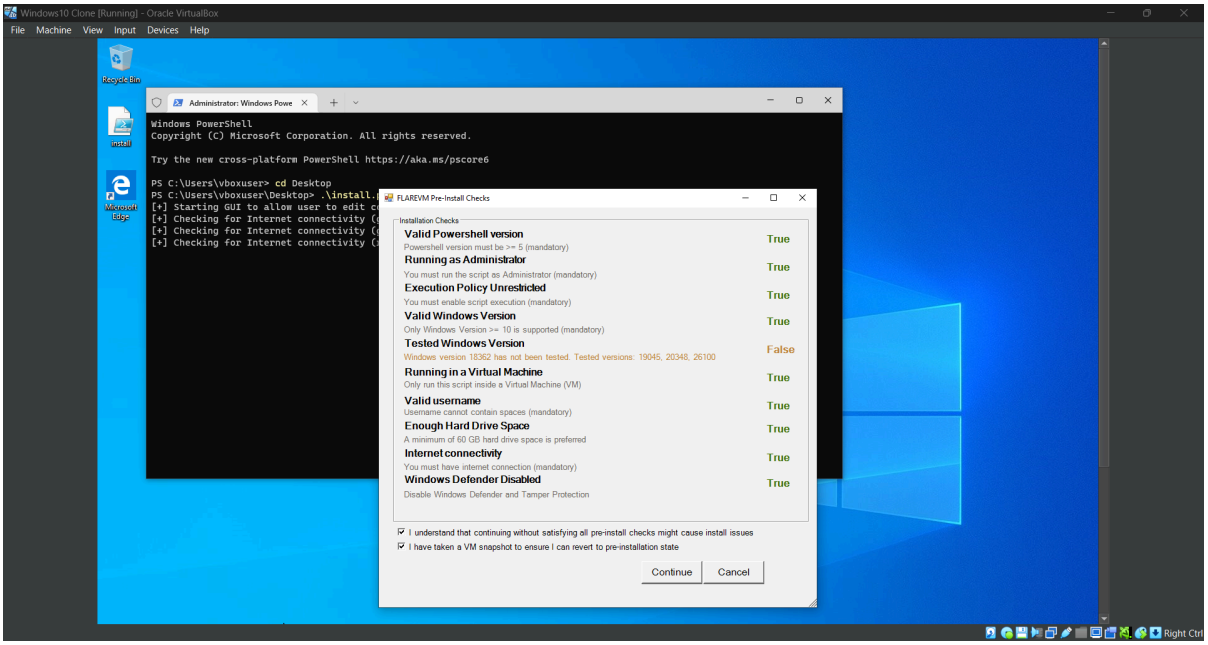
1.7 Installing FLARE VM

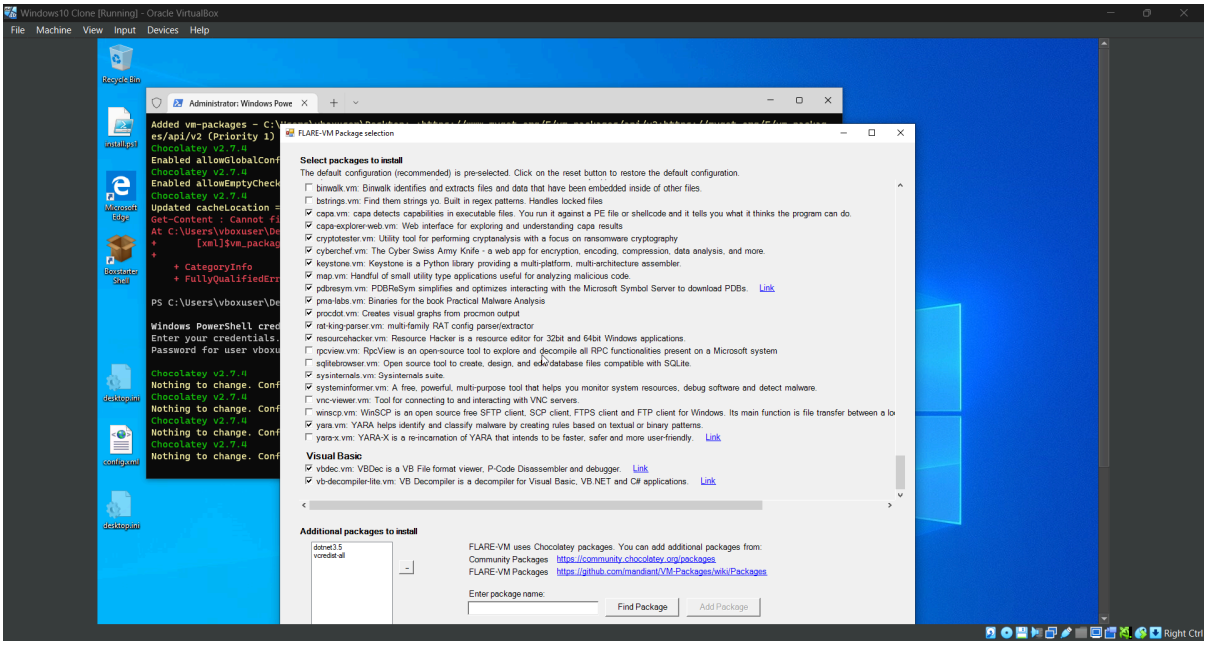
With the environment isolated and usable, I moved on to FLARE VM itself. Rather than installing dozens of analysis tools individually, FLARE VM automates the whole toolset install through a single PowerShell script — the practical choice given how many tools are involved:

- Open a PowerShell prompt as Administrator.
- Download the installer script to the Desktop: (**New-Object net.webclient).DownloadFile('https://raw.githubusercontent.com/mandiant/flare-vm/main/install.ps1', "$([Environment]::GetFolderPath('Desktop'))\install.ps1"**)
- Unblock the script, since it was downloaded from the internet: **cd Desktop**, then **Unblock-File .\install.ps1**
- Enable script execution: **Set-ExecutionPolicy Unrestricted -Scope Process -Force**
- Run the installer: **.\install.ps1**

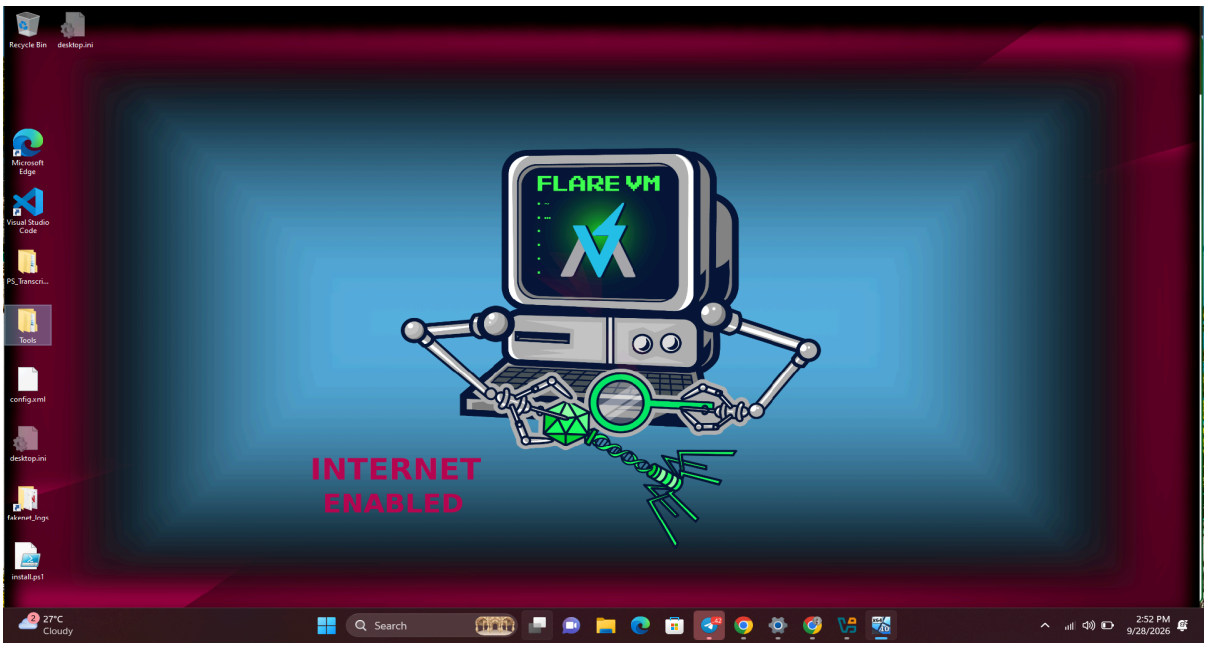
This step took considerably longer than I expected. The toolset is large, and depending on connection speed it can realistically take days rather than hours to complete.







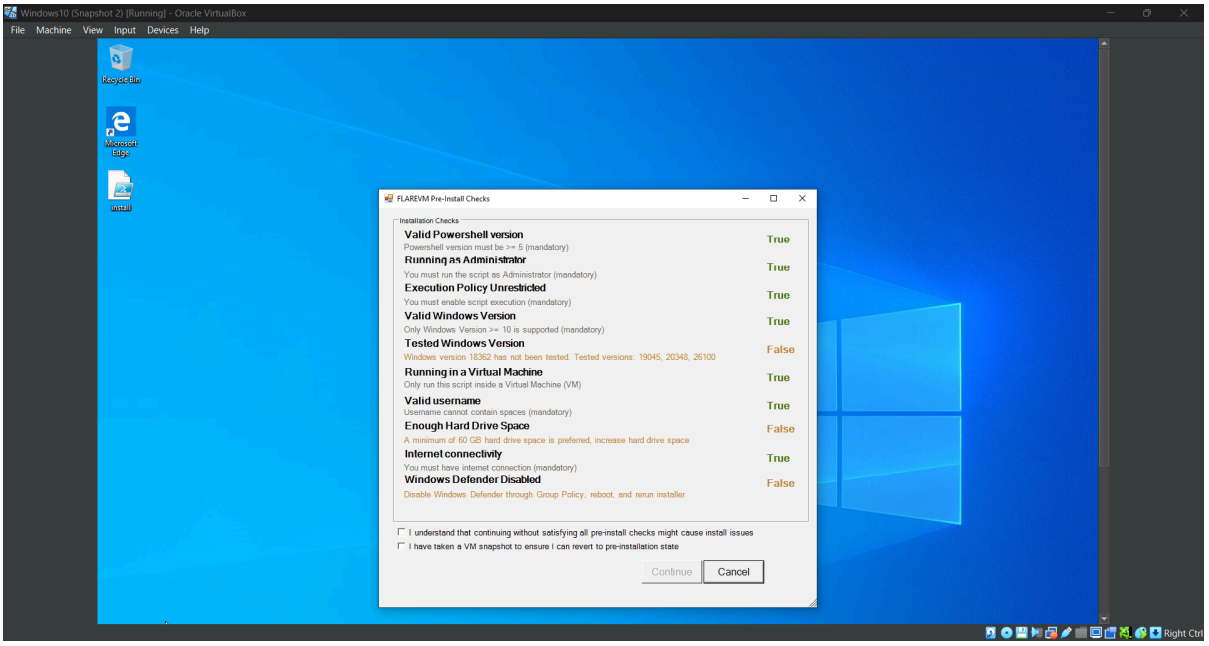
FLARE VM installation in progress



A completed FLARE VM installation

1.8 Troubleshooting the Installation

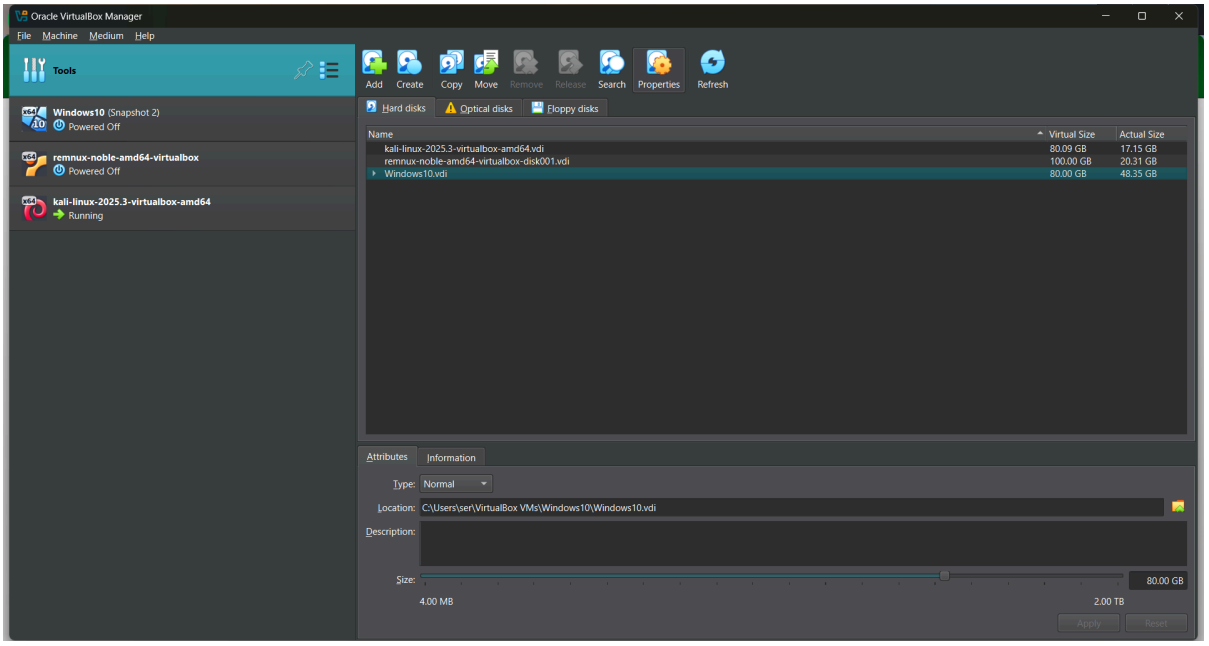
Issue 1 — Defender still flagged as active. This is the exact failure most people hit, and it came from the gap described in Section 1.5: disabling Defender through the Security UI isn't sufficient, because the installer checks the Group Policy state specifically. The fix was the Group Policy change already covered above, After applying it, I rebooted and reran install.ps1 to confirm the "Windows Defender Disabled" check now reads True.



The installer failing its Defender check

Issue 2 — Insufficient disk space. FLARE VM requires a minimum of 60 GB. I ran into this despite initially allocating 50 GB of storage, the base virtual disk still needed resizing. In VirtualBox, resizing is done through **File → Tools → Virtual Media Manager**, but it won't apply while active snapshots exist on that disk.

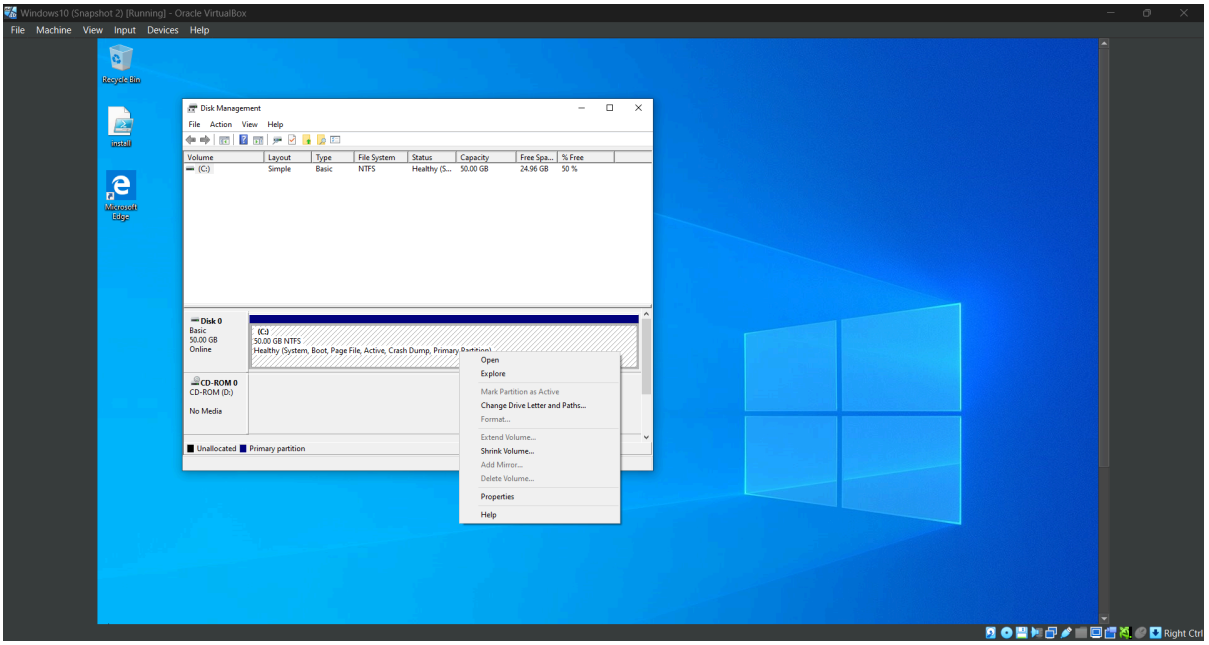
This left two options: **merge** (delete the existing snapshots to flatten everything into the base drive, then resize) or **clone** (create a full, snapshot-free copy of the current VM state, and resize that instead). I chose to clone rather than merge; it meant a bit more disk usage temporarily, but it preserved the original VM and its snapshot history untouched in case anything went wrong with the resize.

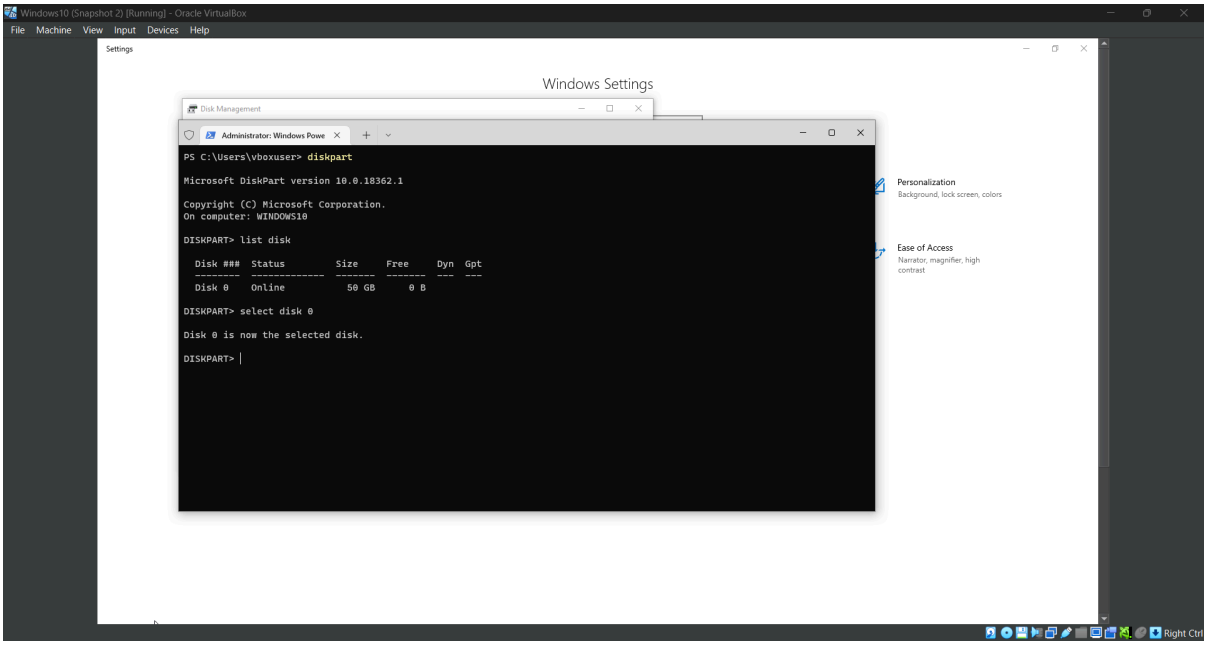


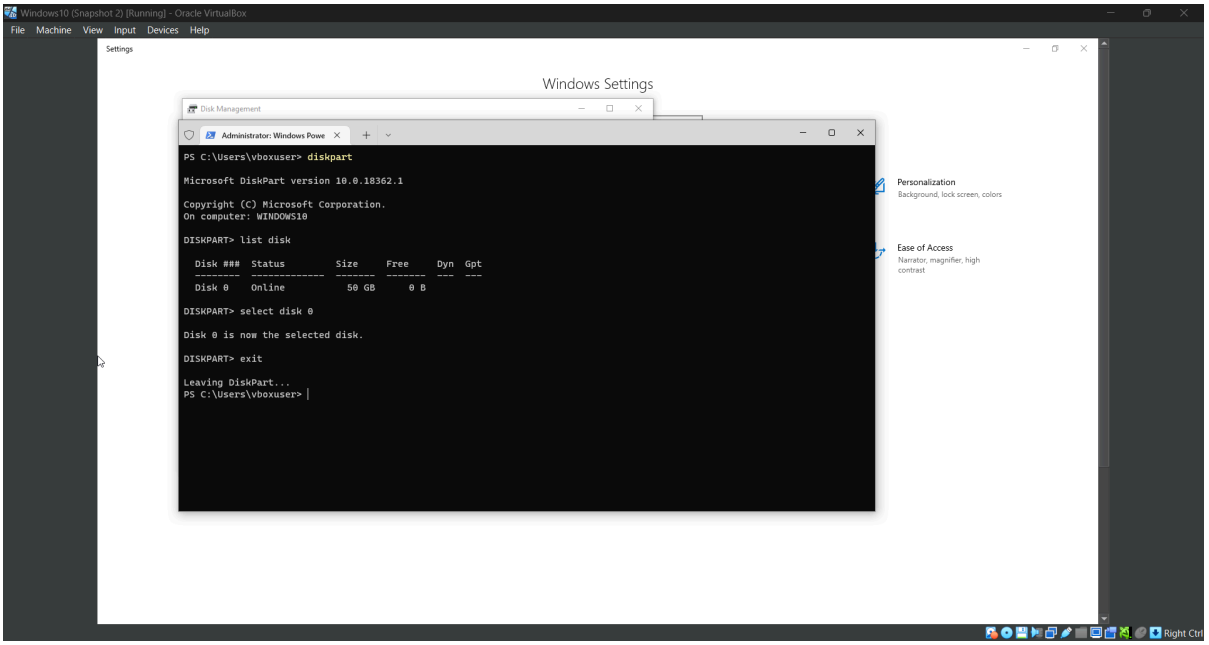
Resizing the virtual disk in Virtual Media Manager

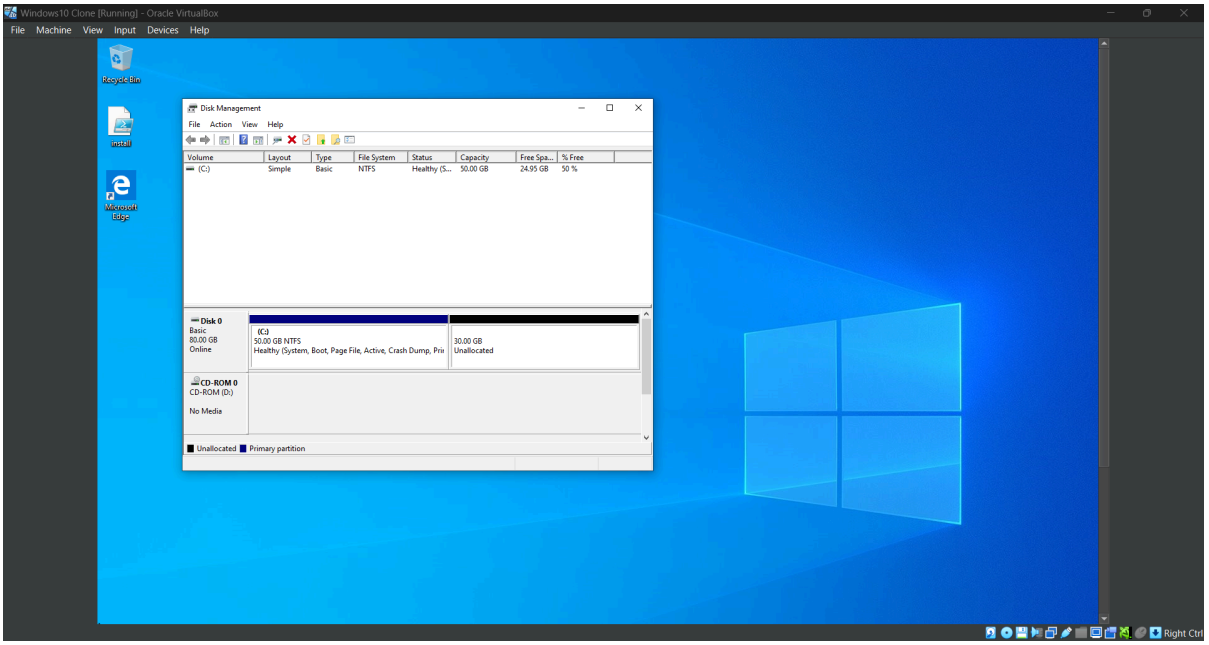
After resizing, the new space still needed to be allocated to the actual Windows volume:

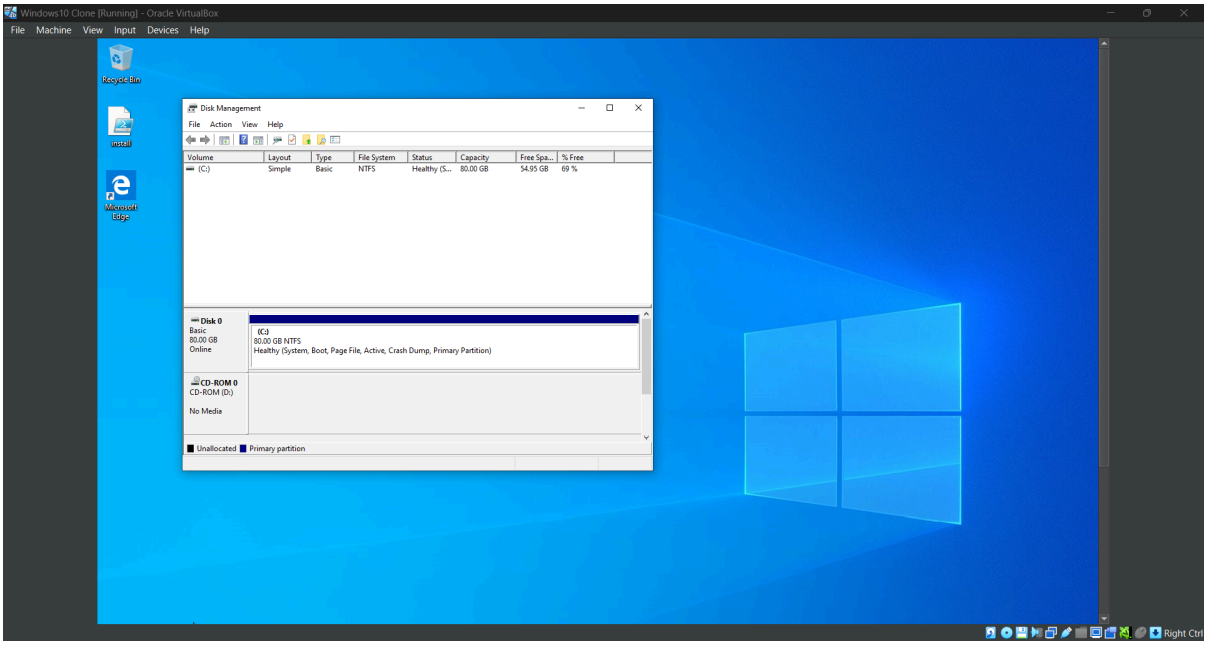
- Boot the cloned VM (leaving the original powered off).
- Open Disk Management (**Windows Key + X**).
- Right-click the C: drive, select **Extend Volume**, and absorb the newly unallocated space. Where a recovery partition blocked this, I used **Diskpart** to remove it first.


Windows10 (Snapshot 2) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Recycle Bin
install
Microsoft Edge
Disk Management
File Action View Help
Volume Layout Type File System Status Capacity Free Spa... % Free
(C:) Simple Basic NTFS Healthy (S... 50.00 GB 24.96 GB 50 %
Disk 0
Basic
50.00 GB
Online
(C:)
50.00 GB NTFS
Healthy (System, Boot, Page File, Active, Crash Dump, Primary Partition)
CD-ROM 0
CD-ROM (D:)
No Media
Unallocated
Primary partition
Open
Explore
Mark Partition as Active
Change Drive Letter and Paths...
Format...
Extend Volume...
Shrink Volume...
Add Mirror...
Delete Volume...
Properties
Help
Right Ctrl


Windows10 (Snapshot 2) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Settings
Windows Settings
Disk Management
Administrator: Windows Powe
PS C:\Users\vboxuser> diskpart
Microsoft DiskPart version 10.0.18362.1
Copyright (C) Microsoft Corporation.
On computer: WINDOWS10
DISKPART> list disk
Disk ### Status Size Free Dyn Gpt
Disk 0 Online 50 GB 0 B
DISKPART> select disk 0
Disk 0 is now the selected disk.
DISKPART>
Personalization
Background, lock screen, colors
Ease of Access
Narrator, magnifier, high contrast
Right Ctrl


Windows10 (Snapshot 2) [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Settings
Windows Settings
Disk Management
Administrator: Windows Powe
PS C:\Users\vboxuser> diskpart
Microsoft DiskPart version 10.0.18362.1
Copyright (C) Microsoft Corporation.
On computer: WINDOWS10
DISKPART> list disk
Disk ### Status Size Free Dyn Gpt
Disk 0 Online 50 GB 0 B
DISKPART> select disk 0
Disk 0 is now the selected disk.
DISKPART> exit
Leaving DiskPart...
PS C:\Users\vboxuser>
Personalization
Background, lock screen, colors
Ease of Access
Narrator, magnifier, high contrast
Right Ctrl


Windows10 Clone [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Recycle Bin
install
Microsoft Edge
Disk Management
File Action View Help
Volume Layout Type File System Status Capacity Free Spa... % Free
(C:) Simple Basic NTFS Healthy (S... 50.00 GB 24.95 GB 50 %
Disk 0
Basic
80.00 GB
Online
(C:)
50.00 GB NTFS
Healthy (System, Boot, Page File, Active, Crash Dump, Prii
30.00 GB
Unallocated
CD-ROM 0
CD-ROM (D:)
No Media
Unallocated Primary partition
Right Ctrl

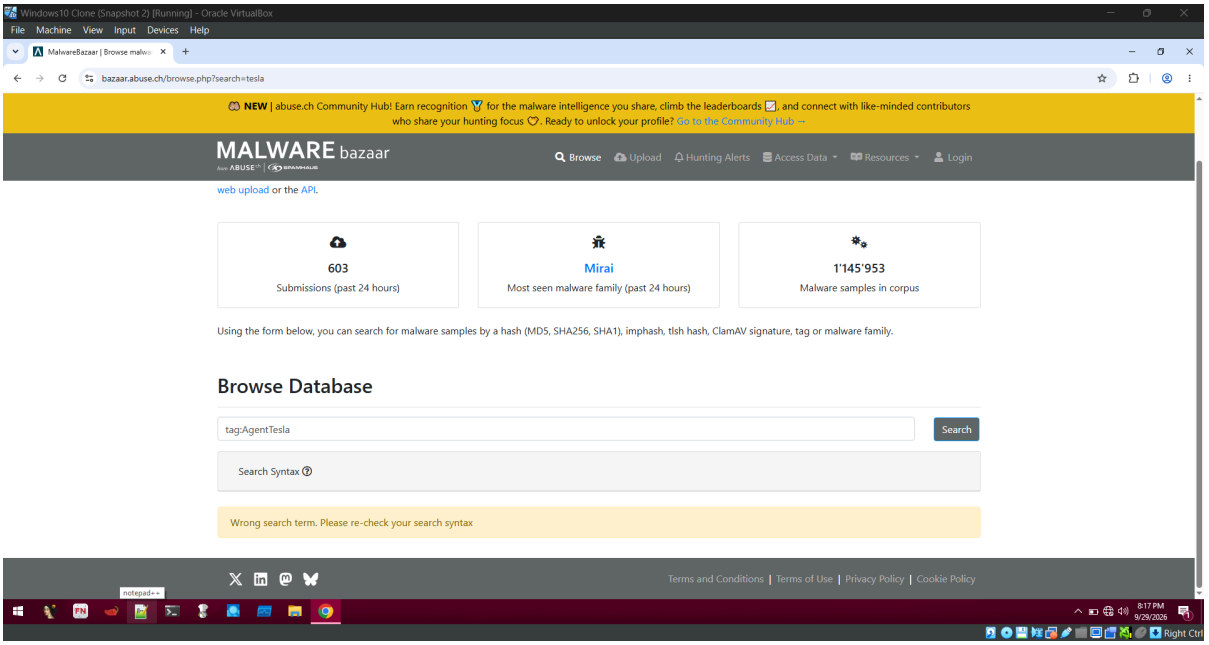


Extending the C: volume to absorb the resized disk space

I verified the fix by checking **this PC** — the C: drive showed roughly 80 GB total capacity, comfortably clearing FLARE VM's requirement — then reran the installer.

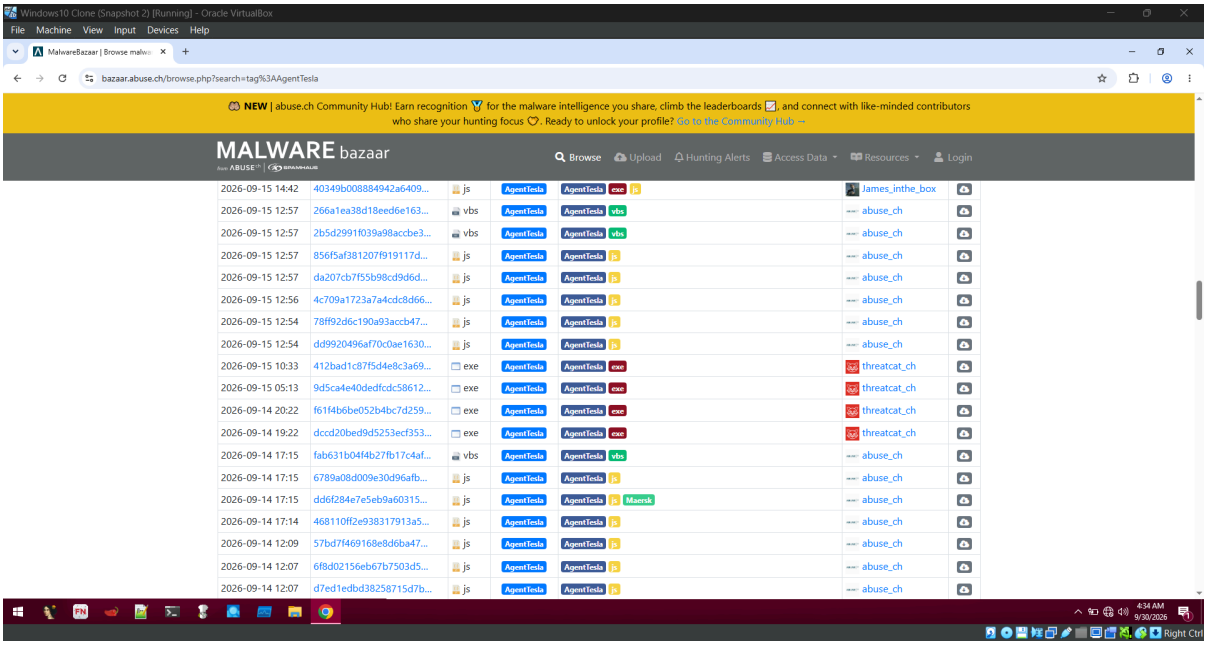
1.9 Malware Sample Installation

Before isolating the environment, I needed to actually get the sample onto the Windows VM and that had to happen while it still had real internet access, since the FLARE VM install in Section 1.7 was the last thing that required it. I used MalwareBazaar for the download. The reasoning behind choosing this specific Agent Tesla sample will be covered in the Static Analysis report; for this section, what matters is how it was obtained.

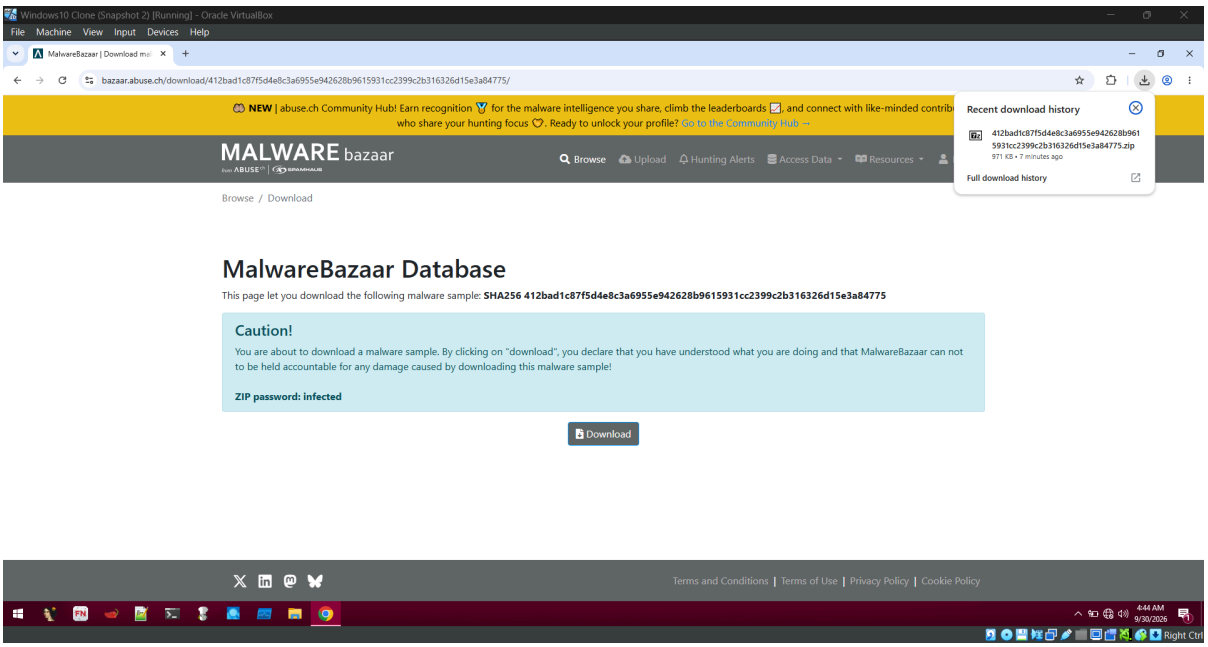


Initial MalwareBazaar search interface

Using the tag **tag:AgentTesla** returned a full result set, including the sample I'd already decided on from VirusTotal (412bad1c...):

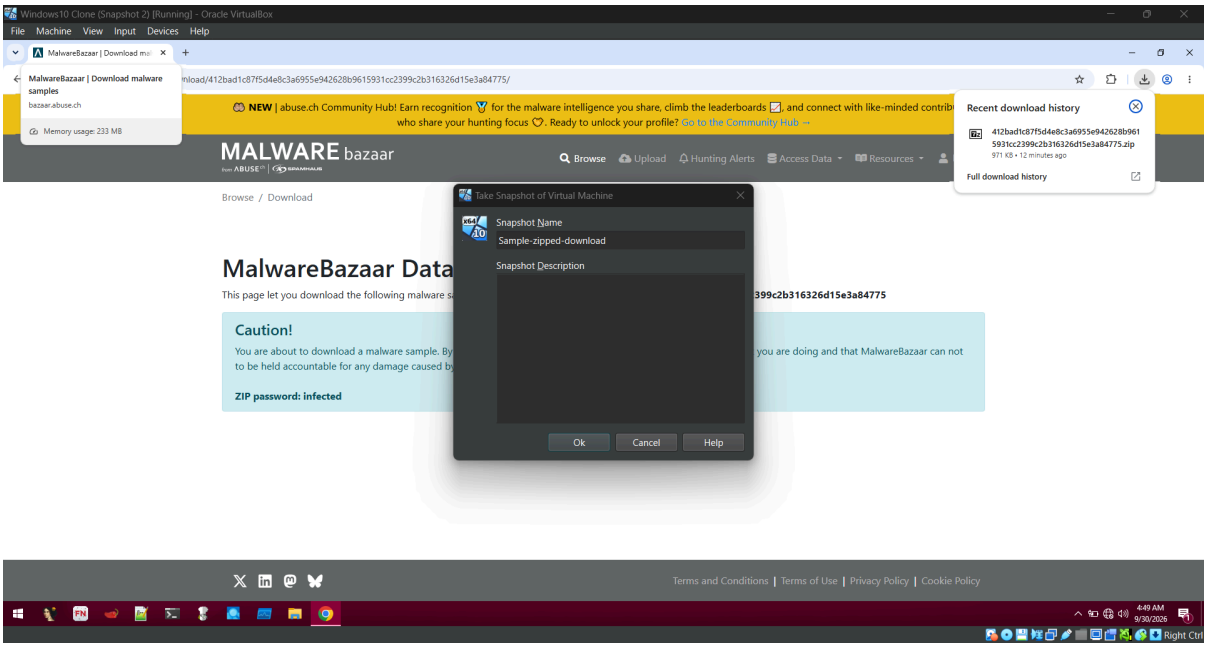


Search results for tag:AgentTesla



Downloading the selected sample (SHA256 412bad1c...) as a password-protected zip

With the sample sitting on the VM, still zipped, I took a snapshot marking this exact point before any further changes. This becomes the rollback point if anything goes wrong before the sample is actually extracted and analyzed.

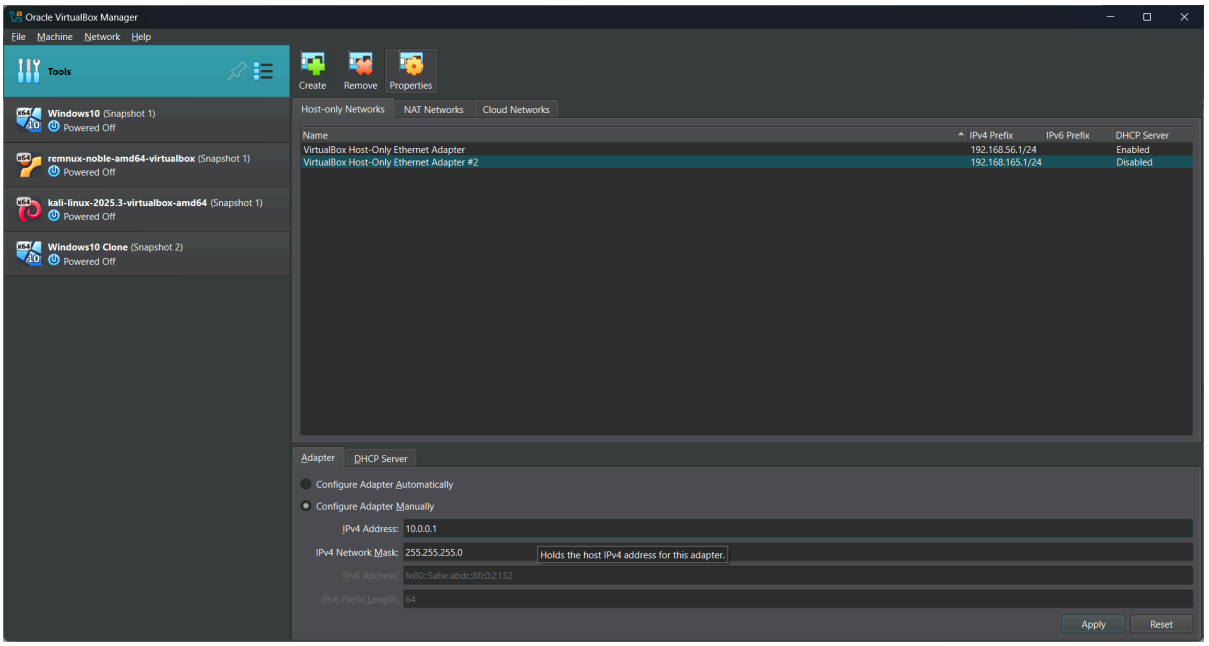


Snapshot taken immediately after the sample download, before network isolation

1.10 Network Isolation Configuration

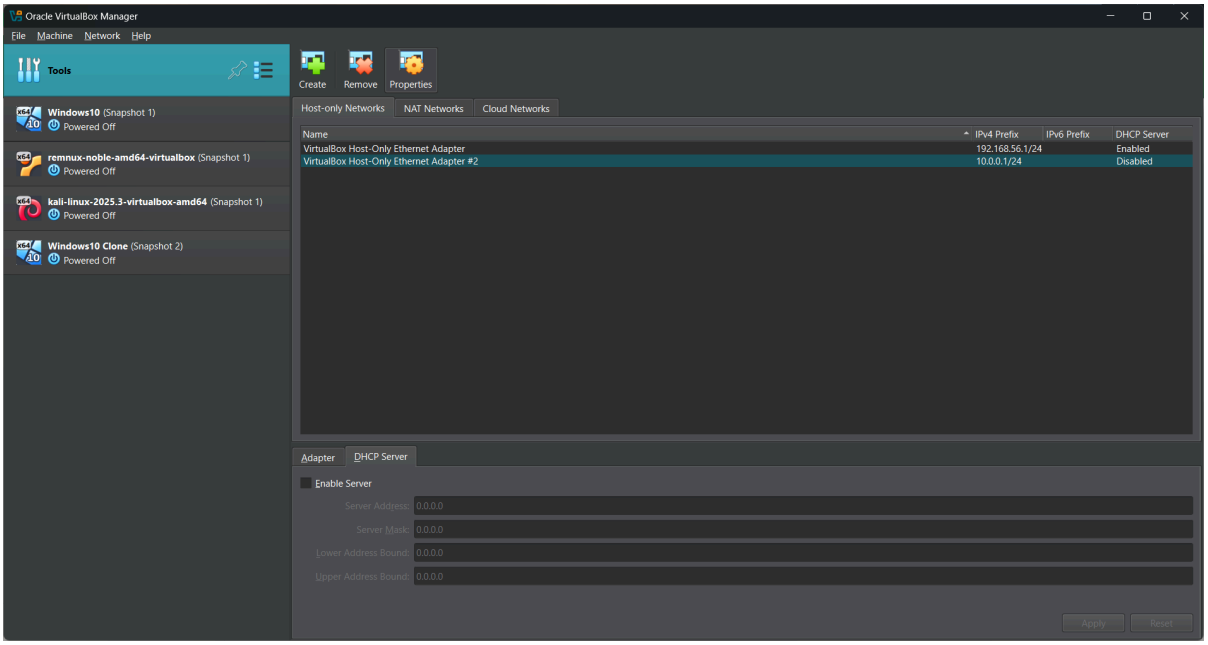
With both VMs built and FLARE VM's tooling installed, the last piece was wiring Windows and REMnux together the way Section 1.1 described: Windows plays the infected host, REMnux plays the network it thinks it's talking to. This has to happen after the FLARE VM install and the sample download above, not before, since both of those still needed real internet access.

Choosing the adapter and addressing scheme. I attached both VMs to a second VirtualBox Host-only adapter (**VirtualBox Host-Only Ethernet Adapter #2**), created specifically for this lab through **Tools** → **Network Manager** → **Host-only Networks** → **Create**, so it stays separate from the default host-only network VirtualBox ships with.



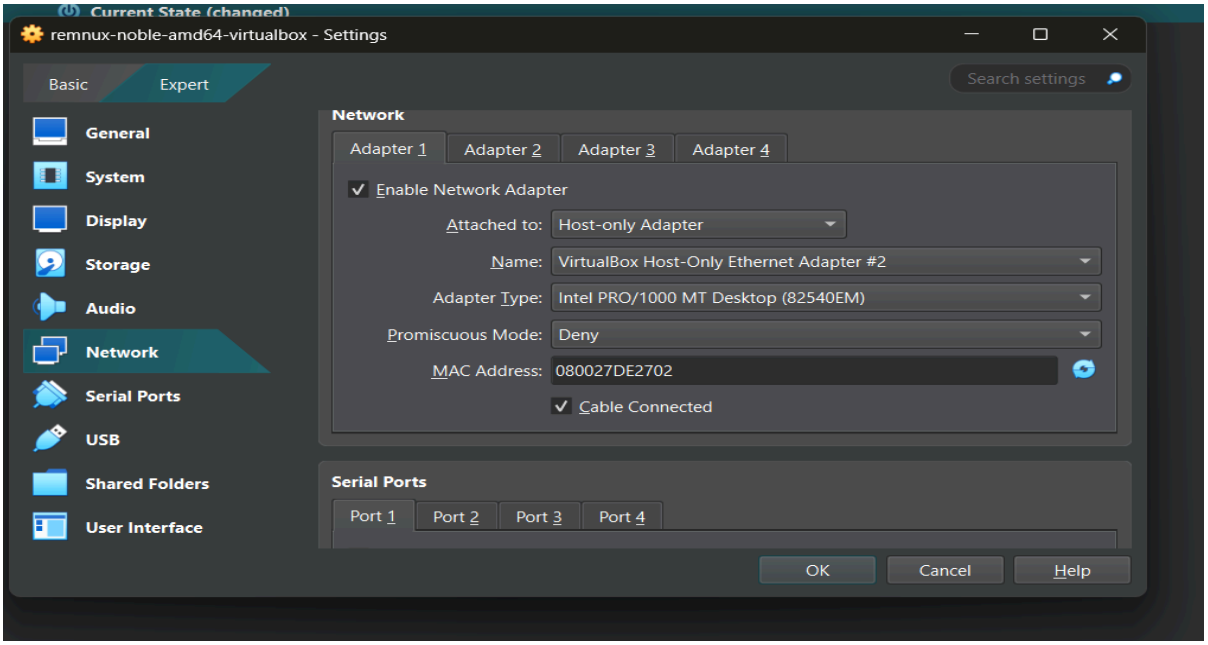
Creating the dedicated Host-only Adapter #2 and setting its manual IPv4 address

I deliberately left its DHCP server disabled and assigned static addresses by hand instead — REMnux at **10.0.0.1**, Windows at **10.0.0.2/24**, with Windows' gateway and DNS both pointed at REMnux. The reasoning: with DHCP involved, there's a risk the Windows VM leases an address with the wrong gateway or DNS server, which would mean the malware's lookups never reach REMnux at all. Static addressing removes that uncertainty entirely.

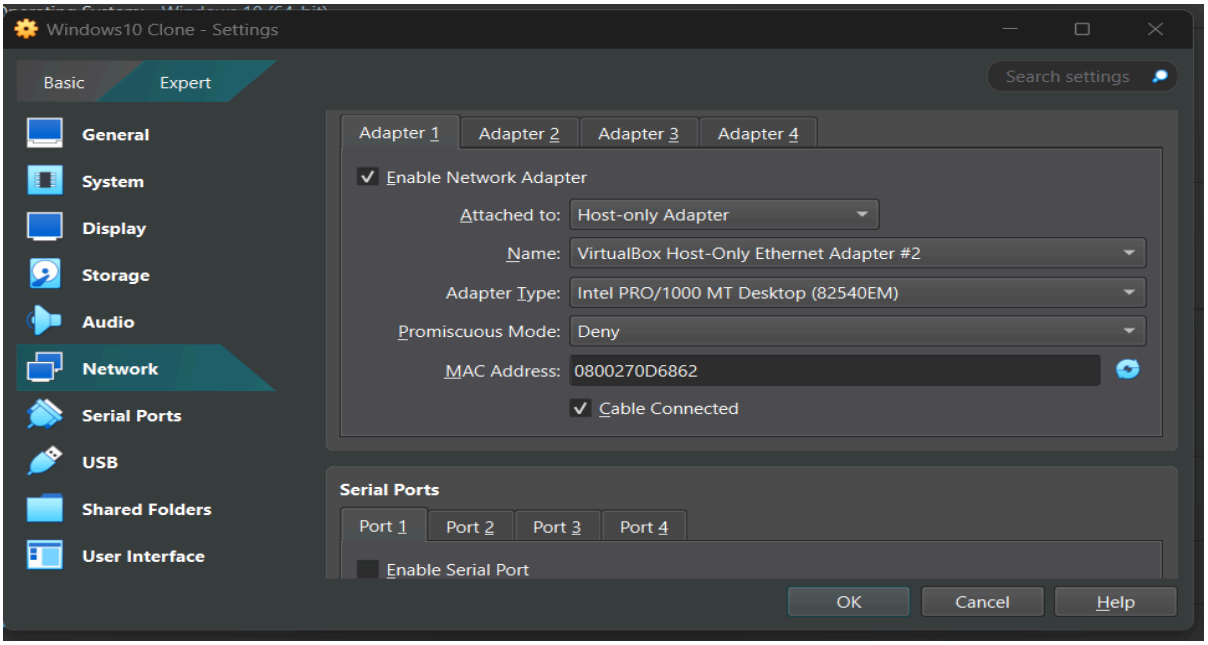


Host-only Adapter #2 finalized at 10.0.0.1/24 with the DHCP server disabled

Both VMs' Adapter 1 was then set to this adapter specifically, not the default one:

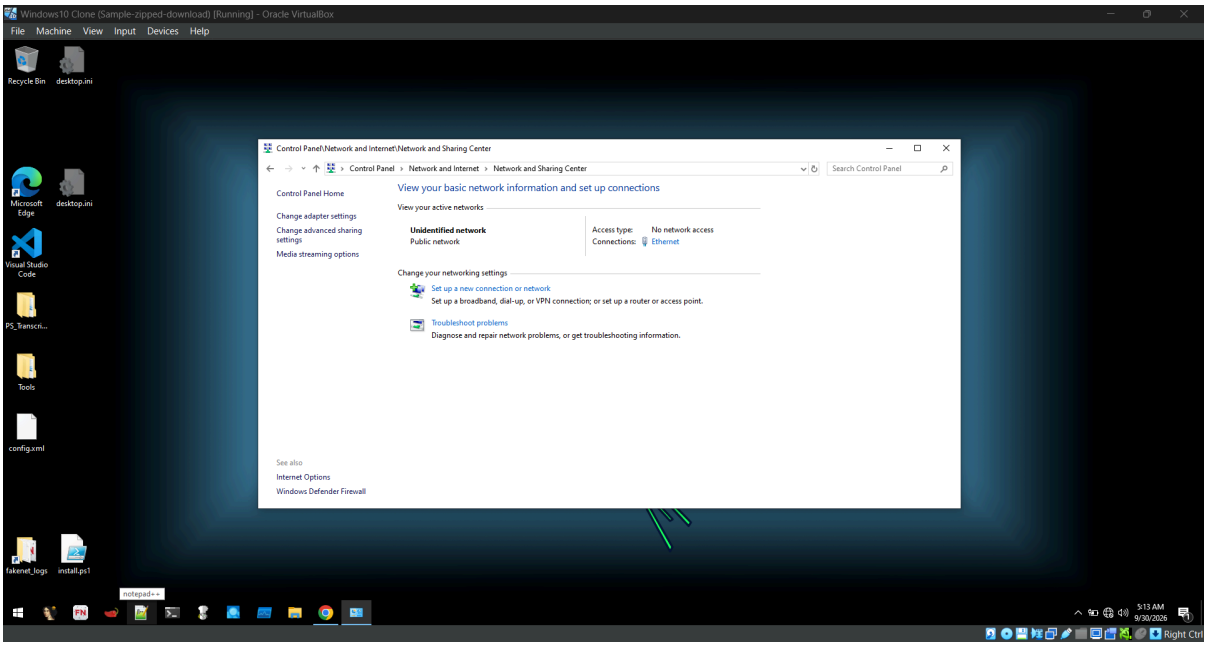


REMnux — Adapter 1 attached to Host-only Adapter #2



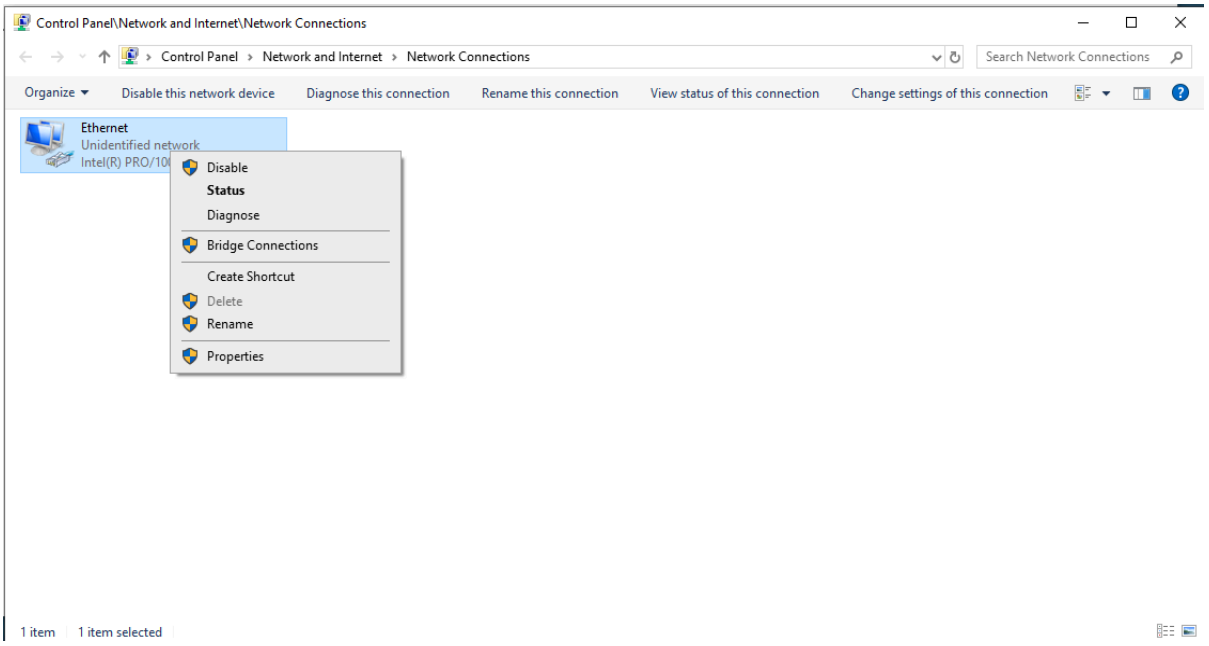
Windows10 Clone — Adapter 1 attached to the same Host-only Adapter #2

Configuring Windows. Setting the static IP meant going to **Control Panel → Network and Internet → Network and Sharing Center → Change adapter settings**. At this point the adapter correctly showed as an "Unidentified network" with no internet access, which is expected since it has no real gateway yet:



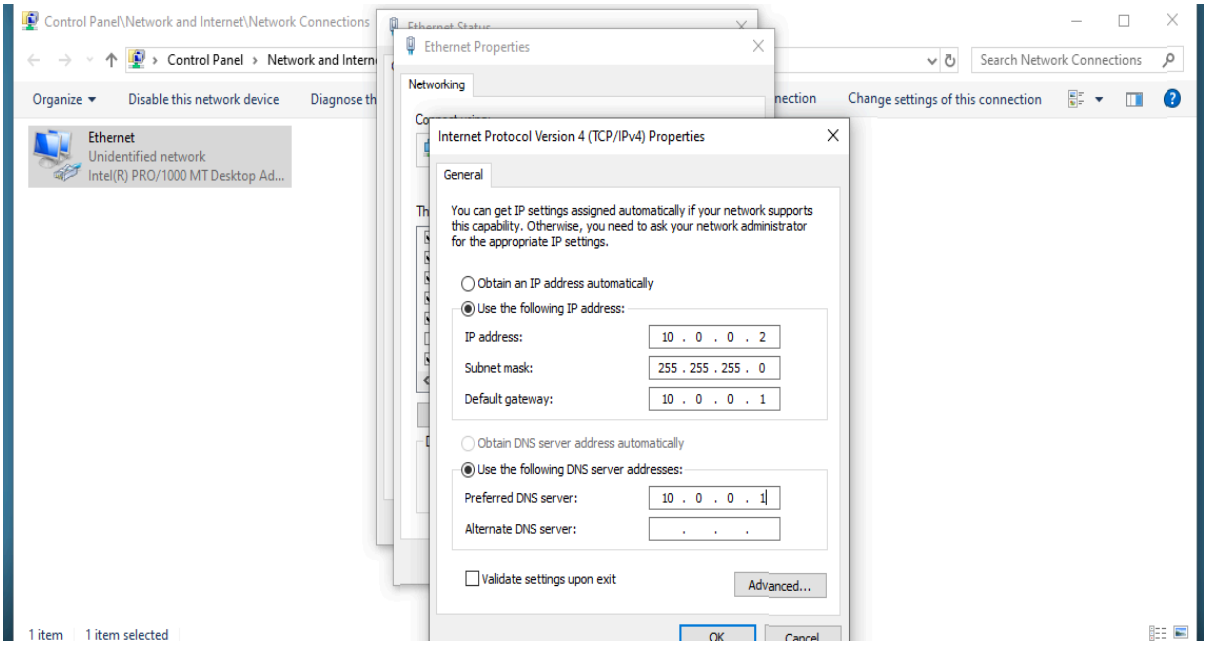
The isolated adapter showing as an 'Unidentified network' with no internet access

Right-clicking it and opening Properties, then clicking **Internet Protocol Version 4 (TCP/IPv4)**:



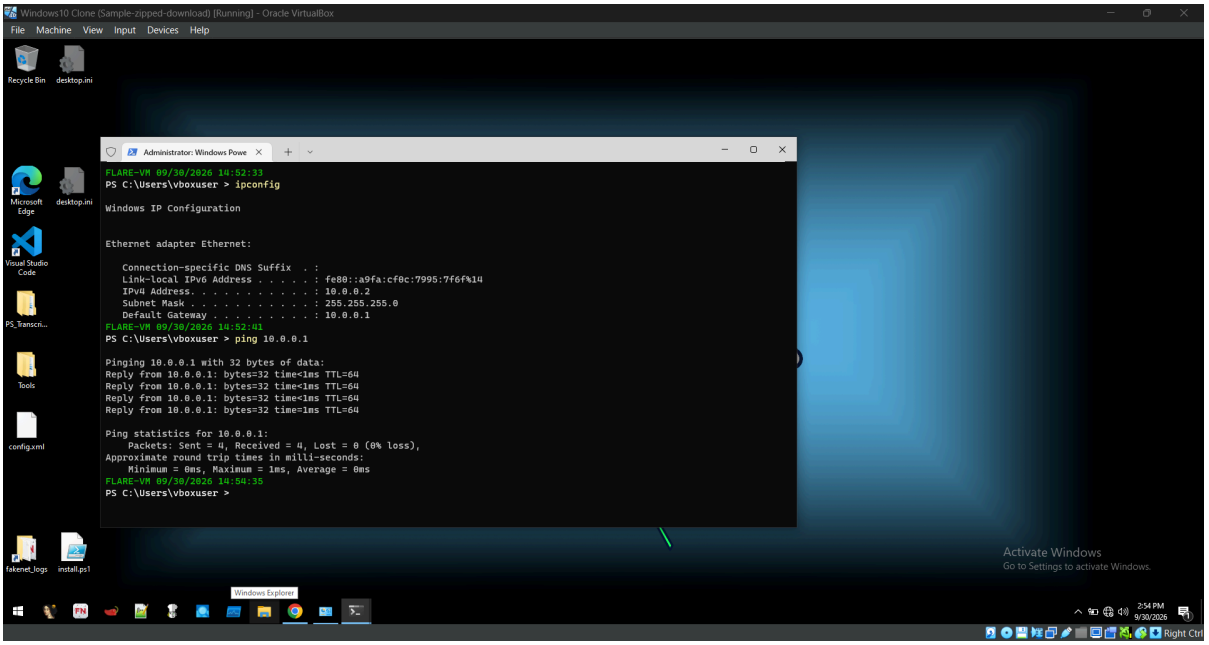
Opening the Ethernet adapter's properties

...and setting the static values: address 10.0.0.2, mask 255.255.255.0, gateway and preferred DNS both 10.0.0.1.



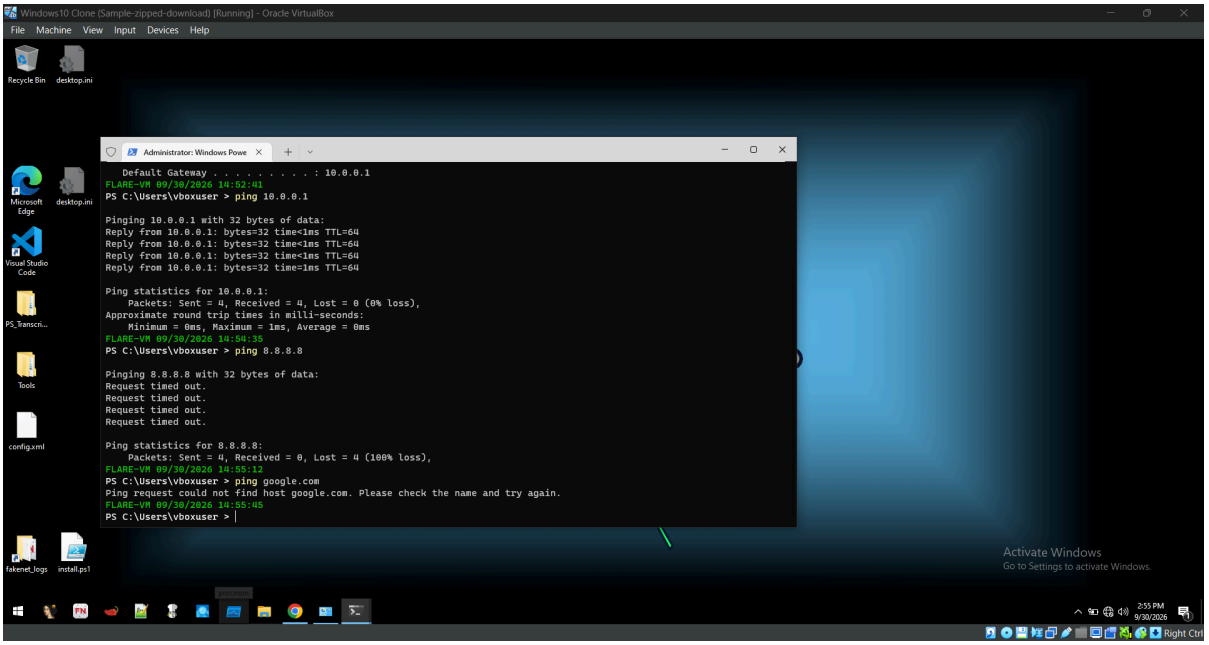
Windows static IP configuration — 10.0.0.2/24, gateway and DNS pointed at REMnux

With REMnux up, I confirmed the link from the Windows side — a successful ping to REMnux confirmed the two VMs could talk:



ipconfig confirming the static IP, followed by a successful ping to REMnux (10.0.0.1)

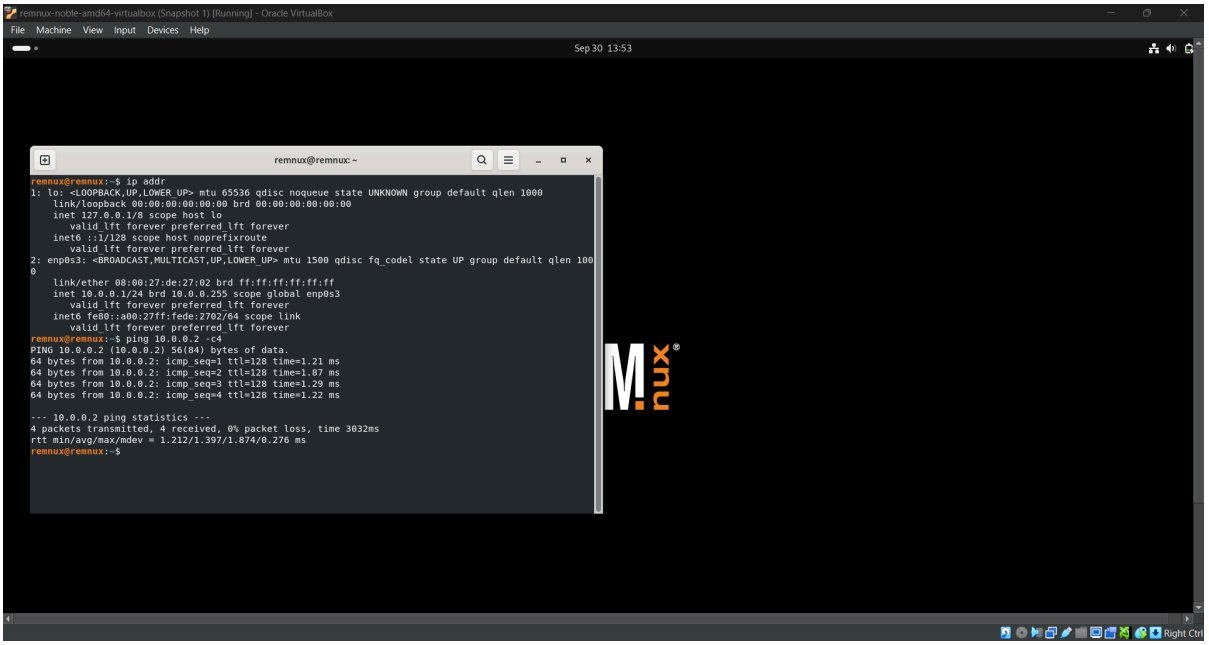
I also pinged a real external address and tried resolving a real domain, both of which failed as expected confirming the isolation itself was actually working, not just the internal link:



Ping to 8.8.8.8 and a DNS lookup for google.com both failing, confirming no route to the real internet

Configuring REMnux. I first applied the address with a plain *ip addr add* command to confirm connectivity quickly, but this doesn't survive a reboot. Trying to make it permanent through NetworkManager (nmtui, then nmcli directly) failed with "device is strictly unmanaged". It turned out REMnux configures its interface through netplan, not NetworkManager, so NetworkManager had no authority over the adapter at all. The actual fix was writing the address directly into REMnux's netplan config and running *netplan apply*. I

confirmed this was genuinely persistent, not just applied, by rebooting REMnux and checking the address was still there with no manual steps.



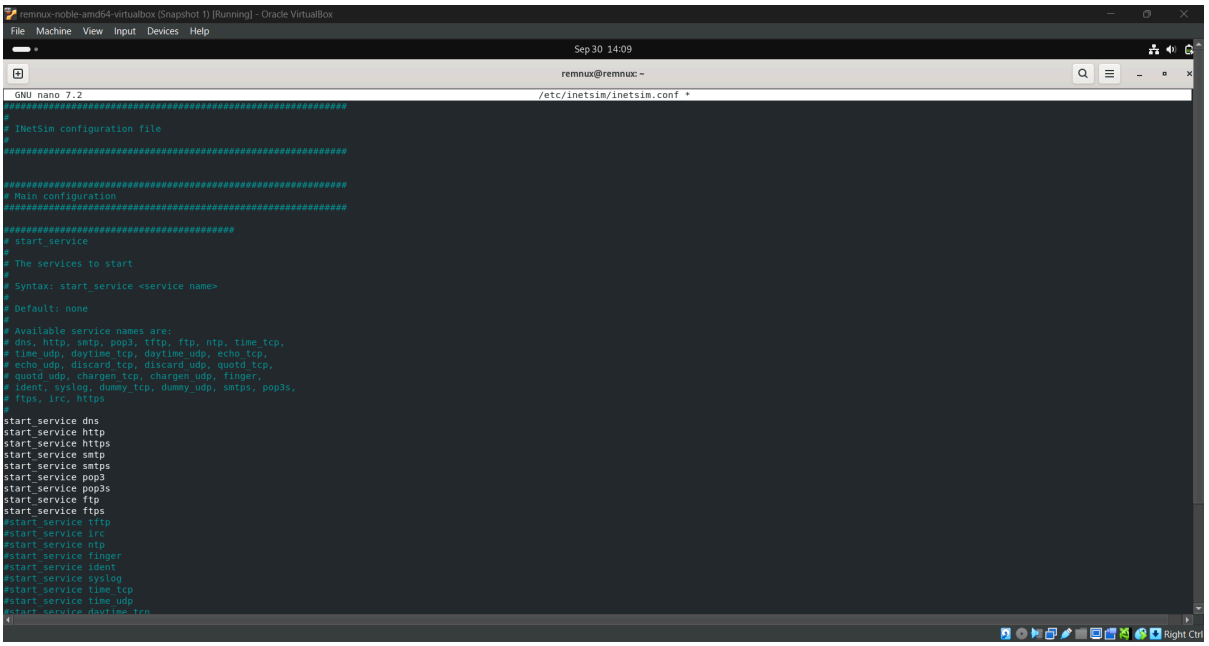
REMnux after the persistent netplan config survived a reboot, with a successful ping back to the Windows VM

Verifying the link and an unexpected detour. Pinging between the two VMs had initially surfaced a one-directional problem: REMnux could ping Windows, but Windows couldn't ping REMnux back. Checking REMnux's ARP table showed it already had Windows' MAC address resolved, which confirmed the two VMs could see each other fine at the link layer. So the fault had to be something filtering ICMP specifically on the Windows side, not a cable or adapter issue. That pointed at Windows Firewall. Even though Defender's protections were disabled back in Section 1.5, this host-only adapter is a separate network interface that Windows evaluates independently, and it was blocking inbound ping on its own default rules. Running ***netsh advfirewall set allprofiles state off***, followed by explicit ICMPv4 inbound/outbound allow rules (***netsh advfirewall firewall add rule name="Allow ICMPv4-In" protocol=icmpv4:8,any dir=in action=allow***), resolved it, and pings succeeded cleanly in both directions, as shown above in the screenshots.

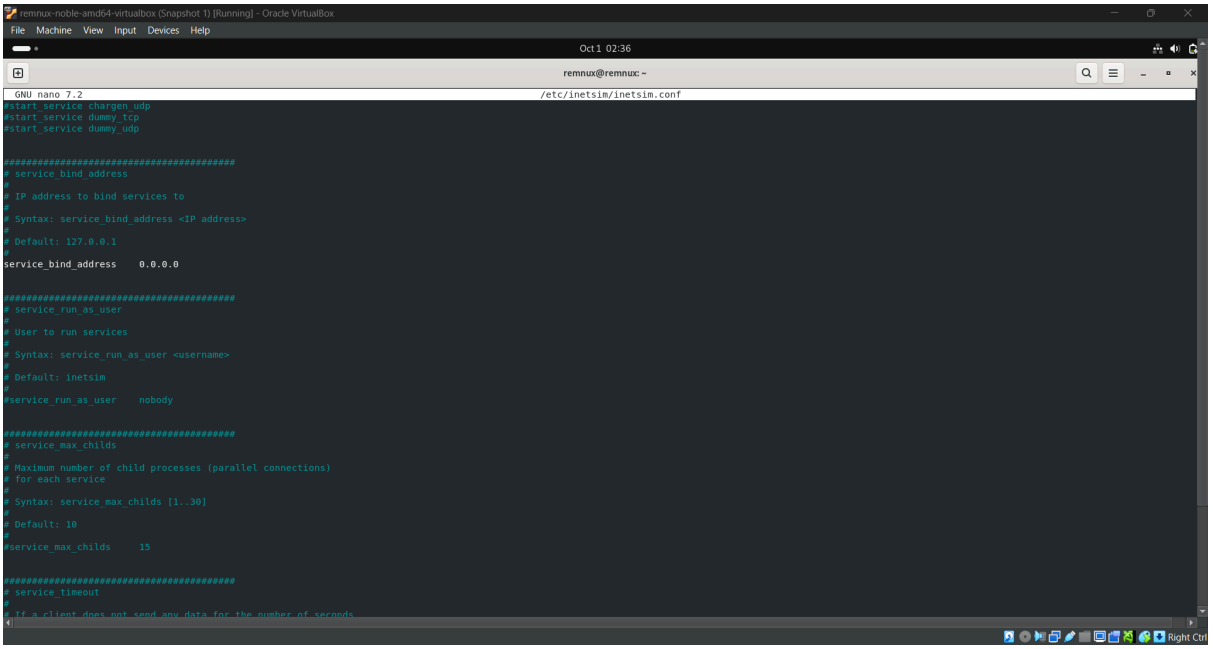
Worth noting for anyone repeating this: the connectivity didn't stay fixed on its own. After leaving REMnux running for a few hours, the same one-directional block reappeared, which suggests this firewall state isn't fully stable from session to session. The practical takeaway is to treat the bidirectional ping check as something to re-run immediately before every detonation, not as a one-time setup step.

Verifying DNS and HTTP redirection with INetSim. A working ping confirms the link, but not that REMnux can actually impersonate the network the malware will try to reach. I edited REMnux's INetSim configuration using the command ***sudo nano /etc/inetsim/inetsim.conf*** to enable the services the sample is most likely to touch DNS, HTTP, HTTPS, SMTP, and FTP

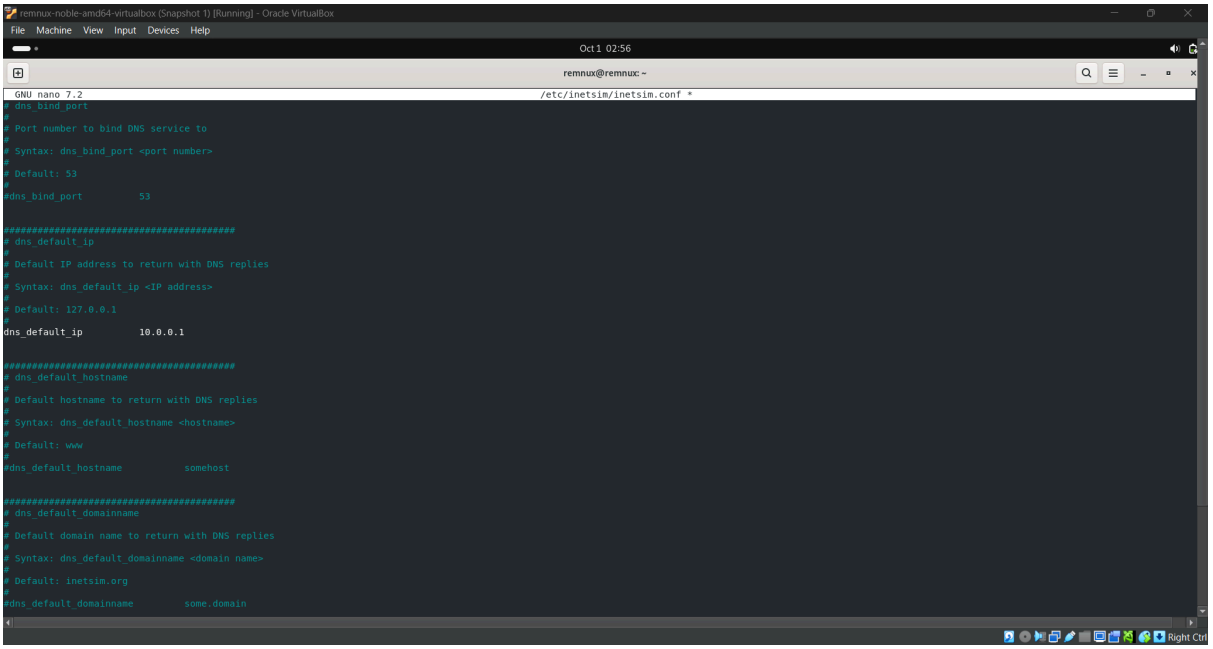
and left the **bind address** at its default **0.0.0.0** and also the **DNS default IP** to **10.0.0.1**



Editing /etc/inetsim/inetsim.conf to enable DNS, HTTP, HTTPS, SMTP, and FTP services

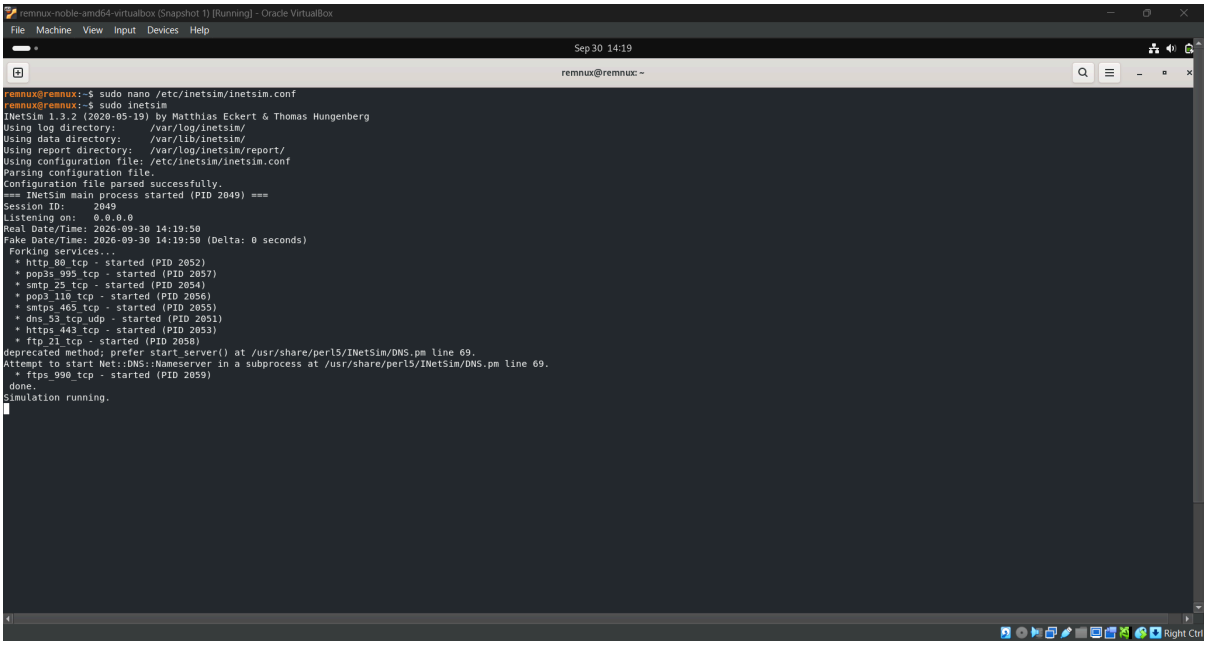


Editing service_bind_address to 0.0.0.0



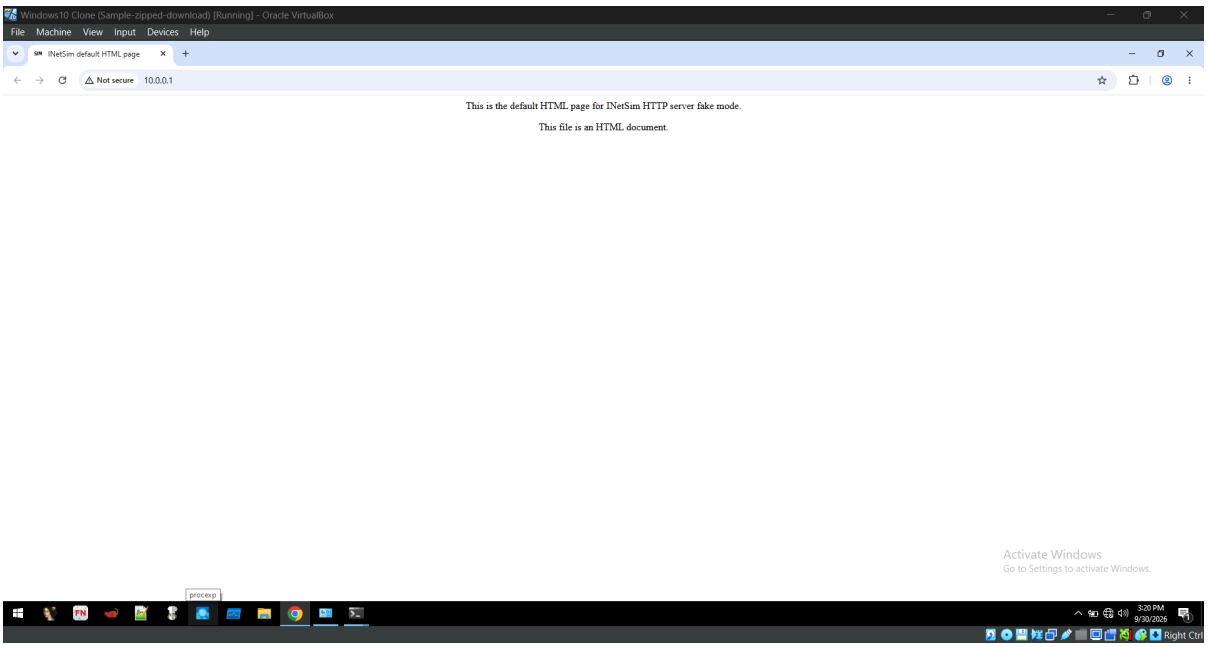
Editing dns_default_ip to 10.0.0.1

Starting INetSim confirmed all of the enabled services came up cleanly:



INetSim running, with DNS, HTTP, HTTPS, SMTP, and FTP services started

From the Windows VM, opening a browser and navigating to an arbitrary address resolved straight to REMnux and returned INetSim's default placeholder page — exactly the behavior needed before the sample can be safely detonated:



Browser on the Windows VM landing on INetSim's default placeholder page at 10.0.0.1

With this confirmed, the environment is genuinely analysis-ready: isolated from the real internet, fully address-resolved to REMnux, and running fake services for whatever the sample reaches out for.

1.11 Final Checkpoint

With FLARE VM fully installed, both installation issues resolved, the sample downloaded, and the network isolation configured and verified end-to-end — link, isolation, and INetSim redirection all confirmed. I took a snapshot of both VMs reflecting this state. This becomes the reference state I'll revert to after every malware detonation going forward, ensuring that each analysis run starts from the same clean, tool-ready, isolated baseline rather than one contaminated by a previous run's changes.

SECTION 2:

STATIC ANALYSIS REPORT

2.1 Sample Selection and Safe Handling

Static analysis means examining a file without running it, so for this entire section the sample was never executed. Every tool I used read the file from disk, and PEStudio opened it read-only. That matters because the sample is live malware, and the whole point of this stage is to learn as much as possible before taking that risk.

Agent Tesla is a widely circulated infostealer that regularly turns up in phishing campaigns, so it's the kind of thing that lands in a SOC queue. It's also .NET-based, which means it can be decompiled and read, so I could explain my findings instead of just reporting detections. I considered Emotet and Formbook, but Emotet's infrastructure is often offline, and Formbook is usually packed in layers that would turn the project into an unpacking exercise.

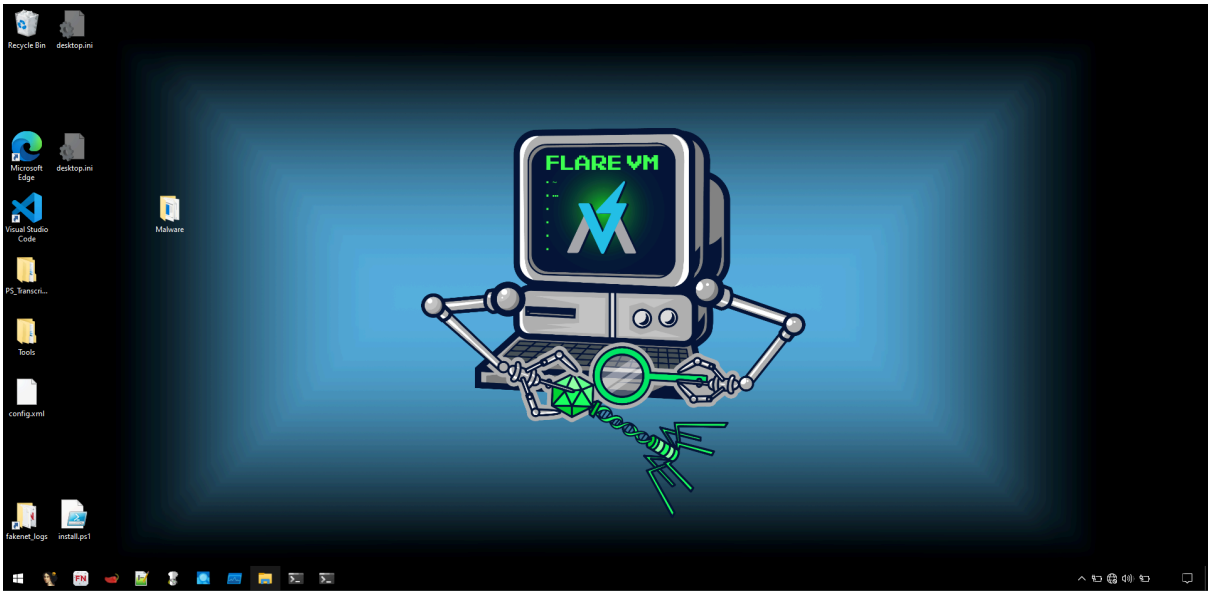
I searched MalwareBazaar for the AgentTesla tag. Most of the recent entries were JavaScript or VBS downloaders, signed executables, or archives. Those add extra delivery-chain work before you reach any payload. I chose a plain, unsigned .exe instead, so the file itself would be the thing under analysis.

Property	Value
Source	MalwareBazaar (abuse.ch), tagged AgentTesla, first seen 2026-09-15
Original filename	rPEq.exe
File type	32-bit PE, .NET GUI executable
Size	1,112,576 bytes (1.06 MiB)
SHA-256	412bad1c87f5d4e8c3a6955e942628b9615931cc2399c2b316326d15e3a84775
MD5	edd1ed5832c6cc4bc65ce5897ac4565f
SHA-1	7435418fd558c94c8d9ebfeeb49abe0ddd37c4b4

Before downloading it, I checked it on VirusTotal. 55 of 70 vendors flagged it as malicious. Vendors disagreed on the family, though: Kaspersky labeled it Noon and Huorong labeled it AgentTesla, while Fortinet called it Formbook. So I treated MalwareBazaar's "AgentTesla" tag as a claim to test, not a fact. Whether this file really is Agent Tesla, or only delivers it, became one of the questions for the rest of the analysis.

With the sample downloaded as a password-protected zip (Section 1.9), I took a snapshot of the VM, so I could always return to a state where the sample was still zipped and inert. I then created a new folder named Malware on the desktop and extracted the sample there. Keeping it in one dedicated folder means I always know where the live file is, and every tool in this section points at the same path. I extracted it only after the VM was isolated on the host-only network (Section 1.10). (HashMyFiles shows the file was created on 10/5/2026,

after the network isolation on 9/30.) Windows Defender was already off (Section 1.5), so nothing would quarantine the file mid-analysis.



The Malware folder on the FLARE VM desktop, holding the extracted sample.

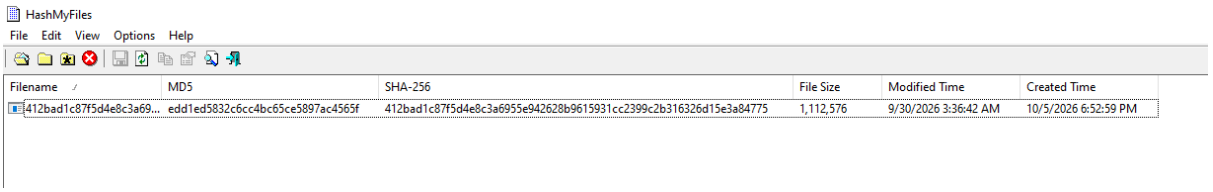
2.2 Establishing Identity: Hashes and File Type

Before looking inside the file, I needed to be sure the file on disk was the one I meant to analyze, and to know what kind of file it was. That decides which tools make sense for everything after it.

For each question in this section I used at least two independent tools. I didn't distrust any single tool, but a tool that misbehaves and returns a wrong answer would quietly undermine everything built on it.

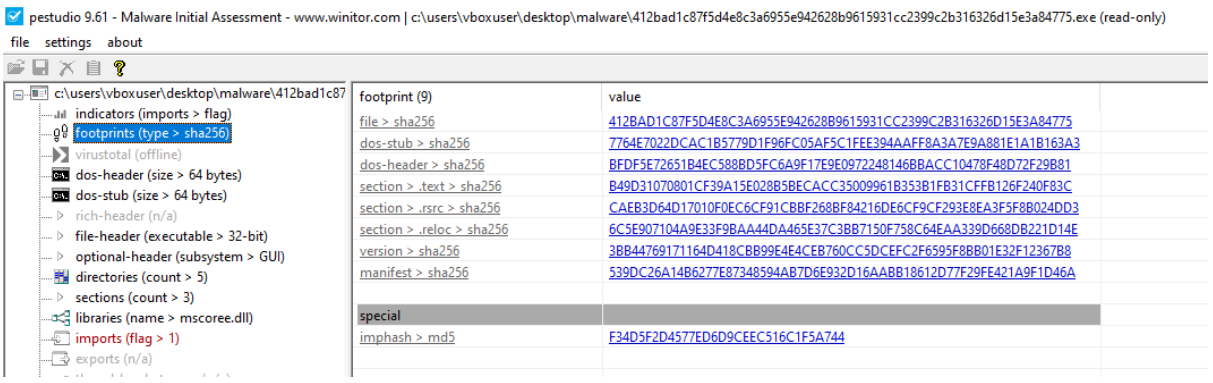
Hashes.

I started with HashMyFiles to get the SHA-256 and MD5, and compared them with the values on MalwareBazaar's download page.



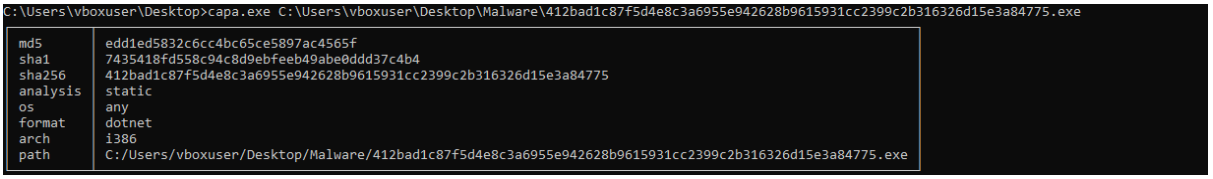
HashMyFiles output for the extracted sample (1,112,576 bytes).

The SHA-256 (412bad1c…a84775) matches the source exactly, and the MD5 is edd1ed5832c6cc4bc65ce5897ac4565f. That confirms the file wasn't corrupted or swapped between download and extraction. I then opened the file in PEStudio, which computes its own hashes independently and showed the same SHA-256.



PEStudio's independent SHA-256 matches HashMyFiles and the source.

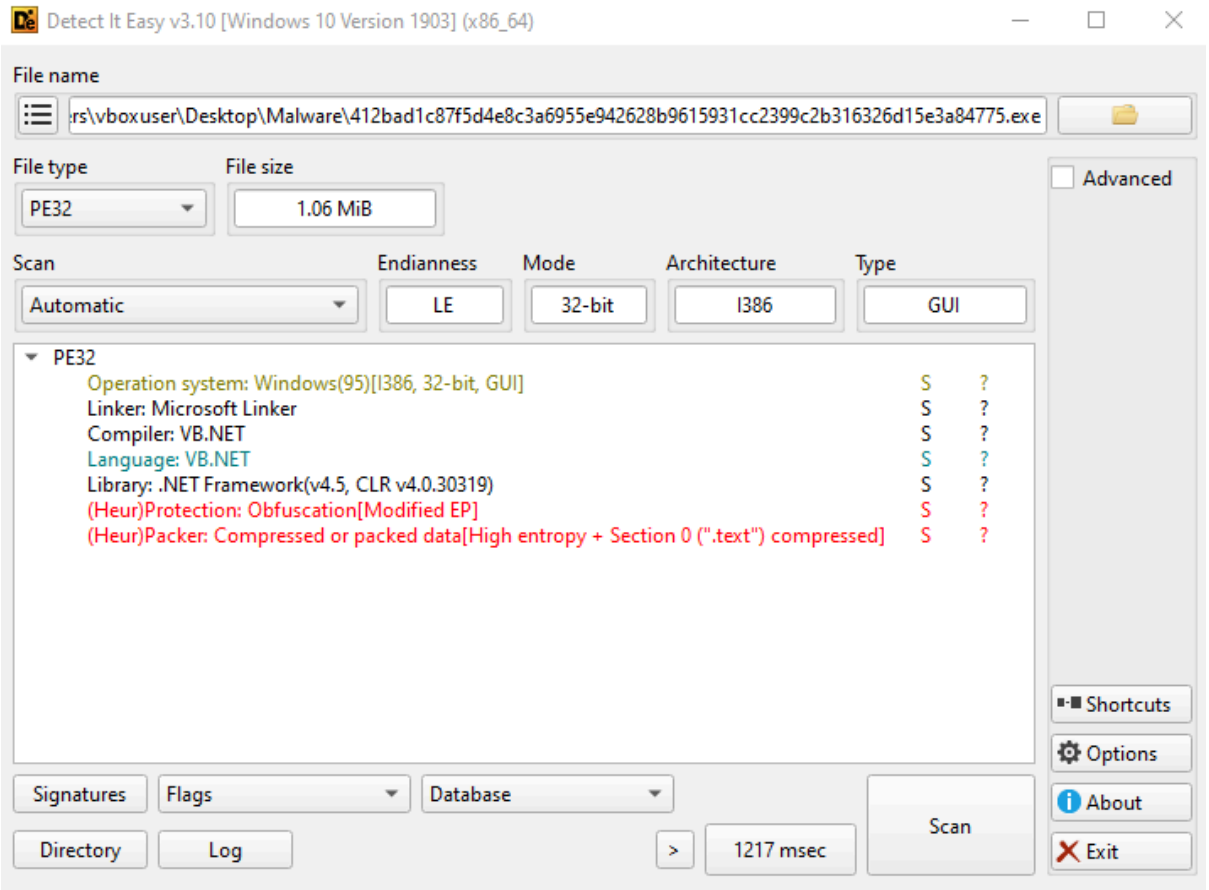
PEStudio's footprints view only showed the SHA-256, so I took the MD5 from capa instead. capa's MD5 and SHA-256 matched the values from HashMyFiles.



PEStudio also gave an import hash (imphash) of F34D5F2D4577ED6D9CEEC516C1F5A744. I recorded it, but it has little value here: a .NET program imports only mscoree.dll, so many unrelated .NET executables are likely to share this value.

File type.

Next I ran Detect It Easy (DIE) to identify the file type, compiler, and any packer or obfuscator.



DIE identifies a 32-bit VB.NET executable, with heuristic flags for obfuscation and packing.

DIE reports a 32-bit PE32 GUI executable, compiled as VB.NET on .NET Framework 4.x. That told me dnSpy could be used for decompilation. For a second opinion I ran the file command from the FLARE VM, which also describes it as a “Mono/.Net assembly”.



The file command independently confirms a .NET assembly.

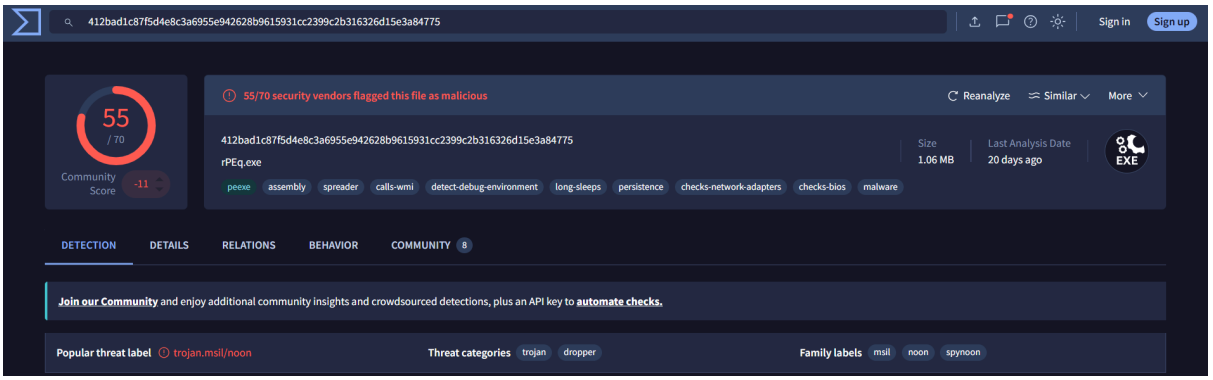
What this told me, and what I did next.

- **It's a .NET assembly.** Both tools agree, and PEStudio lists mscoree.dll as its only library. That means I can use a .NET decompiler like dnSpy later and read close to the original code, instead of struggling through raw assembly.
- **DIE raised two heuristic warnings (shown in red):** "Obfuscation [Modified EP]" only says the entry point differs from what DIE expects, and PEStudio's entry bytes look like the standard .NET stub, so I treated this as the weaker signal. "Compressed or packed data" in .text was stronger, because PEStudio confirmed an entropy of 7.745 (anything above about 7 is very high). Together they suggested the file might be a loader that unpacks something else at runtime. That was only a hypothesis, so I checked entropy and structure next (2.4) before deciding how to approach the code.

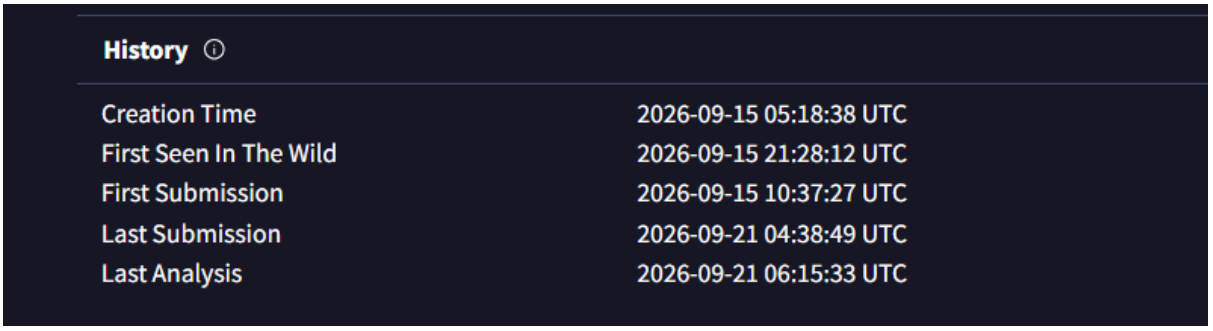
2.3 Existing Intelligence

Before spending time on the file itself, I checked what the security community already knew about it. This sets expectations for what I'd find, and it gives me something to test my own findings against later. I looked up the hash only and never uploaded the file, so I didn't make the sample public.

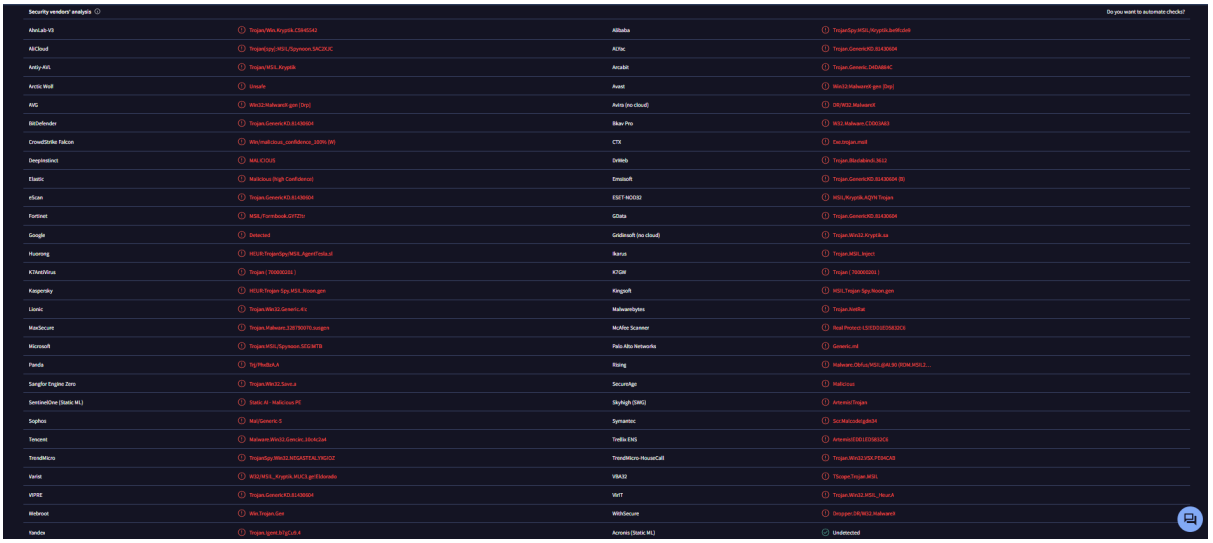
VirusTotal flagged the sample as malicious on 55 of 70 engines. Its summary labels it a trojan and dropper, with the family names **"msil", "noon"** and **"spynoon"**. The file is unsigned, and its first submission came about five hours after its compile time, so it was brand new when it surfaced (2026-09-15).



VirusTotal summary: 55/70 detections, threat label and behavior tags.

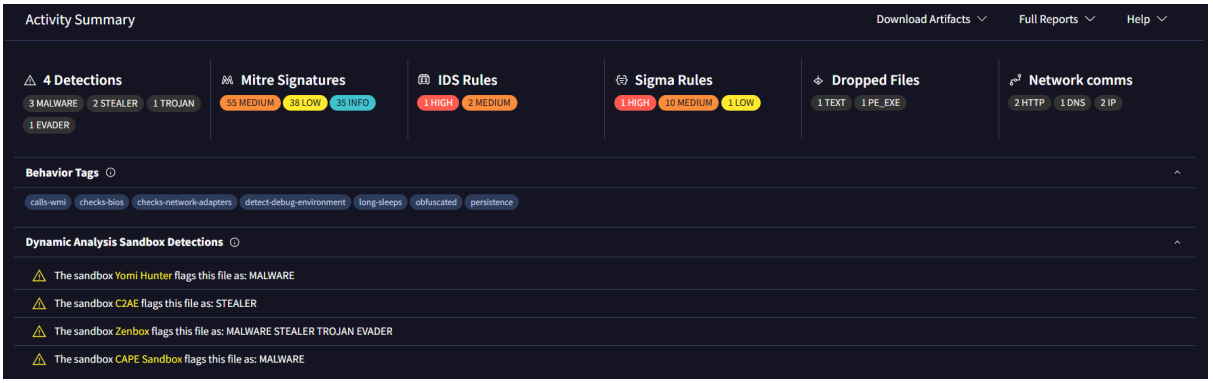


The vendors disagree about the family. The only vendor that says Agent Tesla outright is Huorong. Others say Formbook (Fortinet), Bladabindi (Dr.Web, a name associated with njRAT), or just "spyware", and the "Kryptik" labels describe how the file is packaged, not what it does. Engines label a file by whatever signature it partly matches, and a loader can resemble the family it carries. So I treated MalwareBazaar's AgentTesla tag as a **claim to test, not a fact**. The common thread across vendors is "information-stealing trojan delivered by an obfuscated loader", which fits what DIE and the entropy figures suggested.



Behavior tags.

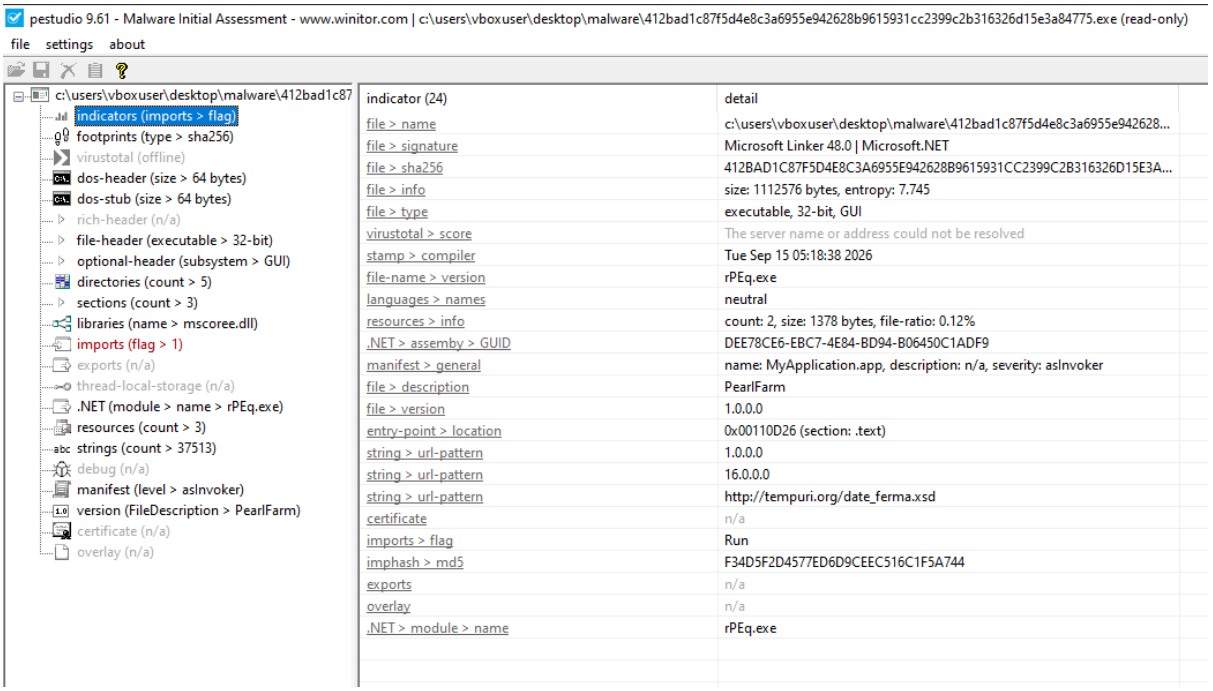
VirusTotal's sandbox runs tagged the file with detect-debug-environment, checks-bios, checks-network-adapters, long-sleeps, calls-wmi, persistence and spreader. The first four are anti-analysis behaviors: checking whether it's inside a VM, then waiting out automated sandboxes. This matters for my own lab, since I should expect the sample to behave differently, or not at all, in VirtualBox. I recorded all seven as hypotheses to confirm. spreader is the one I'd least expect from a stealer, so I wanted to see whether the code supported it.



2.4 PE Structure and Entropy

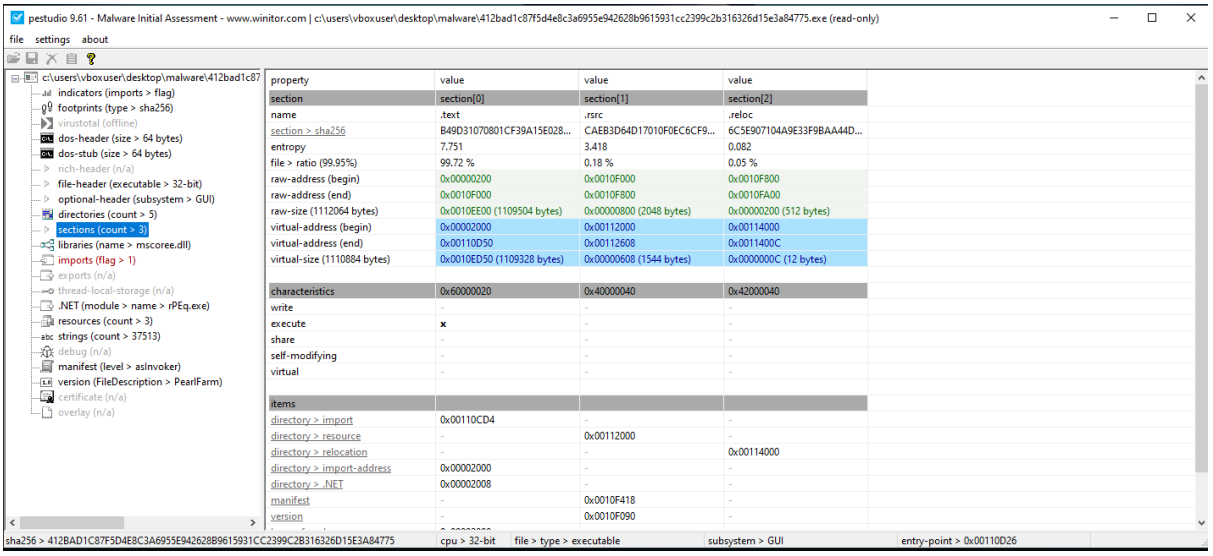
DIE had flagged possible packing, but that was only a heuristic. Entropy measures how random data is, on a scale of 0 to 8. Ordinary code sits around 5 to 6.5, and anything above about 7 usually means compressed or encrypted content. So I needed to know whether the high entropy was confined to one part of the file, which is normal, or spread across the whole file, which usually points to packing or encryption.

PEStudio gave an overall entropy of 7.745 for the file.



PEStudio's **section** view gave the breakdown, and VirusTotal's Details tab, my second tool, matched it figure for figure.

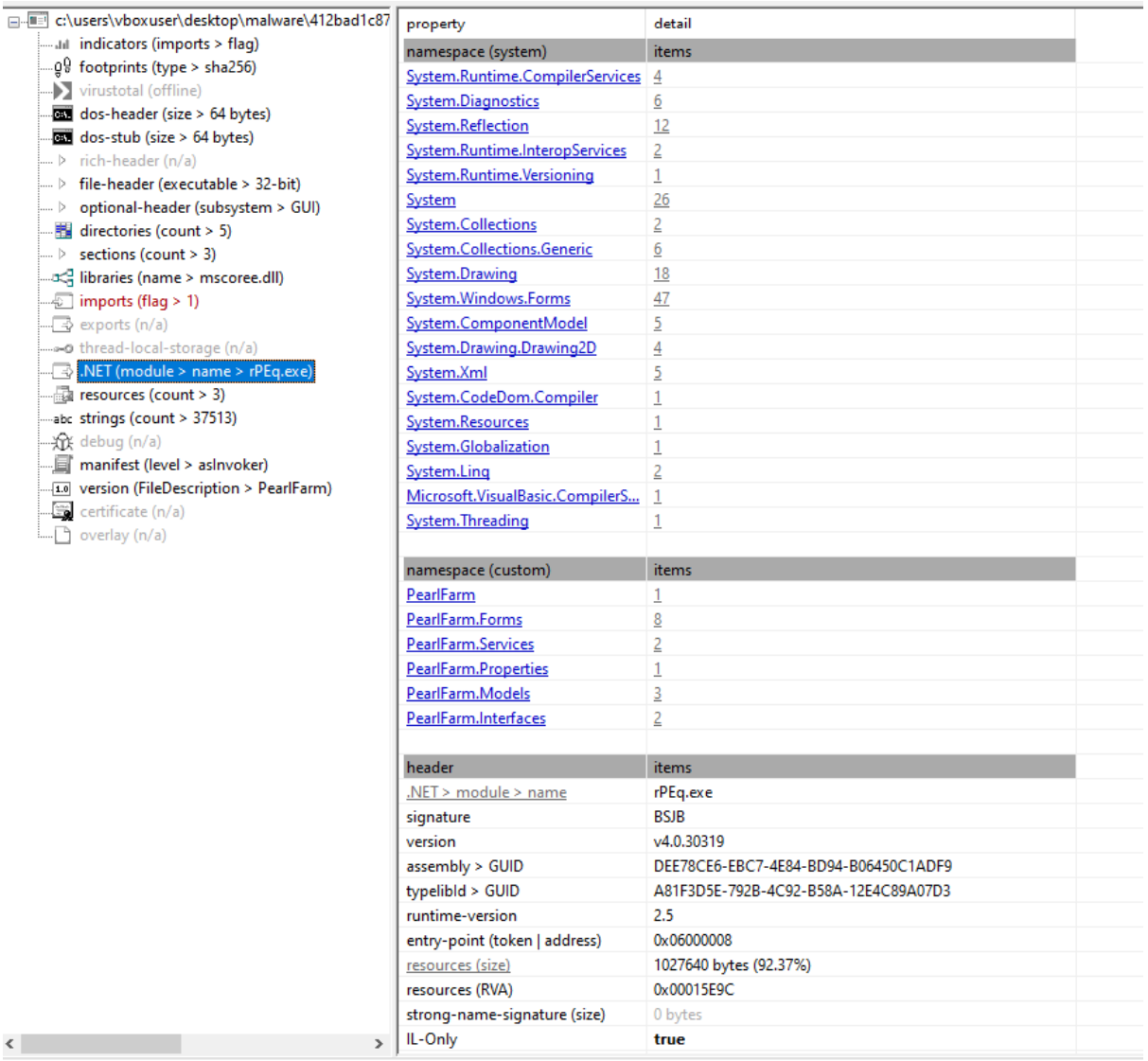
Section	Raw size	Share of file	Entropy	Reading
.text	1,109,504 B	99.72%	7.751	Nearly the entire file, and very high entropy
.rsrc	2,048 B	0.18%	3.418	Tiny, holds only version info and a manifest
.reloc	512 B	0.05%	0.082	Normal for a .NET executable



PEStudio's section table: .text is 99.72% of the file at entropy 7.751.

So the high entropy wasn't just one small section. It covers essentially the whole file. Real program code doesn't look like that, so something large and random-looking was stored inside it.

The .NET view in PEStudio answered that directly. The managed resources total **1,027,640 bytes, which is 92.37% of the file**. Everything else, meaning the program's own types, methods and strings, is small.



PEStudio's .NET view: managed resources make up 92.37% of the file.

VirusTotal lists a single managed resource, PearlFarm.Properties.Resources.resources, and the Win32 resources are only about 1.4 KB. So the bulk of the file is one resource sitting inside .text. I couldn't see what it contains from PEStudio, so that became a question for dnSpy.

The rest of the structure.

- **Imports:** only mscoree.dll, so it's a pure .NET program.
- **Namespaces:** the code lives in a custom PearlFarm namespace split into Forms (8 items), Models, Services, Interfaces and Properties. The program also references System.Drawing (18 items) and System.Windows.Forms (47). So it's an app that handles images.
- **Entry point:** the .NET entry token is 0x06000008. I noted it for later, since in dnSpy it identifies the Main method. The native entry sits at the tail of .text, where the standard .NET startup stub normally is.

- **Timestamp:** the compile time is **2026-09-15 05:18:38 UTC** in both tools. Timestamps can be forged, but this one fits VirusTotal first seeing the sample hours later.

Taken together, these results suggest that this was a small, normal-looking Windows Forms app wrapped around roughly 1 MB of near-random data stored as a resource. That is the profile of a loader carrying something hidden, and not a finished stealer. The next job was to find out what that resource contains and which code touches it.

Portable Executable Info ⓘ

.NET Details

Module Version Id	dee78ce6-ebc7-4e84-bd94-b06450c1adf9
TypeLib Id	a81f3d5e-792b-4c92-b58a-12e4c89a07d3

Header

Target Machine	Intel 386 or later processors and compatible processors
Compilation Timestamp	2026-09-15 05:18:38 UTC
Entry Point	1117478
Contained Sections	3

Sections

Name	Virtual Address	Virtual Size	Raw Size	Entropy	MD5	Chi2
.text	8192	1109328	1109504	7.75	625f5c49ea512aa8eb8f6bed5f71db99	1131697.62
.rsrc	1122304	1544	2048	3.42	cdfa86dc2515a9dd0b2d75ba7f576e0a	164272
.reloc	1130496	12	512	0.08	dc8bdaaff5cda8021b7957ec20c340d3	128522

Imports

+ mscoree.dll

2.5 Strings

I expected strings to reveal domains, file paths, API names and error messages. But 2.4 had already suggested that most of the file is a large, random-looking resource, so I went in expecting a lot of noise.

FLOSS

For the strings search, I used FLARE FLOSS**.** FLOSS can do more than the Windows strings utility. It can also recover strings that are built or decoded at runtime. For this file it told me plainly that it can't: **.NET string extraction isn't supported**, **it doesn't deobfuscate .NET strings**, and so stack, tight and decoded strings were disabled. That left plain static extraction only, which found 11,906 strings (11,503 ASCII, 403 UTF-16LE). I redirected the output to a text file because of the volume. This is a dead end worth recording, because it means FLOSS's "deobfuscated strings" feature told me nothing here.

```
C:\Users\vboxuser
λ FLOSS.exe C:\Users\vboxuser\Desktop\Malware\412bad1c87f5d4e8c3a6955e942628b9615931cc2399c2b316326d15e3a84775.exe > C:\Users\vboxuser\Desktop\Malware\malware.txt
WARNING: floss: .NET language-specific string extraction is not supported yet
WARNING: floss: FLOSS does NOT attempt to deobfuscate any strings from .NET binaries
INFO: floss: disabled string deobfuscation
INFO: floss: extracting static strings
INFO: floss: finished execution after 0.34 seconds
INFO: floss: rendering results
```

FLOSS states it cannot deobfuscate .NET strings, so only static strings were extracted.

```
malware.txt - Notepad
File  Edit  Format  View  Help

FLARE FLOSS RESULTS (version v3.1.1-0-g3cd3ee6)

+-----------------------+-------------------------------------------------------------------------------------------------------+
| file path             | C:\Users\vboxuser\Desktop\Malware\412bad1c87f5d4e8c3a6955e942628b9615931cc2399c2b316326d15e3a84775.exe |
| identified language   | dotnet (version unknown)                                                                              |
| extracted strings     |                                                                                                       |
|  static strings       | 11906 (67377 characters)                                                                              |
|   language strings    |     0 (    0 characters)                                                                              |
|  stack strings        | Disabled                                                                                              |
|  tight strings        | Disabled                                                                                              |
|  decoded strings      | Disabled                                                                                              |
+-----------------------+-------------------------------------------------------------------------------------------------------+


 FLOSS STATIC STRINGS (11906)


+-------------------------------------+
| FLOSS STATIC STRINGS: ASCII (11503) |
+-------------------------------------+

!This program cannot be run in DOS mode.
.text
`.rsrc
@.reloc
```

Sorting through them, they fell into three groups.

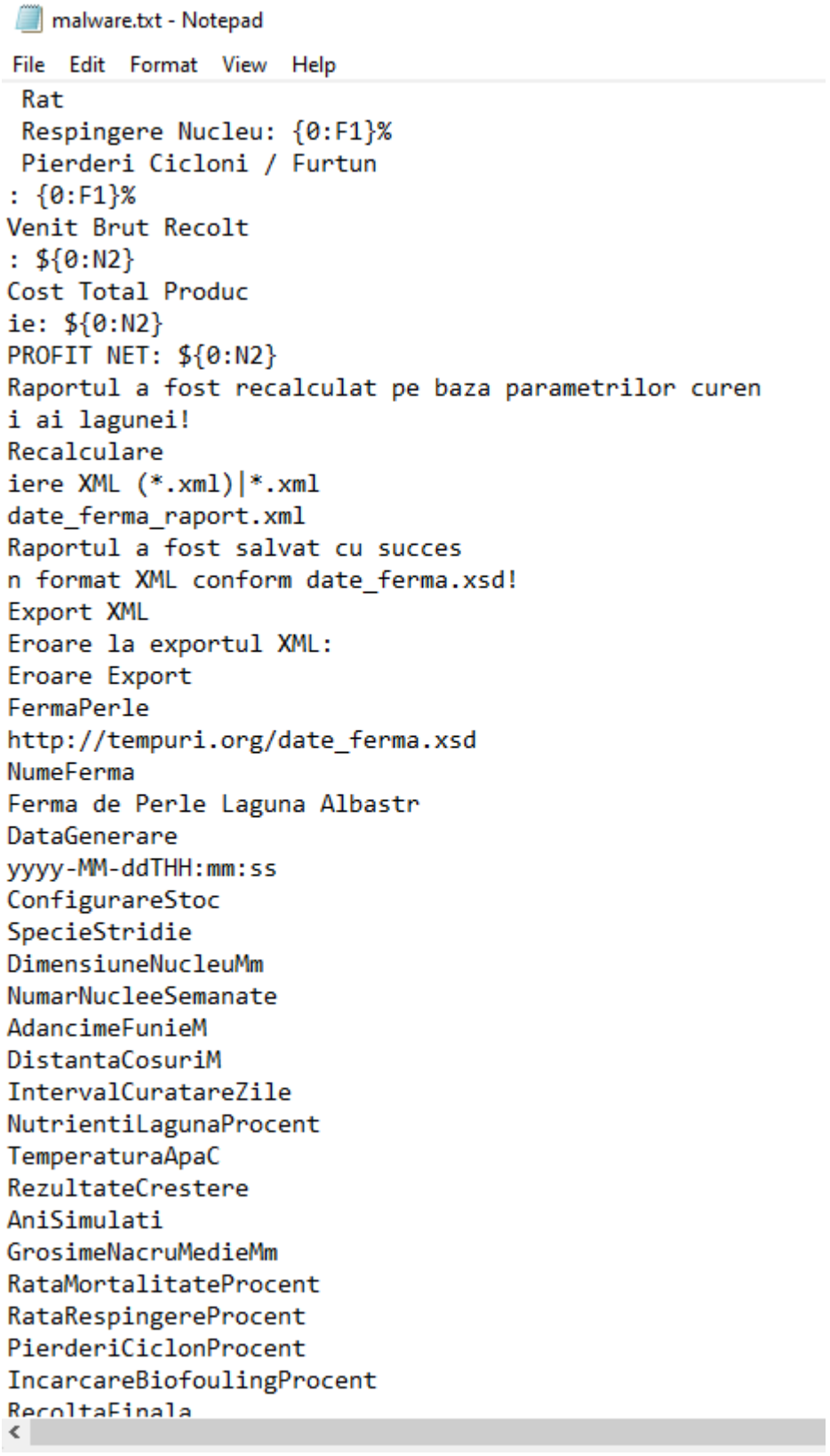
1. **Noise.** The output contains thousands of short, meaningless fragments in long runs. In between them are the markers IHDR, sRGB, gAMA, pHYs, IDAT and IEND, next to System.Drawing.Bitmap. Those are the building blocks of PNG images. So the resource I found in 2.4 holds PNG images, and the fragments are their compressed pixel data. That explains why 92% of the file reads as random.

```
QSystem.Drawing, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a
System.Drawing.Bitmap
Data
QSystem.Drawing, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a
System.Drawing.Bitmap
Data
IHDR
sRGB
gAMA
6IDATHK
IEND
QSystem.Drawing, Version=4.0.0.0, Culture=neutral, PublicKeyToken=b03f5f7f11d50a3a
System.Drawing.Bitmap
Data
IHDR
gAMA
\tpHYs
IDATx^
```

2. **The decoy app.**

   Most of the readable text is in Romanian: “Ferma de Perle” (pearl farm), “Cultivare & Simulare Perle de Laguna” (lagoon pearl cultivation and simulation), “Raport Recolta” (harvest report), species such as *Pinctada margaritifera*, and form names like FormaLaguna, FormaStoc, FormaCrestere. It’s a fully worked-out pearl-farm simulator, so the file survives a casual look.

   One more detail: PEStudio flagged http://tempuri.org/date_ferma.xsd as a URL. That’s Microsoft’s placeholder XML namespace, used by the app’s XML import/export, so it’s a false positive and not infrastructure.


malware.txt - Notepad
File Edit Format View Help
Rat
Respingere Nucleu: {0:F1}%
Pierderi Cicloni / Furtun
: {0:F1}%
Venit Brut Recolt
: ${0:N2}
Cost Total Produc
ie: ${0:N2}
PROFIT NET: ${0:N2}
Raportul a fost recalculat pe baza parametrilor curen
i ai lagunei!
Recalculare
iere XML (*.xml)|*.xml
date_ferma_raport.xml
Raportul a fost salvat cu succes
n format XML conform date_ferma.xsd!
Export XML
Eroare la exportul XML:
Eroare Export
FermaPerle
http://tempuri.org/date_ferma.xsd
NumeFerma
Ferma de Perle Laguna Albastr
DataGenerare
yyyy-MM-ddTHH:mm:ss
ConfigurareStoc
SpecieStridie
DimensiuneNucleuMm
NumarNucleeSemanate
AdancimeFunieM
DistantaCosuriM
IntervalCuratareZile
NutrientiLagunaProcent
TemperaturaApaC
RezultateCrestere
AniSimulati
GrosimeNacruMedieMm
RataMortalitateProcent
RataRespingereProcent
PierderiCiclonProcent
IncarcareBiofoulingProcent
RecoltaFinala

3. **Strings that don't belong in a pearl-farm app.** These mattered most:

String(s)	What it is	Why it matters
CompressionInspiredExtractor, Extract, ComputeCompressionMetrics	Class and method names	Neither has any reason to exist in a farm simulator
PixelByteSequence, PixelByteEnumerator, GetPixel, Bitmap, _bitmap, _cachedPixel, _byteLimit, _totalBytes	Names inside that class	Code that reads bytes out of an image's pixels. That fits an image-based payload, possibly steganography (hiding data inside an image), which is not confirmed in 2.7
~***111***111~6551546F~***111***111~6F6578~***111***111~ and a standalone ~***111***111~	A delimiter plus hex tokens	The tokens decode to eQTo and oex (verified)
get_eQTo, and eQTo / Menu / Fee beside PearlFarm.Properties.Resources	Resource accessors and names	eQTo is the only name that doesn't fit the farm vocabulary, and it matches the decoded token
GetObject, get_ResourceManager	Resource lookup	Fetching a resource by its name
GetExportedTypes, GetMethods, InvokeMember, BindingFlags, LateBinding, LateCall, and the literal Invoke	Reflection	Finding and calling code at runtime, which is how a freshly loaded assembly is usually run
ABABABCDABABCDEFABABCDABGHIJABAB with the field BCE73E6E…F985	A 32-byte block	The SHA-256 of that exact string is that field name, so it's the byte array in ComputeCompressionMetrics. It's repeating filler, not data

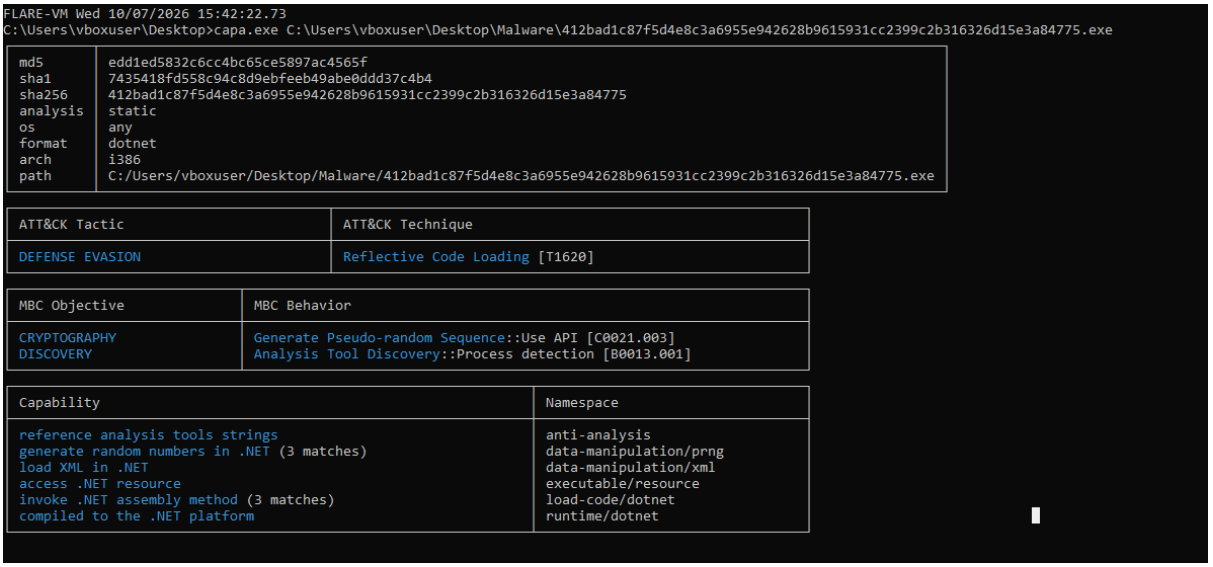
There were no IP addresses or domains (apart from the harmless tempuri.org), no SMTP, FTP or Telegram references, no browser or credential paths, no registry Run keys, and no keylogging APIs. Agent Tesla needs all of those, so they're either hidden or in another file. That fits the hypothesis from 2.4 that this is only a loader. It doesn't prove it.

Strings didn't show what the malware does, but they showed how it hides it: a decoy app, an image-based extractor, and an encoded resource name. My next steps were to run capa for an automated second opinion (2.6), then decompile the loader and follow eQTo in dnSpy (2.7). I also used an AI model to help me interpret the strings.

2.6 Capability Triage with capa

Strings told me how the file hides things, but not what the code was built to do. capa reads the program itself and compares it against a large library of known behavior patterns. It then summarizes the capabilities it finds and maps them to MITRE ATT&CK, so it's a quick way to triage a file without reading all the code. Its output is a set of leads. I treated every line as a hypothesis to confirm, not a conclusion.

I ran capa against the sample. Its header identified the file as a 32-bit .NET executable and listed the same MD5 and SHA-256 as the earlier tools, plus the SHA-1, which matches VirusTotal's.



capa's static analysis of the sample: one ATT&CK technique, two MBC behaviors, and six capabilities.

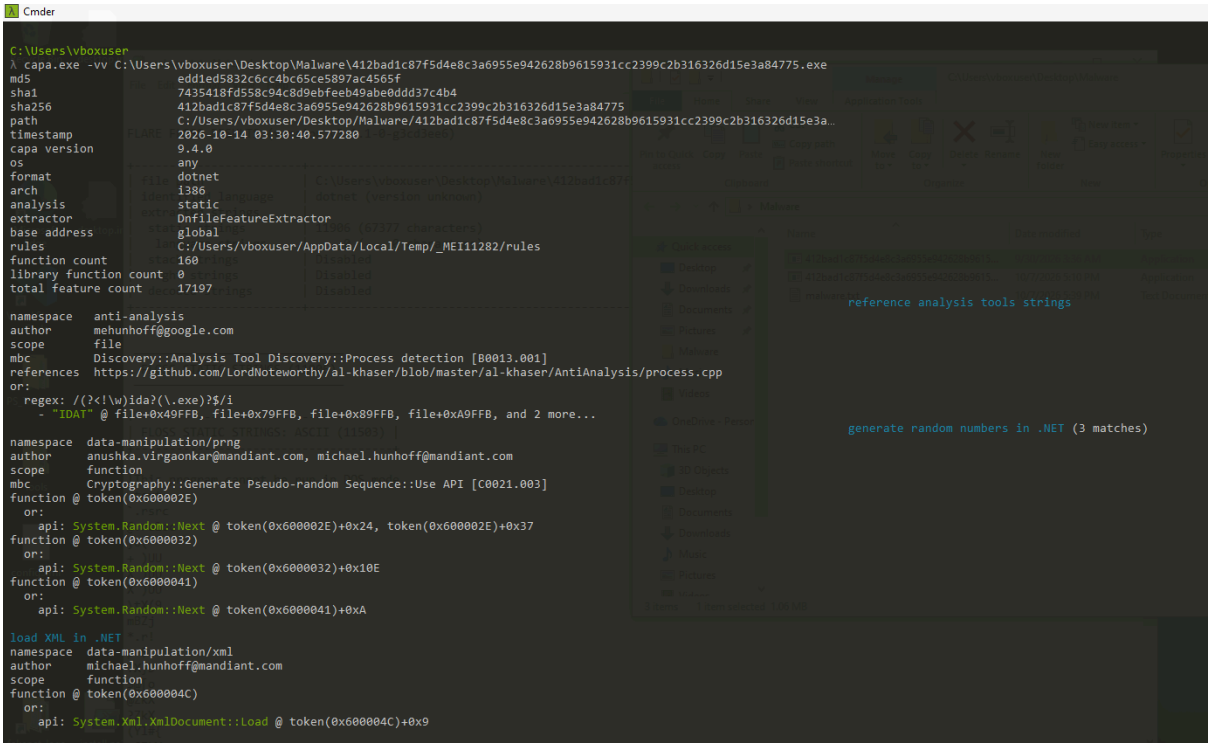
Capa finding	Namespace	My reading
Reflective Code Loading (ATT&CK T1620, labeled Defense Evasion by capa; Stealth in ATT&CK v20)	load-code/dotnet	The most important line. Running code from memory instead of from a file on disk is a loader technique. This is the ATT&CK tag on the "invoke .NET assembly method" rule below, so it is not a separate finding
Access .NET resource	executable/resource	Reads data out of the embedded resources, which matches the large resource from 2.4
Invoke .NET assembly method (3 matches)	load-code/dotnet	Calls into assembly code by reflection, which fits the reflection strings from 2.5

Reference analysis tools strings (MBC B0013.001, process detection)	anti-analysis	Suggests the file looks for analysis tools. Unconfirmed, see below
Generate random numbers in .NET (3 matches) (MBC C0021.003)	data-manipulation/prng	Looks like a false lead, see below
Load XML in .NET	data-manipulation/xml	Matches the decoy app's XML import/export
Compiled to the .NET platform	runtime/dotnet	Confirms what earlier tools already said

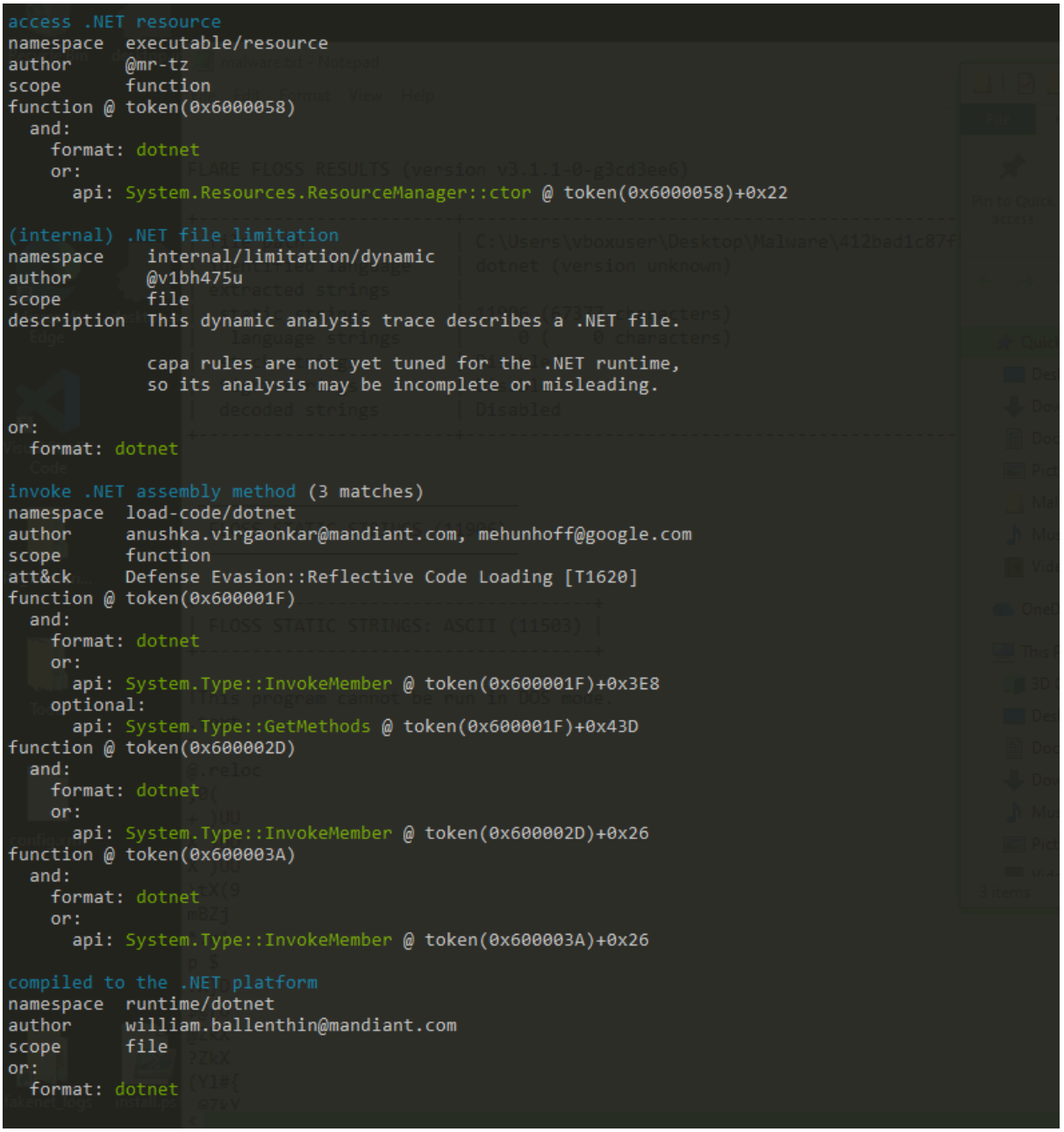
Of these six findings, only a few say anything about the file's real purpose.

- **The loader signals are the ones that matter.** Accessing a resource and invoking an assembly method, together with reflective code loading, form a chain: fetch embedded data, turn it into runnable code, call it. That agrees with what 2.4 and 2.5 suggested from entirely different evidence, which raised my confidence, although capa alone is weak evidence for obfuscated .NET code.

I re-ran capa in verbose mode (-vv) so I could see exactly what each rule matched. It analyzed 160 functions, and it warns that its rules are not tuned for .NET. I treated every match as a lead.



capa -vv output with the exact strings and function tokens behind each match.



```
access .NET resource
namespace  executable/resource
author     @mr-tz
scope      function
function @ token(0x6000058)
  and:
    format: dotnet
    or:
      api: System.Resources.ResourceManager::ctor @ token(0x6000058)+0x22

(internal) .NET file limitation
namespace    internal/limitation/dynamic
author       @v1bh475u
scope        file
description  This dynamic analysis trace describes a .NET file.

             capa rules are not yet tuned for the .NET runtime,
             so its analysis may be incomplete or misleading.

or:
  format: dotnet

invoke .NET assembly method (3 matches)
namespace  load-code/dotnet
author     anushka.virgaonkar@mandiant.com, mehunhoff@google.com
scope      function
att&ck     Defense Evasion::Reflective Code Loading [T1620]
function @ token(0x600001F)
  and:
    format: dotnet
    or:
      api: System.Type::InvokeMember @ token(0x600001F)+0x3E8
    optional:
      api: System.Type::GetMethods @ token(0x600001F)+0x43D
function @ token(0x600002D)
  and:
    format: dotnet
    or:
      api: System.Type::InvokeMember @ token(0x600002D)+0x26
function @ token(0x600003A)
  and:
    format: dotnet
    or:
      api: System.Type::InvokeMember @ token(0x600003A)+0x26

compiled to the .NET platform
namespace  runtime/dotnet
author     william.ballenthin@mandiant.com
scope      file
or:
  format: dotnet
```

Most of what it reported turned out to be noise. The “reference analysis tools strings” rule matched the four-letter marker IDAT at six offsets inside the resource, which is PNG image data, not a reference to an analysis tool. “Generate random numbers” and “load XML” match the pearl-farm simulator’s random events and XML import. “Access .NET resource” is the standard generated resource accessor.

One result held up. The “invoke .NET assembly method” rule (ATT&CK T1620, Reflective Code Loading) matched three functions that call InvokeMember. Two look like the double-buffering helper. The third, token 0x0600001F, is a large method that also calls GetMethods, and in dnSpy it is MainForm.InitializeComponent(), the method that calls the bitmap extractor. I count T1620 as a hypothesis from a tool, not a confirmed behavior.

capa did not show several behaviors that VirusTotal’s sandbox tagged, such as WMI calls, BIOS and network-adapter checks, long sleeps and persistence. capa only sees what is

visible in the file itself, while those tags come from runtime observation. They most likely belong to the hidden payload after it loads. I'm recording them as things to look for in dynamic analysis, and I won't treat their absence here as evidence that they're absent from the sample's behavior.

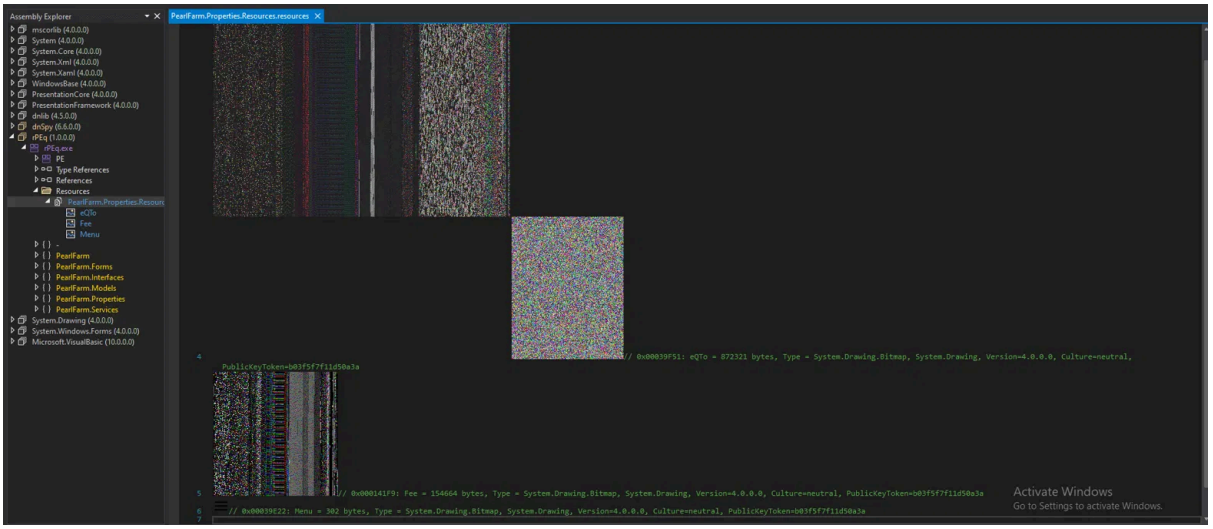
capa raised my confidence that the file is a loader and gave me one ATT&CK technique to carry into my findings (T1620), though it confirmed neither. It also showed the limits of a rule-based tool on obfuscated code, since two of its six findings were noise. My next step was to stop inferring and read the code itself.

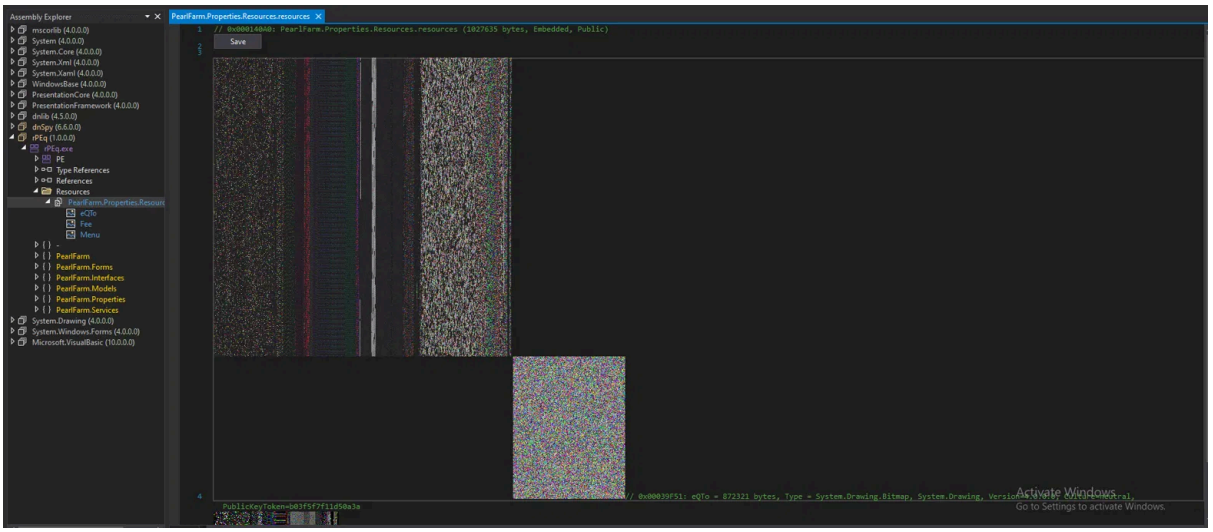
2.7 Decompilation with dnSpy

The sample is .NET, so I opened it in dnSpy, which turns the compiled code back into readable C#. I used it only to read. dnSpy has a built-in debugger (the Start button), but pressing it would run the sample and end the static phase, so I left it alone.

The Resources folder holds one file, PearlFarm.Properties.Resources.resources, with three images:

Resource	Size	Share of the .exe	Notes
eQTo	872,321 B	78.4%	Renders as static, not a picture. Its name matches the hex-decoded string from 2.5
Fee	154,664 B	13.9%	Also noise-like. Purpose unknown
Menu	302 B	~0%	Small, probably an ordinary icon

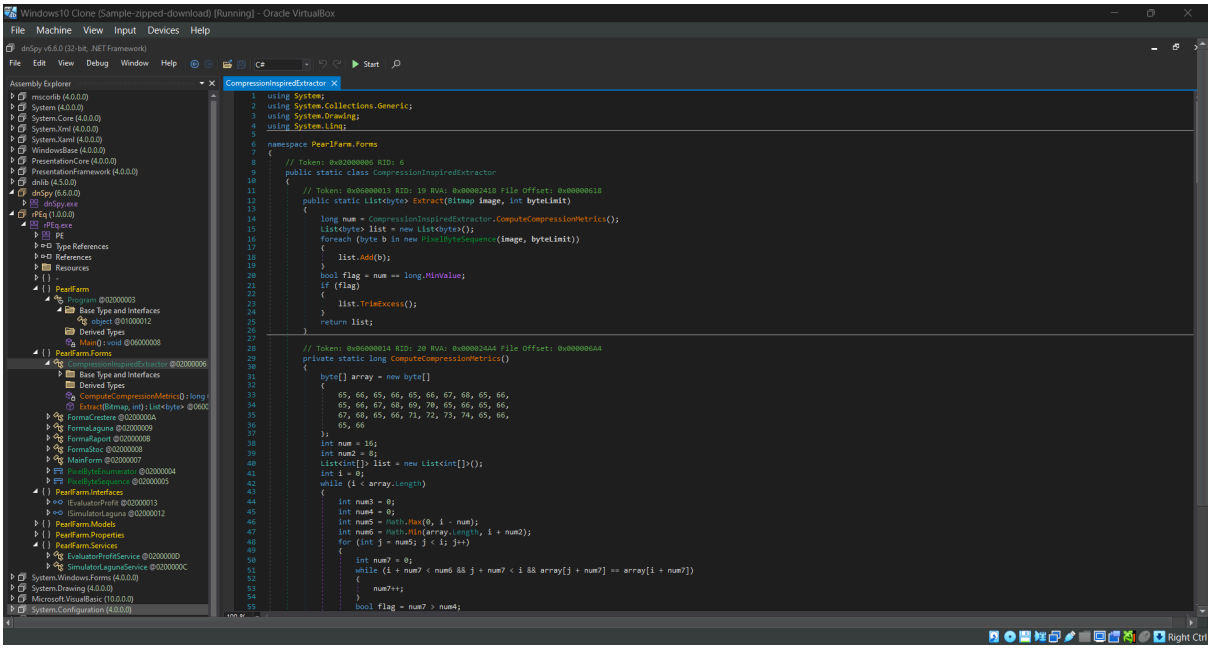




“The Resources folder lists three bitmaps: eQTo, Fee and Menu.” and “eQTo is 872,321 bytes and displays as static.”

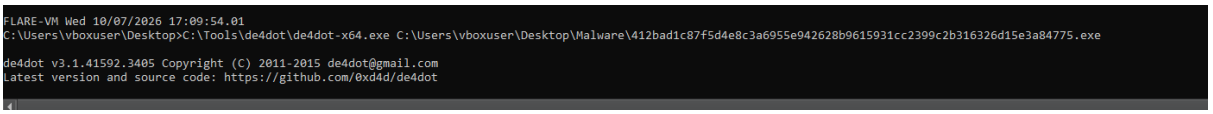
A normal program’s images are logos and buttons. These are large and random-looking, so I treated them as data wearing an image’s format. I put that down as a hypothesis until I’d seen the code that reads them.

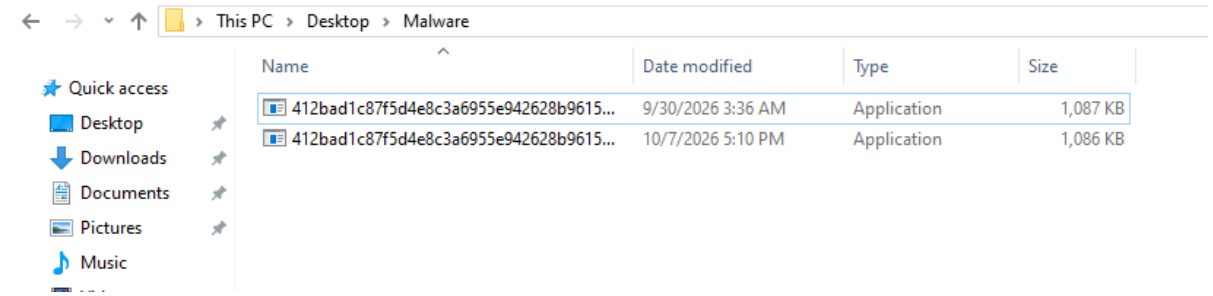
In CompressionInspiredExtractor, Extract(Bitmap, int) copies bytes from a PixelByteSequence (a pixel reader) into a list, up to a byte limit, and returns the list. Before that it calls ComputeCompressionMetrics(), the large block of about 259 lines, but the result is only compared with long.MinValue to decide whether to call TrimExcess(), which affects memory capacity and not the returned bytes. So the metrics call cannot change what Extract() returns. The method itself runs a fixed 32-byte array through textbook compression routines (LZ77, Huffman, Shannon entropy, arithmetic coding, Burrows–Wheeler, move-to-front and LZW) and combines the results into one value that nothing uses. It appears to exist only to make the file look like a legitimate compression tool. I used an AI model to help me read this long method.



"Extract() returns bytes read from the bitmap; the metrics call can't change them."

I also ran the de4dot tool to produce a cleaned copy of the file, but it made little difference, which fits the plain, readable names seen in dnSpy.





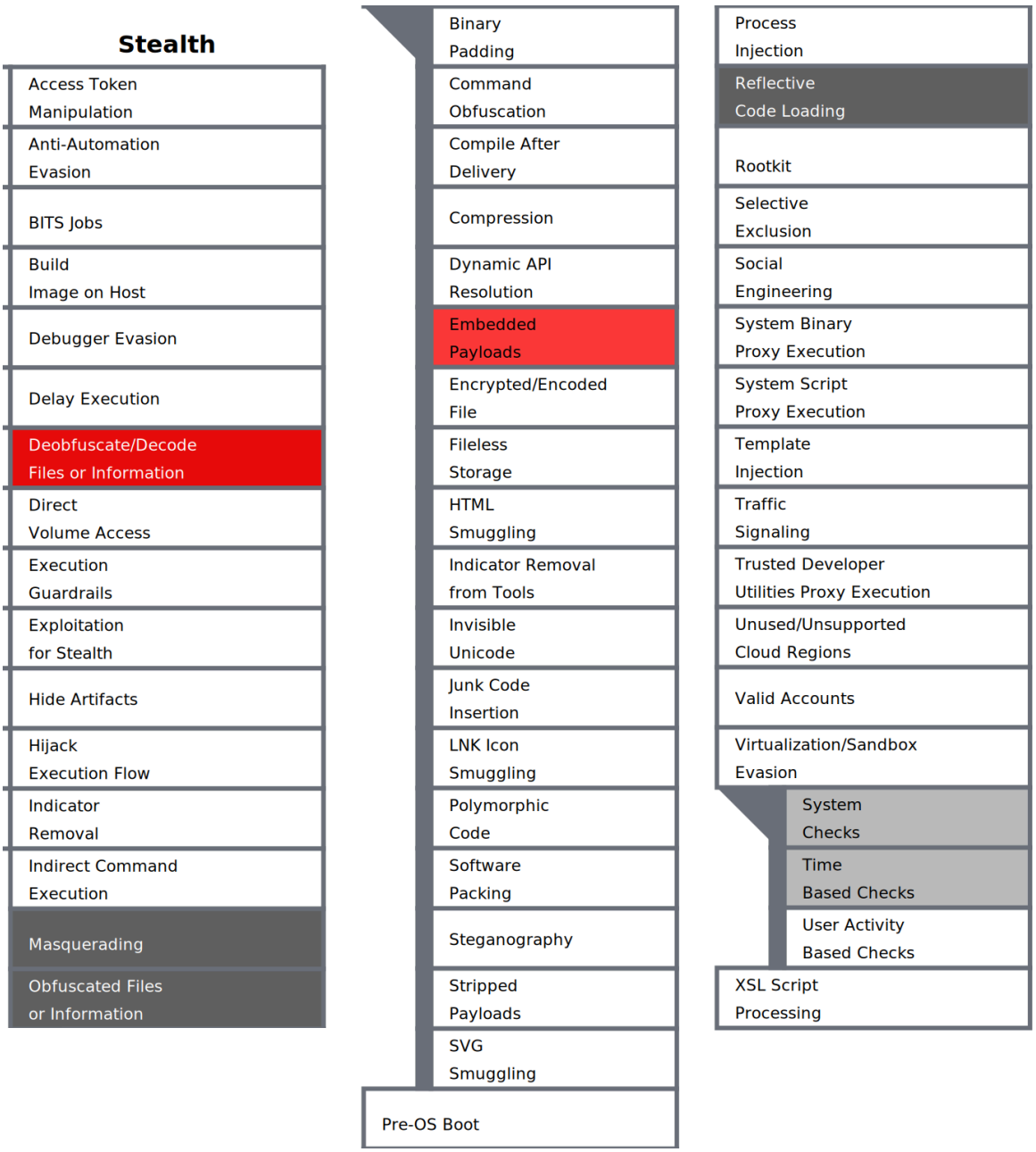
The screenshot below shows the cleaned file, which differed only slightly from the original.

Limits of this analysis. I did not complete the path from the image to a running payload. I did not confirm which color channels PixelByteSequence reads (which separates true steganography from a payload stored directly as pixel values), or what MainForm.InitializeComponent() does with the extracted bytes, so I could not extract or examine the second stage. These are the first questions for the next phase, and they are why the steganography and embedded-payload rows in 2.8 are marked as hypotheses.

2.8 Findings: MITRE ATT&CK Mapping

Tier	Technique	Evidence	Report section
Hypothesis (tool-reported)	T1620 Reflective Code Loading	capa matched three InvokeMember calls; one is in MainForm.InitializeComponent(). Not yet confirmed in the code	2.6, 2.7
Observed	T1027 Obfuscated Files or Information	DIE obfuscation flag, entropy 7.75 on .text, 92% of the file in a resource, hex-encoded resource name	2.2, 2.4, 2.5
Observed	T1036 Masquerading	Fake “PearlFarm” product, company and copyright metadata, plus a working decoy app	2.4, 2.5
Hypothesis (unconfirmed)	T1027.003 Steganography	Extract(Bitmap), PixelByteSequence, GetPixel	2.5, 2.7
Hypothesis	T1027.009 Embedded Payloads	Large noise-like image resources (eQTo, Fee) hold most of the file	2.4, 2.7
Hypothesis	T1140 Deobfuscate/Decode Files or Information	Hex tokens decoding to eQTo, plus the extractor	2.5, 2.7
Reported externally	T1497.001 / T1497.003 (system checks, time-based evasion)	VirusTotal tags: checks-bios, detect-debug-environment, long-sleeps. capa’s analysis-tool hit was a false positive (it matched the PNG marker IDAT), so it doesn’t support these	2.3, 2.6

The figure below shows the Stealth column of the Navigator layer (ATT&CK v20), the tactic where all of the mapped techniques sit. I saved the full layer as a Navigator JSON file so that the colors and comments can be reviewed there.



ATT&CK Navigator layer for rPEq.exe (ATT&CK v20), Stealth column shown in three parts.

2.9 Draft Detection

To turn the findings into something a SOC could use, I wrote two YARA rules. YARA scans files for patterns, so it can hunt for this loader, or close relatives of it, across a disk or a sample collection. For each trait I asked whether it was specific to this loader and unlikely to appear in legitimate software.

Features I used. Four class names that have no reason to exist in a farm simulator (CompressionInspiredExtractor, ComputeCompressionMetrics, PixelByteSequence and PixelByteEnumerator), plus three supporting traits: the project's assembly GUID (which usually survives rebuilds, unlike the module ID), the 32-byte filler array from ComputeCompressionMetrics, and the ~***111***111~ delimiter that appears as UTF-16 text. The rule needs three of the four class names, or the delimiter plus one class name, or the GUID plus the filler array, so no single common string can trigger it.

Features I left out. Generic APIs such as InvokeMember, GetPixel and GetObject, which appear in ordinary Windows Forms and image programs. Short strings such as eQTo and Fee, which match random data. The Romanian application text, which belongs only to this decoy. The imphash, compile time and file hashes: .NET programs share the imphash, and the others change with every build. File hashes belong in an IOC list, not in a rule.

A second, broader rule. The first rule is high confidence for this build. The second looks at structure instead: a .NET executable where the .text section makes up more than 90% of the file and has an entropy above 7.4. This sample scored 99.72% and 7.751, so the thresholds leave a margin. That trait is harder for an author to hide, but legitimate software that bundles compressed content, such as apps merged into a single file, can trigger it. I labeled it low confidence, for hunting only.

The rules:

```
import "pe"
import "math"

rule MAL_DotNet_PearlFarm_ImageLoader
{
    meta:
        description = "PearlFarm decoy .NET loader (payload hidden in bitmap resource)"
        author      = "Samuel Adetola O."
        date        = "2026-10-15"
        sample      = "412bad1c87f5d4e8c3a6955e942628b9615931cc2399c2b316326d15e3a84775"
        confidence  = "high for this build, medium for related builds"
        status      = "draft - pending testing"

    strings:
        $cls1  = "CompressionInspiredExtractor" ascii
        $cls2  = "ComputeCompressionMetrics" ascii
        $cls3  = "PixelByteSequence" ascii
        $cls4  = "PixelByteEnumerator" ascii
        $guid  = "a81f3d5e-792b-4c92-b58a-12e4c89a07d3" ascii wide
        $array = "ABABABCDABABCDEFABABCDABGHIJABAB" ascii
        $delim = "~***111***111~" wide

    condition:
        uint16(0) == 0x5A4D and
        filesize < 5MB and
```

```
        (
            3 of ($cls*) or
            ($delim and 1 of ($cls*)) or
            ($guid and $array)
        )
}

rule HUNT_DotNet_Exe_Dominant_HighEntropy_Text
{
    meta:
        description = "Hunting only: .NET EXE, .text is most of the file and high
entropy"
        author      = "Samuel Adetola O."
        date        = "2026-10-15"
        confidence  = "low - expect false positives from packed or bundled apps"

    condition:
        uint16(0) == 0x5A4D and
        filesize > 500KB and filesize < 5MB and
        pe.imports("mscoree.dll", "_CorExeMain") and
        for any s in pe.sections : (
            s.name == ".text" and
            s.raw_data_size * 10 > filesize * 9 and
            math.entropy(s.raw_data_offset, s.raw_data_size) > 7.4
        )
}
```

2.10 IOC TABLE

Type	Value
SHA-256	412bad1c87f5d4e8c3a6955e942628b9615931cc2399c2b316326d15e3a84775
MD5	edd1ed5832c6cc4bc65ce5897ac4565f
SHA-1	7435418fd558c94c8d9ebfeeb49abe0ddd37c4b4
Filenames seen	rPEq.exe, bB.exe, 5bxr076t.exe, PO 3574842.exe
Metadata	Product “PearlFarm”, company “Pearl Farm Inc.”
Assembly GUID	Not checked (3.4, 3.5)
Resources	eQTo (872,321 B), Fee (154,664 B), Menu (302 B)
Class names	CompressionInspiredExtractor, PixelByteSequence, PixelByteEnumerator, ComputeCompressionMetrics
Strings	~***111***111~; hex tokens 6551546F (“eQTo”) and 6F6578 (“oex”)
Command line	powershell.exe Add-MpPreference -ExclusionPath "<path to the sample>

SECTION 3:

DYNAMIC ANALYSIS REPORT

3.1 Overview and Goals

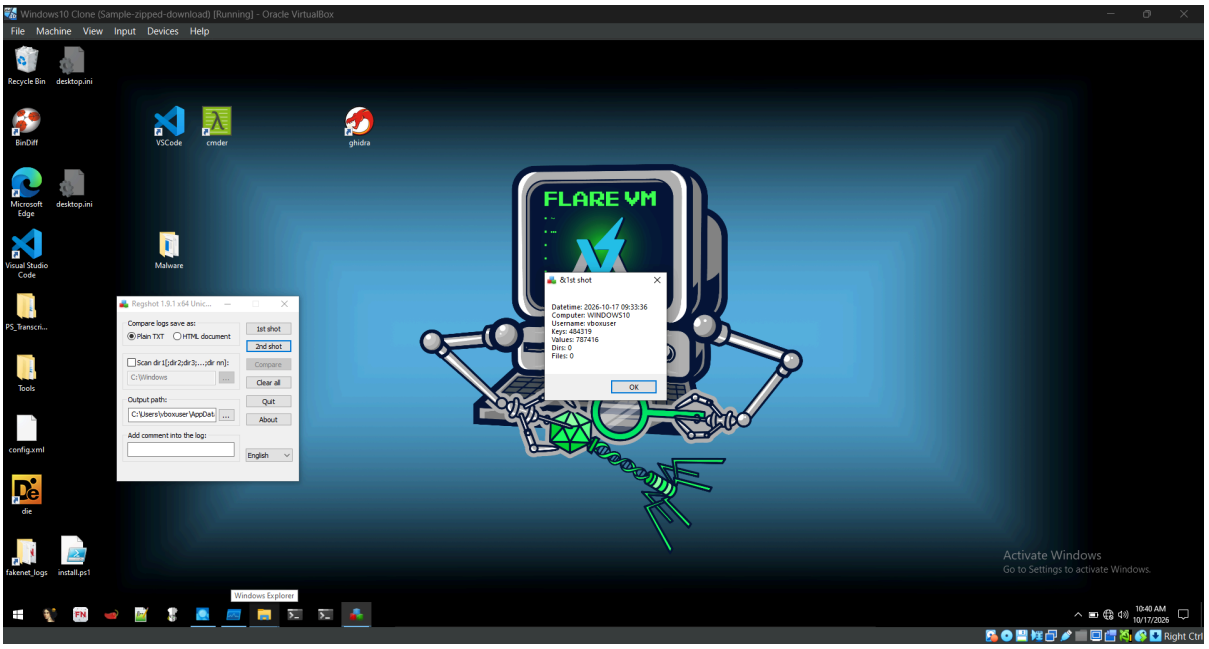
Static analysis showed that the sample is a loader, but it could not show what the loader does when it runs. Dynamic analysis answers that by running the sample inside the isolated lab and watching what happens. I looked at four things: process activity, network activity, registry activity and file activity.

Section 2 left four questions open: does the sample start other processes or load a second stage; does it reach out over the network; does it set up persistence; and does it behave differently inside a virtual machine, as the VirusTotal tags (anti-analysis checks and long sleeps) suggested?

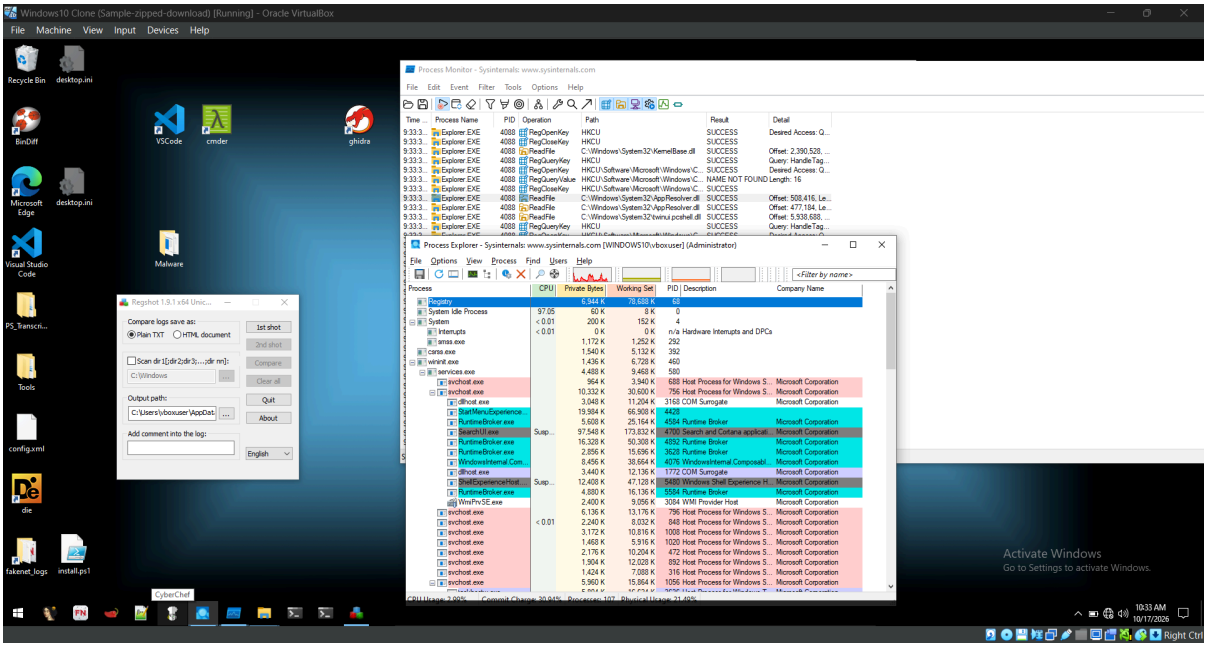
3.2 Setup and Baseline

Before running anything I took a snapshot of the Windows VM, so I could always return to a clean state. I then opened the monitoring tools: Process Monitor (Procmon) to record file, registry and process events, Process Explorer to watch the process tree live, and Regshot to compare the registry before and after. On REMnux I used Wireshark and INetSim to watch the network (see 3.5).

I took Regshot's 1st shot first, so I had a record of the system before the malware ran. It counted 484,319 keys and 787,416 values. It counted 0 files and 0 directories, because I had not ticked Regshot's directory-scan option, so Regshot could not compare the file system. I covered files separately in 3.7.



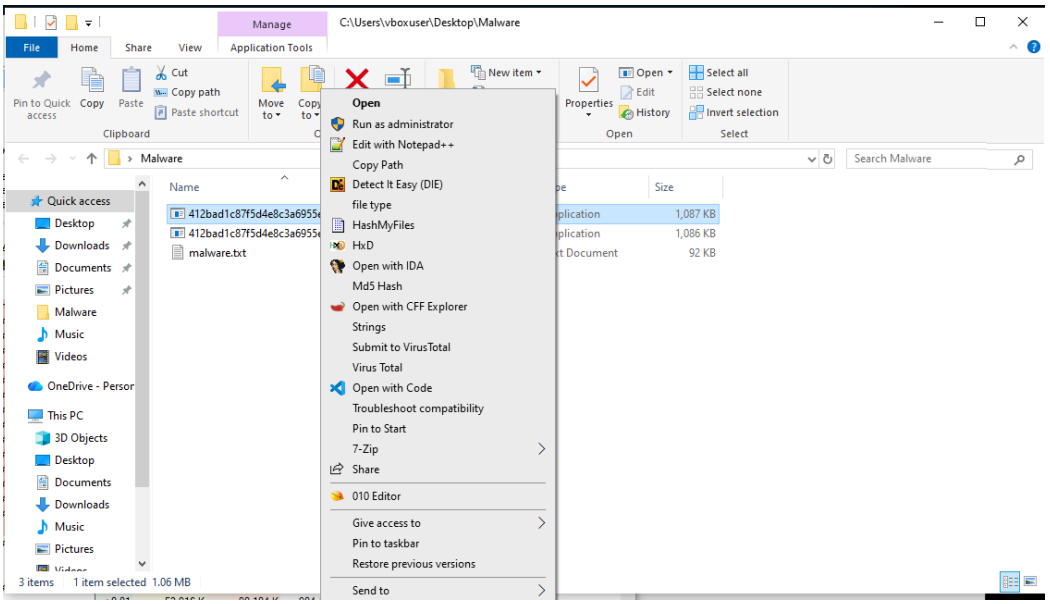
Regshot 1st shot, taken before running the sample (keys 484,319; values 787,416; directories 0; files 0).



Procmon, Process Explorer and Regshot open and ready before execution.

3.3 Execution

I ran the sample by right-clicking it and choosing **Run as administrator**. I kept all of the tools running while it executed, and left the VM running for about an hour before taking the second Regshot shot.



Running the sample with Run as administrator.

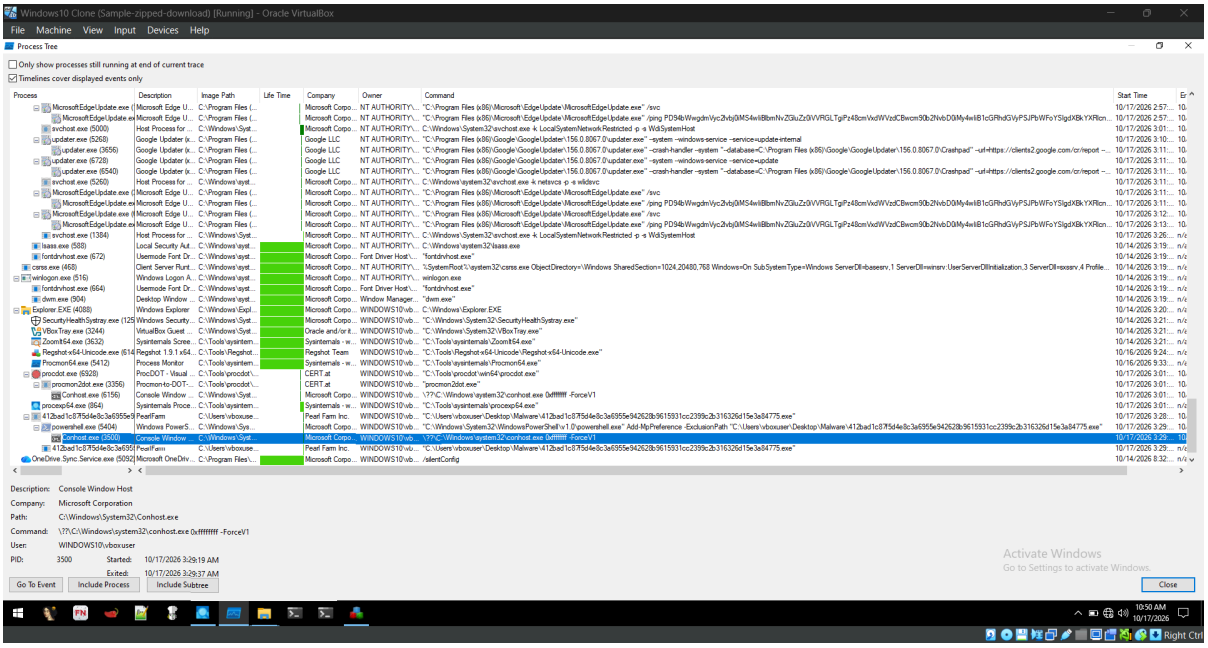
3.4 Process Activity

I opened Procmon's process tree to see what the sample started. The tree showed the sample (shown as "PearlFarm", company "Pearl Farm Inc.", matching the fake metadata from Section 2) starting a child process, powershell.exe, with this command line:

powershell.exe Add-MpPreference -ExclusionPath
"C:\Users\vboxuser\Desktop\Malware\412bad1c…775.exe"

Observation. "Add-MpPreference -ExclusionPath" tells Windows Defender not to scan a path. Here the path is the sample's own file. The console host that PowerShell used (conhost.exe) started at 3:29:19 AM and exited at 3:29:37 AM on the process tree's clock, so the command ran for only about 18 seconds.

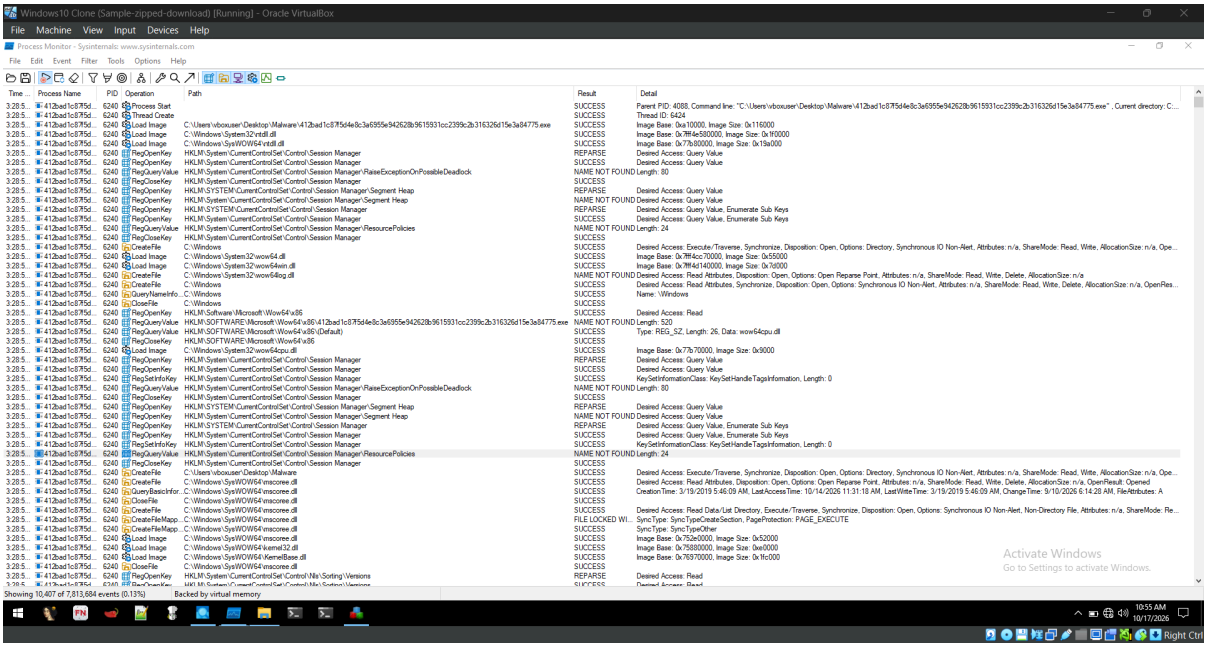
Interpretation. The loader is trying to protect itself from antivirus before it goes further. That is a defense-evasion step, and it fits the loader hypothesis from Section 2. I could not confirm that the command worked: Defender had already been turned off by policy in Section 1.5, and I did not check whether the exclusion was actually added.



Procmon process tree: the sample starts powershell.exe with an Add-MpPreference -ExclusionPath command for its own file; a second copy of the file is also running.

The tree also shows a second copy of the same file running (PID 5992), which started at 3:29:19 AM, the same second as the PowerShell process. The first copy had already exited by then, and the second was still running when I captured the tree (its exit time shows n/a). This looks like the sample relaunching itself after adding the exclusion. I have not yet confirmed which process started it.

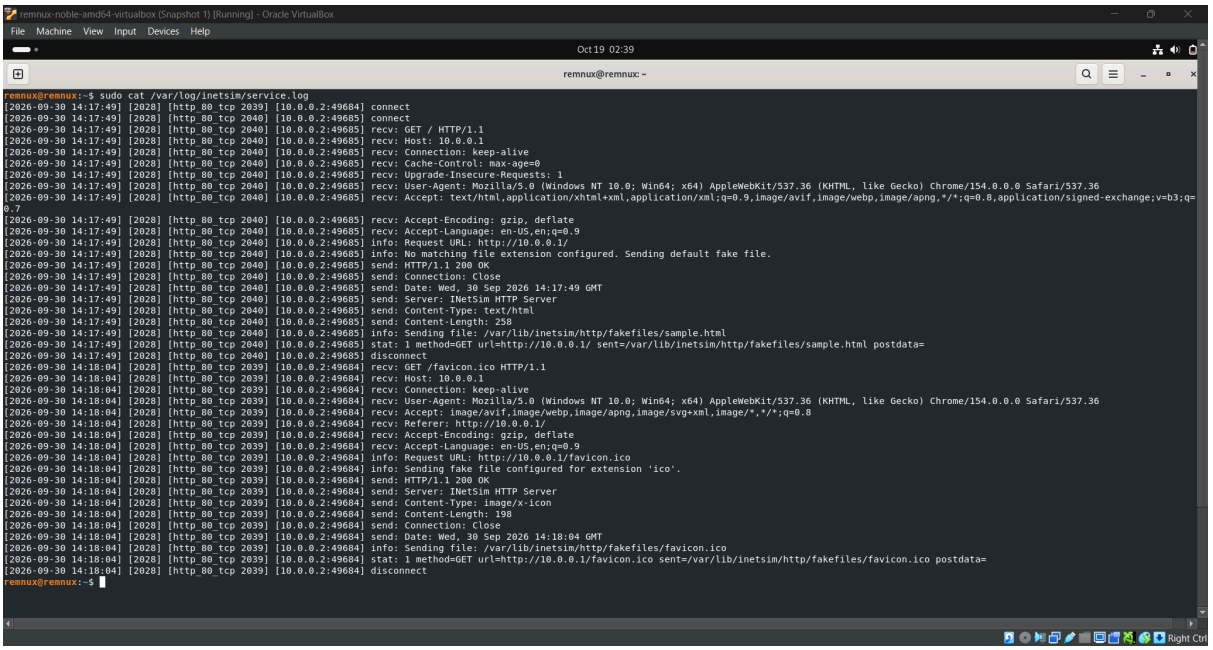
I then filtered Procmon to the sample's first process (PID 6240). The first events are the process starting from Explorer (parent PID 4088), then loading ntdll.dll, the 32-bit WOW64 files and mscoree.dll, the .NET runtime. That matches the 32-bit .NET executable identified in Section 2. The filter showed 10,407 of 7,813,684 events (0.13%).



Procmon filtered to the sample's first process (PID 6240): process start, then ntdll, WOW64 files and mscoree.dll.

3.5 Network Activity

I used REMnux for this, running Wireshark to capture traffic and INetSim to answer DNS, HTTP, SMTP and FTP requests from the Windows VM, so the sample would think it had a network.

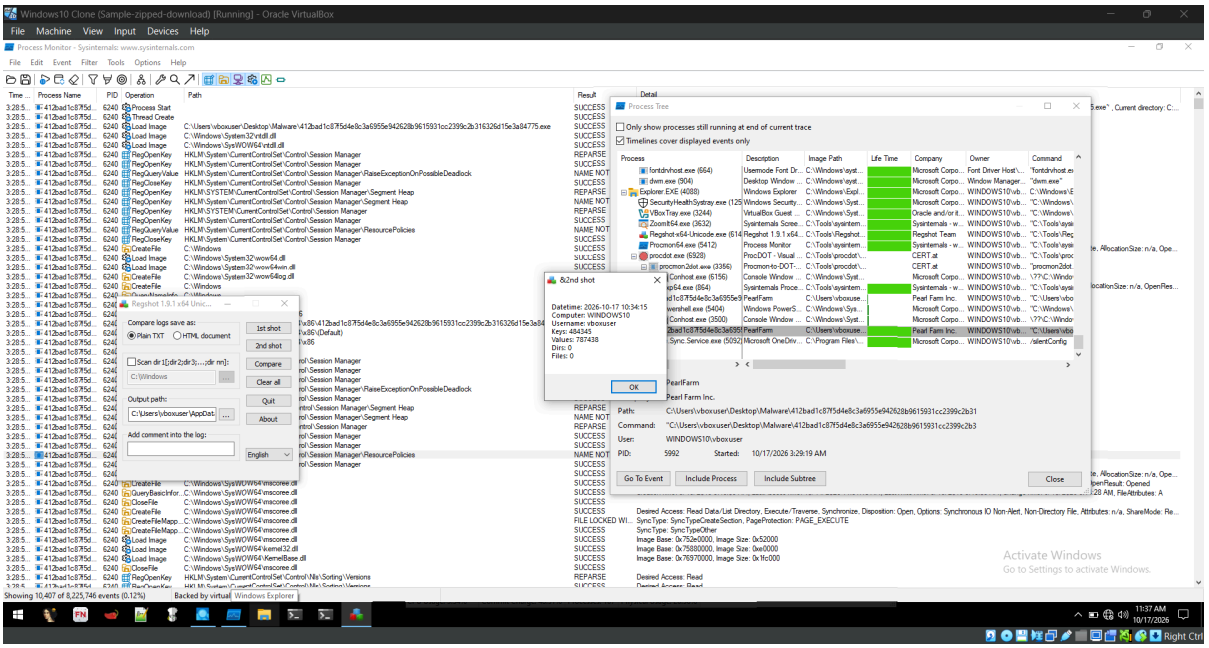


REMnux then crashed. I saved its VM state with Wireshark still running, but the saved state could not be resumed and I had to boot REMnux from scratch. That cleared the temporary folder where Wireshark keeps an unsaved capture, so the capture was lost.

Interpretation. Because the capture was lost, I cannot say whether the sample made DNS lookups or connections during the run. Silence in INetSim's log is not proof that the sample stayed quiet: it may have been waiting out a sleep timer, or it may have stopped after detecting the VM, as the VirusTotal tags suggested. Network behavior is therefore listed as unconfirmed.

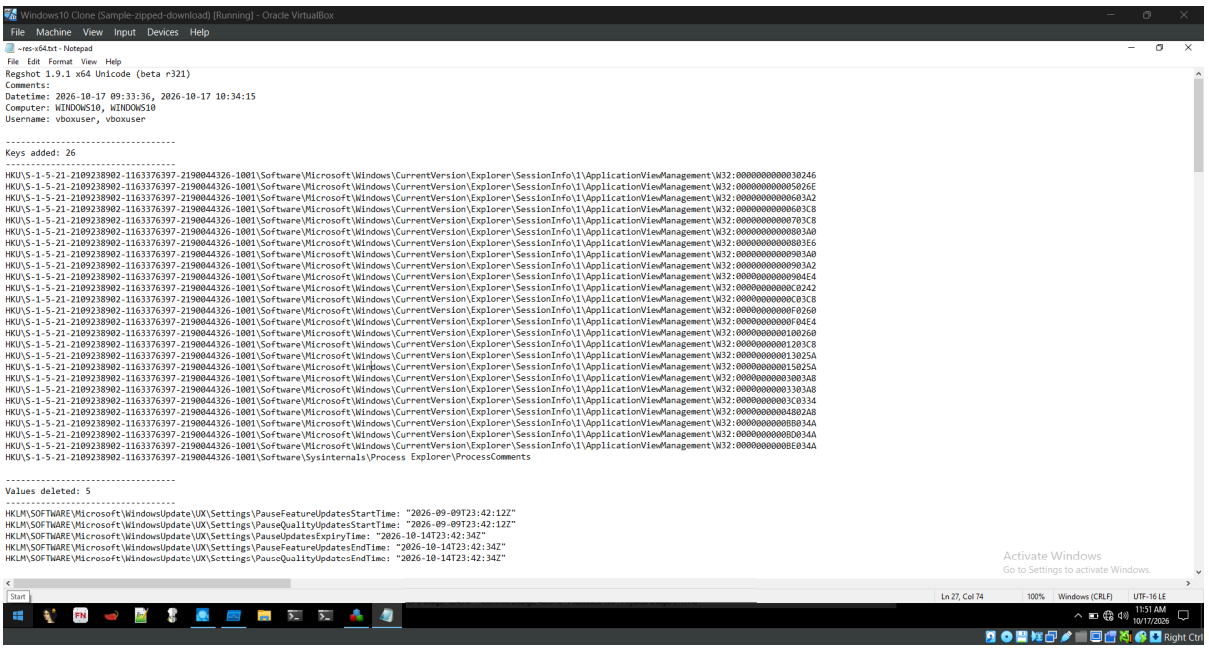
3.6 Registry Activity

After about an hour I took Regshot's 2nd shot and compared it with the first. The second shot counted 484,345 keys and 787,438 values, which is 26 more keys and 22 more values than the baseline.



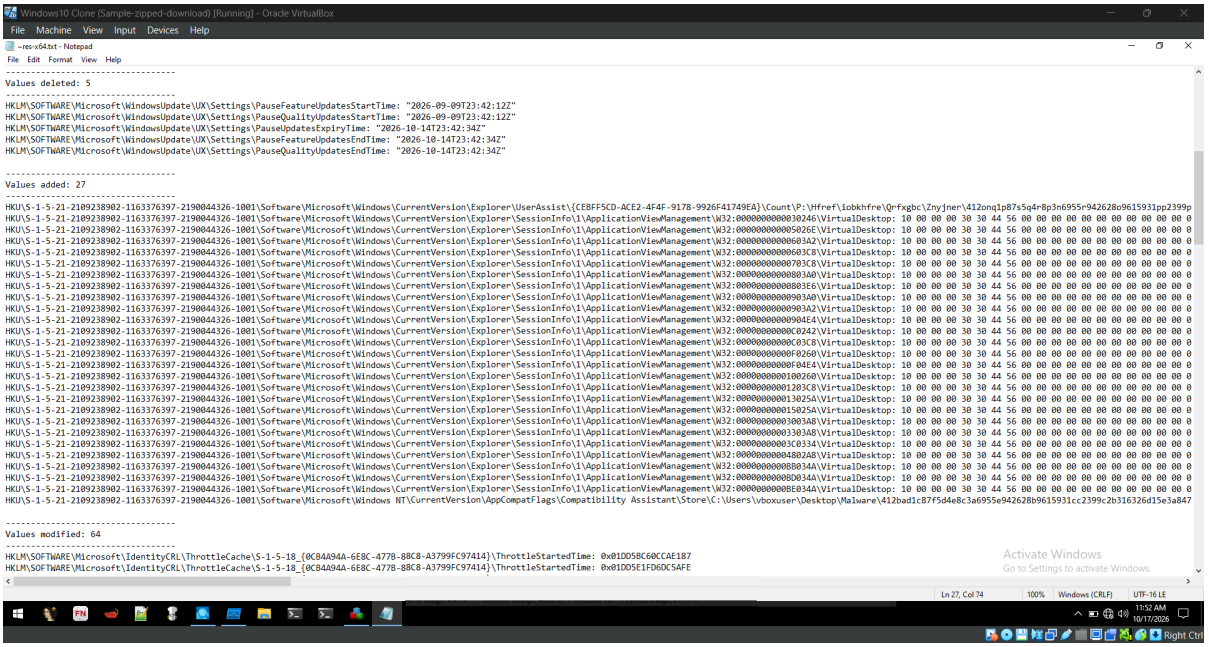
Regshot 2nd shot (keys 484,345; values 787,438), with the process tree open on the second copy of the sample (PID 5992).

The comparison reported 26 keys added, 5 values deleted, 27 values added and 64 values modified. Going through the results:



Regshot comparison: 26 keys added (mostly Explorer window-management entries) and 5 values deleted.

Mostly background noise. The 26 added keys are Explorer's virtual-desktop bookkeeping under ...\Explorer\SessionInfo\1\ApplicationViewManagement, plus one key left by Process Explorer itself (Sysinternals\Process Explorer\ProcessComments). The 5 deleted values are Windows Update pause settings. None of this points to the sample.



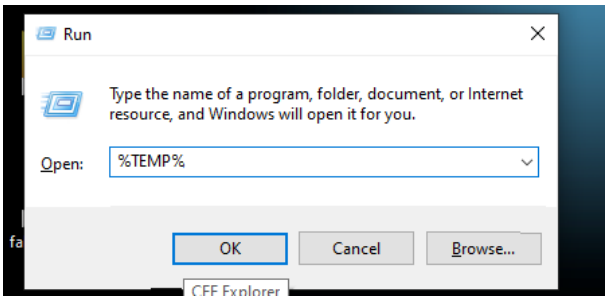
Regshot comparison: 27 values added, including a UserAssist entry for the sample, and 64 values modified.

Evidence that the sample ran. Two of the 27 added values mention the sample. One is a UserAssist entry. UserAssist stores names in ROT13, and the key name decodes to the sample's own path, C:\Users\vboxuser\Desktop\Malware\412bad1c…. It records that Explorer launched the program. The other is an entry under AppCompatFlags\Compatibility Assistant\Store with the same path, which Windows also writes when a program is run.

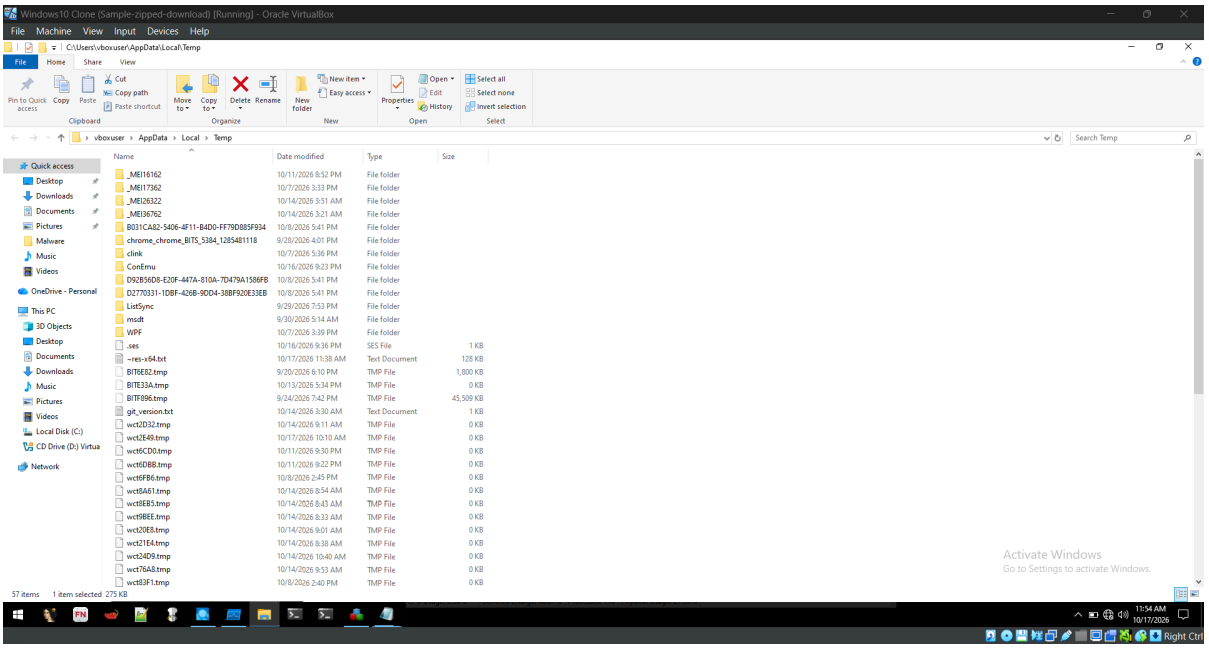
No persistence found. I searched specifically for new values under HKCU\Software\Microsoft\Windows\CurrentVersion\Run and RunOnce, and found none. I did not review the 64 modified values in detail.

3.7 File Activity

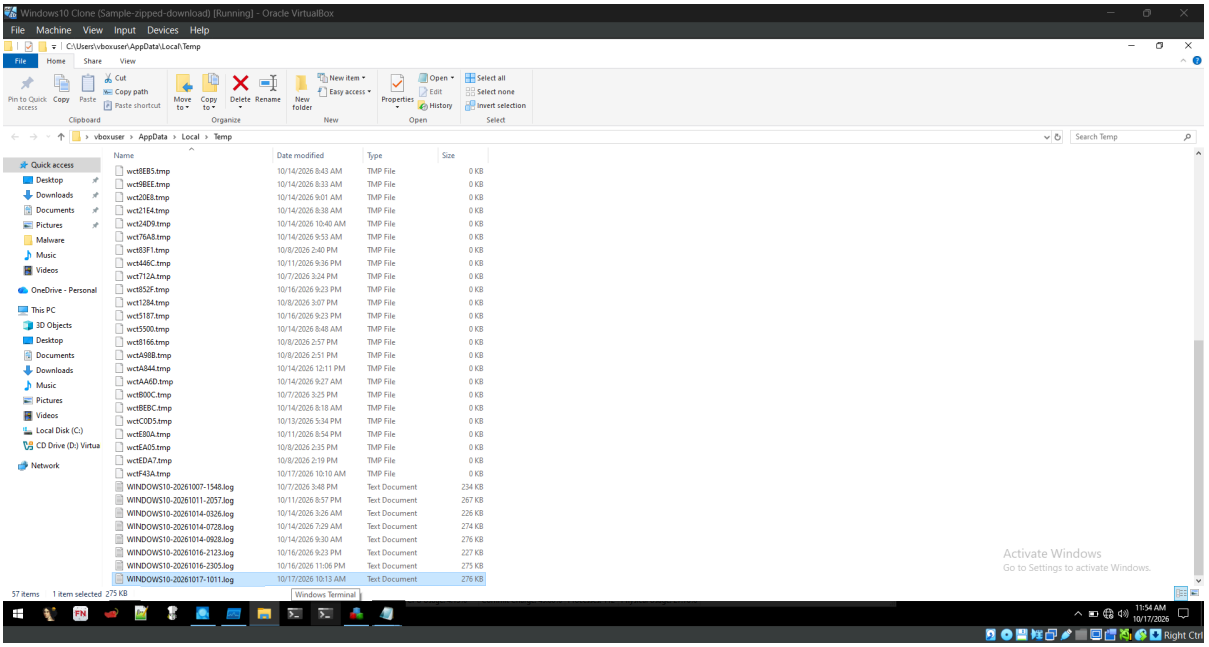
Because Regshot did not scan the file system, I checked the places malware usually writes to by hand. I looked in %TEMP%, in the user Startup folder (shell:startup), in the all-users Startup folder (shell:common startup) and in C:\Users\vboxuser\AppData\Roaming.



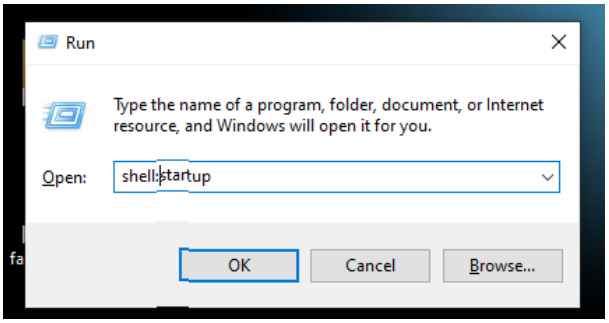
Opening %TEMP%.



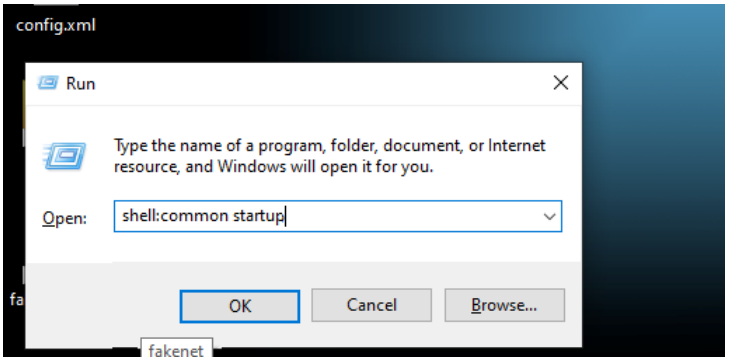
Contents of %TEMP%: nothing from the sample. The newest items are Regshot's output file and FLARE VM log files.



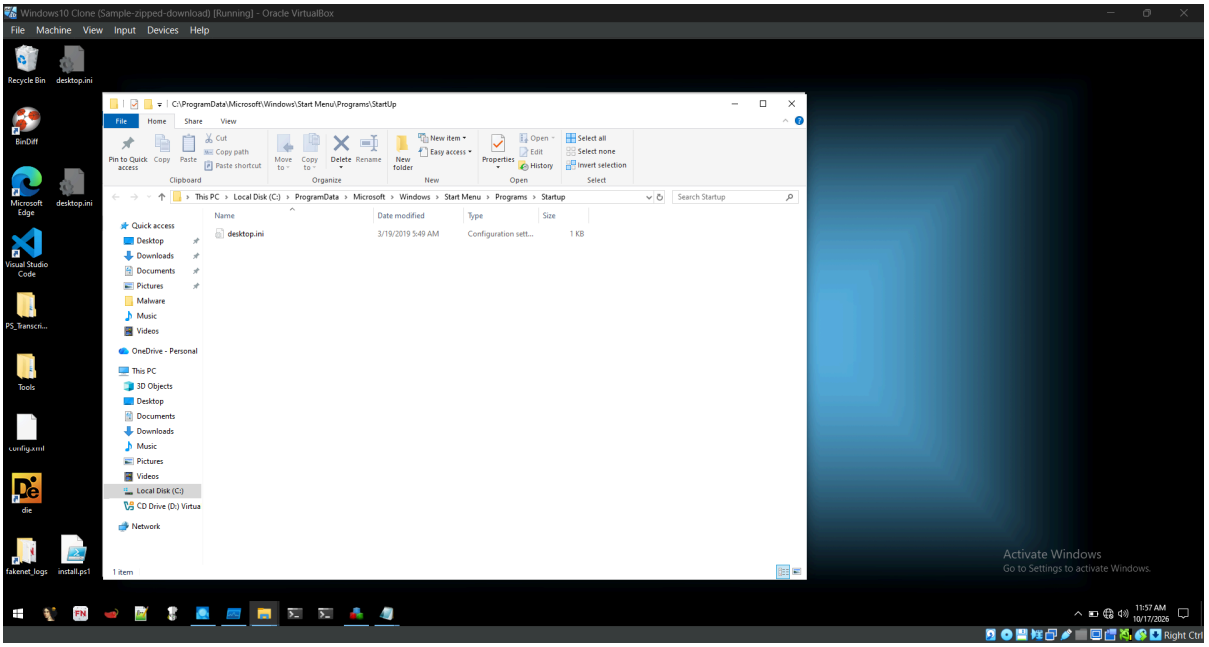
Contents of %TEMP% (bottom of the list).



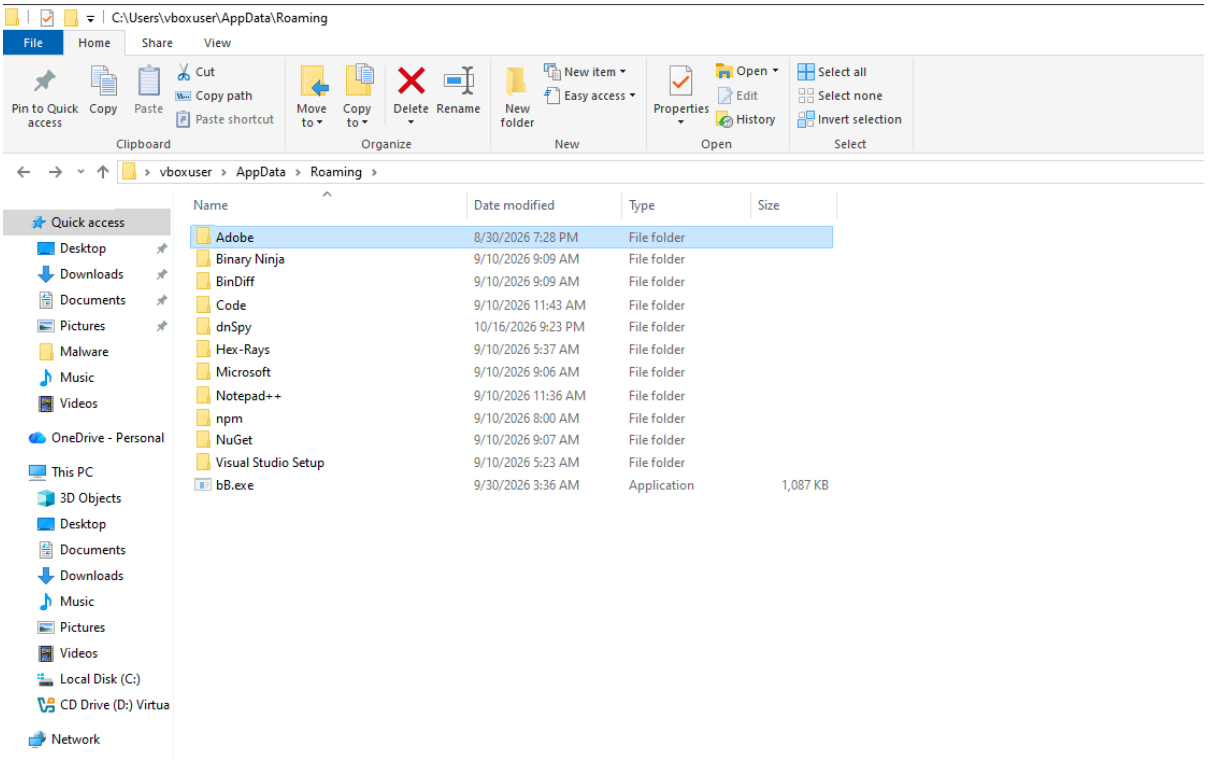
Opening shell:startup.



Opening shell:common startup.



The all-users Startup folder contains only desktop.ini.



C:\Users\vboxuser\AppData\Roaming: nothing new.

I found nothing new in any of these places.

3.8 Findings and Limits

Category	Finding	Evidence
Observed	Sample runs under the fake product name “PearlFarm”	Process tree (3.4)
Observed	Starts PowerShell to add a Defender exclusion for its own file	Process tree command line (3.4)
Observed	A second copy of the file starts the same second as PowerShell	Process tree (3.4)
Observed	Registry traces of execution (UserAssist, Compatibility Assistant)	Regshot (3.6)
Not observed	Run or RunOnce persistence; new files in Temp, Startup, Common Startup or Roaming	Regshot and manual checks (3.6, 3.7)
Not confirmed	Network activity, a second stage in memory, scheduled tasks, services, HKLM Run keys, browser credential access, success of the Defender exclusion	Not checked (3.4, 3.5)

Limits. Regshot compared the registry only; its file-system scan was switched off, so I checked files by hand. I kept the VM running for about an hour but did not step through what the sample did over that time. All of the tools agree on the date, 10/17/2026, but the clocks do not match on the time of day: the process tree shows the sample starting at about 3:29 AM, while Regshot’s first shot is stamped 09:33 and the taskbar clock reads about 10:3x AM. This is probably a time-zone display difference, so I used the order of events inside each tool and did not compare absolute times between tools.

3.9 Effect on the Earlier Analysis

The PowerShell command adds two techniques to the ATT&CK mapping in 2.8: PowerShell (T1059.001) and Disable or Modify Tools (shown in Navigator's Defense Impairment column; T1685). Persistence is still not confirmed. The loader hypothesis from Section 2 is partly supported, since the sample launches its own helper process, but I have not yet seen the hidden second stage, so the claim that this is Agent Tesla remains unconfirmed.

3.10 Lessons Learned

I reverted the VM before I had collected everything and so I could not return or repeat the run. The results in this section came from a single run.

What I would have done differently is to maybe take a snapshot at every stage of my running and again after the sample run without reverting, tick the Regshot's directory scan so files are compared as well as the registry. Clear Procmon and be sure that all the tools' clocks agree before the run.

CONCLUSION

The evidence supports a .NET loader that hides data in large image resources, disguises itself as a farm-management app, and tries to exclude itself from Defender. It does not yet support the claim that the payload is Agent Tesla. The family tag, the second stage, the network behavior and any persistence remain unconfirmed, and whether the hidden data is true steganography or raw pixel data is untested.